

# Mémoire de Master

Pour l'obtention du diplôme de Master en Business Intelligence  
and Big Data Analytics

Sur le sujet de :

Gestion de l'hétérogénéité des  
données tabulaires pour la gé-  
nération de Knowledge Graphs  
(Big Data + KG)

*Par :*

**CHOUFA Zouhair**

Soutenu le : 10-07-2026 , devant le jury composé de :

**Pr. A. MADANI**  
**Pr.**  
**Pr.**

Faculté des Sciences - El Jadida  
Faculté des Sciences - El Jadida  
Faculté des Sciences - El Jadida

Encadrant  
Examinateur  
Examinateur

Année universitaire : 2025-2026

## DÉDICACE

---

*À mes très chers parents,*

dont l'amour inconditionnel, les sacrifices silencieux et la confiance jamais démentie ont tracé, bien avant moi, le chemin de chacune de mes réussites. Que ce modeste travail soit le reflet, même imparfait, de tout ce que je vous dois.

*À mes frères et sœurs,*

pour votre soutien constant, vos encouragements dans les moments de doute, et la complicité qui nous unit depuis toujours.

*À mes amis et collègues de la promotion BIBDA,*

compagnons de cette aventure académique exigeante, avec qui chaque nuit blanche de débogage et chaque réussite ont pris un sens partagé.

*À tous ceux qui croient qu'une donnée brute, aussi hétérogène soit-elle, peut devenir connaissance structurée pour peu qu'on lui consacre la rigueur et la patience nécessaires,*

ce mémoire vous est dédié.



*“Les données ne parlent pas d'elles-mêmes ;  
c'est la structure qu'on leur donne qui les fait parler.”*

# REMERCIEMENTS

---

Au terme de ce travail, il est adressé une gratitude sincère à toutes les personnes ayant contribué, directement ou indirectement, à la réalisation de ce mémoire.

Les remerciements les plus sincères sont exprimés au **Professeur A. MADANI**, encadrant de ce travail à la Faculté des Sciences d'El Jadida, pour la qualité de son encadrement, sa disponibilité constante et la pertinence de ses conseils tout au long de ce projet. Sa rigueur scientifique et la confiance accordée dans la conduite de ce sujet de recherche ont permis de mener cette étude dans les meilleures conditions.

Les membres du jury sont également remerciés pour l'honneur qu'ils font en évaluant ce travail. Leur lecture attentive et leurs remarques constitueront un apport précieux à la poursuite de cette réflexion scientifique.

Une reconnaissance particulière est adressée au corps professoral et administratif du **Département d'Informatique** de la Faculté des Sciences d'El Jadida, ainsi qu'à l'**Université Chouaib Doukkali**, pour la qualité de la formation dispensée durant ce Master, dont les enseignements théoriques et pratiques ont constitué le socle de ce travail.

Les camarades de la promotion BIBDA sont remerciés pour l'esprit d'entraide et les échanges techniques qui ont accompagné ce parcours, en particulier durant les phases les plus intenses de développement et de rédaction.

Une pensée est également adressée à la communauté scientifique et technique open source — notamment les contributeurs des projets **Morph-KGC**, **RDFLib**, **Splink**, **Sentence-Transformers** et **LangChain** — dont les outils ont rendu possible l'implémentation du pipeline présenté dans ce mémoire.

Enfin, les remerciements les plus profonds vont à la famille, pour son soutien indéfectible, ses encouragements constants et les sacrifices consentis tout au long de ce parcours académique. Sans ce soutien, ce mémoire n'aurait pu voir le jour.

# Résumé

La dimension « Variété » du Big Data se traduit, en contexte industriel, par la coexistence de multiples sources de données tabulaires hétérogènes (formats, schémas, conventions de nommage), rendant leur intégration sémantique difficile à automatiser à grande échelle. Les Knowledge Graphs (KG), fondés sur le modèle RDF et les standards du Web sémantique, offrent un cadre formel pour unifier ces sources autour d’une ontologie commune ; toutefois, les pipelines de construction existants restent le plus souvent fragmentés, partiellement manuels, ou insuffisamment évalués de bout en bout.

Ce mémoire propose un pipeline complet, modulaire et reproductible de transformation de données tabulaires hétérogènes en Knowledge Graph. L’architecture proposée s’articule en quatre modules : (i) ingestion multi-source et profilage automatique de la qualité des données ; (ii) *schema matching* hybride combinant une approche symbolique par embeddings Sentence-BERT et un agent LLM (LLaMA 3.1 via Groq, orchestré par LangChain), suivi de la génération automatique de règles de mapping YARRRML ; (iii) triplification sémantique déclarative des données au moyen du moteur Morph-KGC ; (iv) résolution d’entités inter-sources par le modèle probabiliste de Fellegi-Sunter (bibliothèque Splink), complétée par une validation formelle du graphe produit via des contraintes SHACL. L’ensemble du pipeline est conteneurisé avec Docker Compose afin de garantir sa reproductibilité et son industrialisation potentielle.

L’approche a été évaluée sur un gold standard annoté, construit à partir d’un jeu de données transactionnel augmenté artificiellement (**Faker**) et scindé en deux sources hétérogènes simulant des silos organisationnels distincts. Les résultats montrent que l’architecture hybride de *schema matching* atteint un F1-score de 0,870 après calibration du seuil de décision ( $\tau^* = 0,82$ ), égalant ou surpassant chacune de ses composantes utilisée isolément tout en maîtrisant le coût des appels LLM. Le module de triplification génère 82 420 triplets RDF en seulement 7 secondes, tandis que la résolution d’entités, évaluée sur 1 600 paires réelles, atteint un F1-score de 0,990 après recalibrage des règles de blocage ( $\lambda^* = 0,90$ ). Une discussion critique structurée autour des menaces classiques à la validité expérimentale (interne, externe, de construit et de conclusion) complète cette évaluation et identifie les limites du travail, notamment le caractère synthétique du jeu de données et la sensibilité du modèle de résolution d’entités à la configuration des règles de blocage.

Ce travail démontre ainsi la faisabilité d’un pipeline hybride symbolique-neuronal pour la construction automatisée et évaluable de Knowledge Graphs à partir de données tabulaires hétérogènes, et ouvre plusieurs perspectives : validation sur des données réelles anonymisées, définition automatique de clés de blocage adaptatives, et substitution des appels LLM distants par des modèles open source déployés localement afin de réduire la latence à grande échelle.

**Mots-clés :** Big Data, Knowledge Graph, hétérogénéité des données, *schema matching*, Large Language Models, RDF, YARRRML, Morph-KGC, résolution d’entités, SHACL, reproductibilité.

# Abstract

The "Variety" dimension of Big Data manifests, in industrial settings, as the coexistence of multiple heterogeneous tabular data sources differing in format, schema, and naming conventions, making their semantic integration difficult to automate at scale. Knowledge Graphs (KGs), built upon the RDF model and Semantic Web standards, provide a formal framework for unifying such sources around a shared ontology; however, existing construction pipelines remain, for the most part, fragmented, partially manual, or insufficiently evaluated end-to-end.

This thesis proposes a complete, modular, and reproducible pipeline for transforming heterogeneous tabular data into a Knowledge Graph. The proposed architecture is organized into four modules : (i) multi-source ingestion and automated data quality profiling; (ii) a hybrid schema matching approach combining a symbolic Sentence-BERT embedding method with an LLM agent (LLaMA 3.1 via Groq, orchestrated through LangChain), followed by the automatic generation of YARRRML mapping rules; (iii) declarative semantic triplification of the data using the Morph-KGC engine; (iv) cross-source entity resolution based on the Fellegi-Sunter probabilistic model (Splink library), complemented by a formal validation of the resulting graph through SHACL constraints. The entire pipeline is containerized with Docker Compose to ensure reproducibility and potential industrialization.

The approach was evaluated on an annotated gold standard, built from a transactional dataset artificially augmented via **Faker** and split into two heterogeneous sources simulating distinct organizational silos. Results show that the hybrid schema matching architecture achieves an F1-score of 0.870 after calibrating the decision threshold ( $\tau^* = 0.82$ ), matching or outperforming each of its components used in isolation while keeping LLM API costs under control. The triplification module generates 82,420 RDF triples in only 7 seconds, while entity resolution, evaluated on 1,600 real pairs, reaches an F1-score of 0.990 after recalibrating the blocking rules ( $\lambda^* = 0.90$ ). A critical discussion structured around the classical threats to experimental validity (internal, external, construct, and conclusion validity) complements this evaluation and identifies the work's limitations, notably the synthetic nature of the dataset and the sensitivity of the entity resolution model to blocking rule configuration.

This work thus demonstrates the feasibility of a hybrid symbolic-neural pipeline for the automated and evaluable construction of Knowledge Graphs from heterogeneous tabular data, and opens several avenues for future work : validation on real anonymized data, automatic definition of adaptive blocking keys, and the substitution of remote LLM calls with locally deployed open-source models to reduce latency at scale.

**Keywords** : Big Data, Knowledge Graph, data heterogeneity, schema matching, Large Language Models, RDF, YARRRML, Morph-KGC, entity resolution, SHACL, reproducibility.

# Table des matières

|                                                                              |           |
|------------------------------------------------------------------------------|-----------|
| <b>Remerciements</b>                                                         | <b>2</b>  |
| <b>Résumé</b>                                                                | <b>3</b>  |
| <b>Abstract</b>                                                              | <b>4</b>  |
| <b>Liste des Abréviations</b>                                                | <b>12</b> |
| <b>Introduction Générale</b>                                                 | <b>15</b> |
| Contexte général . . . . .                                                   | 15        |
| Problématique . . . . .                                                      | 15        |
| Objectifs du mémoire . . . . .                                               | 16        |
| Contributions . . . . .                                                      | 16        |
| Organisation du mémoire . . . . .                                            | 17        |
| <b>1 État de l’art</b>                                                       | <b>18</b> |
| 1.1 Introduction . . . . .                                                   | 18        |
| 1.2 Le Paradigme Big Data et les Défis de l’Intégration . . . . .            | 19        |
| 1.2.1 Les dimensions du Big Data . . . . .                                   | 19        |
| 1.2.2 La Variété : source directe de l’hétérogénéité . . . . .               | 19        |
| 1.2.3 Scalabilité et besoins architecturaux . . . . .                        | 20        |
| 1.3 Introduction aux Knowledge Graphs . . . . .                              | 20        |
| 1.3.1 Définition et fondements . . . . .                                     | 20        |
| 1.3.2 Le modèle RDF et le rôle des URIs . . . . .                            | 20        |
| 1.3.3 Les composants du Web Sémantique . . . . .                             | 21        |
| 1.4 Construction Automatique et Intégration de Données Hétérogènes . . . . . | 22        |
| 1.4.1 Formalisation du problème . . . . .                                    | 22        |
| 1.4.2 Pipelines d’entreprise . . . . .                                       | 22        |
| 1.4.3 Taxonomie de l’hétérogénéité . . . . .                                 | 23        |
| 1.4.4 Intégration multi-sources et approches récentes . . . . .              | 24        |
| 1.5 Transformation de Données Tabulaires vers RDF . . . . .                  | 24        |
| 1.5.1 Les standards W3C : R2RML et RML . . . . .                             | 24        |
| 1.5.2 Morph-KGC et les vues RML . . . . .                                    | 25        |
| 1.5.3 YARRRML et les approches déclaratives . . . . .                        | 26        |

|          |                                                                     |           |
|----------|---------------------------------------------------------------------|-----------|
| 1.5.4    | SemTab : le banc d'essai communautaire . . . . .                    | 26        |
| 1.6      | Schema Matching et Ontology Alignment . . . . .                     | 26        |
| 1.6.1    | Définition et enjeux . . . . .                                      | 26        |
| 1.6.2    | Évolution des approches . . . . .                                   | 26        |
| 1.6.3    | La rupture paradigmatique des embeddings . . . . .                  | 27        |
| 1.6.4    | L'Ontology Alignment Evaluation Initiative (OAEI) . . . . .         | 27        |
| 1.7      | L'Apport des Large Language Models . . . . .                        | 28        |
| 1.7.1    | Panorama général . . . . .                                          | 28        |
| 1.7.2    | LLMatch : schema matching par LLMs . . . . .                        | 28        |
| 1.7.3    | SCHEMORA : alignement par prompt engineering . . . . .              | 29        |
| 1.7.4    | iText2KG : construction incrémentale de KGs . . . . .               | 29        |
| 1.7.5    | Limites communes aux approches LLM . . . . .                        | 30        |
| 1.8      | Défis et Limites des Approches Existantes . . . . .                 | 30        |
| 1.9      | Positionnement de notre Travail . . . . .                           | 31        |
| 1.10     | Conclusion . . . . .                                                | 31        |
| <b>2</b> | <b>Méthodologie et Conception de l'Architecture</b>                 | <b>33</b> |
| 2.1      | Introduction et Justification des Choix Architecturaux . . . . .    | 33        |
| 2.2      | Architecture Globale du Pipeline . . . . .                          | 34        |
| 2.3      | Module 1 : Ingestion et Prétraitement des Données . . . . .         | 36        |
| 2.3.1    | Rôle fonctionnel . . . . .                                          | 36        |
| 2.3.2    | Sources supportées et stratégie de scalabilité . . . . .            | 36        |
| 2.3.3    | Profilage automatique des données . . . . .                         | 37        |
| 2.3.4    | Staging et Mapping Store dans MongoDB . . . . .                     | 37        |
| 2.3.5    | Normalisation syntaxique . . . . .                                  | 38        |
| 2.4      | Module 2 : Schema Matching Hybride et Génération YARRRML . . . . .  | 39        |
| 2.4.1    | Représentation des colonnes sources . . . . .                       | 39        |
| 2.4.2    | L'ontologie du domaine : artefact central du pipeline . . . . .     | 39        |
| 2.4.3    | Approche symbolique : embeddings Sentence-BERT . . . . .            | 40        |
| 2.4.4    | Approche neuronale : agent LLM via LangChain . . . . .              | 41        |
| 2.4.5    | Architecture hybride et seuil adaptatif . . . . .                   | 43        |
| 2.4.6    | Génération automatique des règles YARRRML . . . . .                 | 43        |
| 2.5      | Module 3 : Triplification Sémantique avec Morph-KGC . . . . .       | 44        |
| 2.5.1    | Rôle fonctionnel . . . . .                                          | 44        |
| 2.5.2    | Justification du choix de Morph-KGC . . . . .                       | 44        |
| 2.5.3    | Processus d'exécution . . . . .                                     | 45        |
| 2.5.4    | Gestion des graphes nommés . . . . .                                | 45        |
| 2.6      | Module 4 : Résolution d'Entités et Validation SHACL . . . . .       | 45        |
| 2.6.1    | Phase 1 — Résolution d'entités avec Splink . . . . .                | 45        |
| 2.6.2    | Phase 2 — Validation SHACL avec PySHACL . . . . .                   | 46        |
| 2.7      | Orchestration et Reproductibilité : Docker et Big Data . . . . .    | 47        |
| 2.7.1    | Motivation : de la reproductibilité à l'industrialisation . . . . . | 47        |

|          |                                                                                   |           |
|----------|-----------------------------------------------------------------------------------|-----------|
| 2.7.2    | Architecture Docker Compose . . . . .                                             | 47        |
| 2.7.3    | Réseau interne et résolution DNS . . . . .                                        | 48        |
| 2.7.4    | Gestion des dépendances de démarrage . . . . .                                    | 48        |
| 2.7.5    | Persistance des données et volumes . . . . .                                      | 48        |
| 2.7.6    | Justification Big Data de la conteneurisation . . . . .                           | 48        |
| 2.8      | Protocole d'Évaluation et Constitution du Gold Standard . . . . .                 | 49        |
| 2.8.1    | Nécessité d'un gold standard annoté . . . . .                                     | 49        |
| 2.8.2    | Constitution du gold standard . . . . .                                           | 49        |
| 2.8.3    | Métriques d'évaluation par module . . . . .                                       | 49        |
| 2.9      | Conclusion du Chapitre . . . . .                                                  | 50        |
| <b>3</b> | <b>Implémentation et Réalisations</b>                                             | <b>51</b> |
| 3.1      | Introduction et Objectifs du Chapitre . . . . .                                   | 51        |
| 3.2      | Environnement de Travail et Outils . . . . .                                      | 52        |
| 3.2.1    | Plateforme Cloud : Lightning.ai . . . . .                                         | 52        |
| 3.2.2    | Orchestration : le script maître <code>run_pipeline.py</code> . . . . .           | 52        |
| 3.2.3    | Journalisation : <code>RotatingFileHandler</code> . . . . .                       | 52        |
| 3.3      | Préparation de la Vérité Terrain et <i>Data Augmentation</i> (Module 0) . . . . . | 53        |
| 3.3.1    | Nécessité d'une vérité terrain contrôlée . . . . .                                | 53        |
| 3.3.2    | Base d'origine : <code>online_retail_II.csv</code> . . . . .                      | 54        |
| 3.3.3    | <i>Data Augmentation</i> via la librairie <code>Faker</code> . . . . .            | 54        |
| 3.3.4    | Simulation des silos : scission en deux sources hétérogènes . . . . .             | 54        |
| 3.4      | Module 1 : Ingestion et Profilage Qualité . . . . .                               | 57        |
| 3.4.1    | Chargement multi-format et normalisation syntaxique . . . . .                     | 57        |
| 3.4.2    | Audit automatique de qualité : <code>ydata-profiling</code> . . . . .             | 57        |
| 3.5      | Module 2 : Schema Matching Hybride (IA Locale + LLM) . . . . .                    | 58        |
| 3.5.1    | Architecture hybride et stratégie <i>FinOps</i> . . . . .                         | 58        |
| 3.5.2    | Étape 1 : similarité vectorielle par Sentence-BERT . . . . .                      | 59        |
| 3.5.3    | Étape 2 : résolution des ambiguïtés par l'agent LLaMA 3.1 (Groq API) . . . . .    | 59        |
| 3.5.4    | Résultats : 7 correspondances validées, 1 rejet justifié . . . . .                | 60        |
| 3.6      | Résultats de l'Étude d'Ablation du Module 2 . . . . .                             | 61        |
| 3.6.1    | Variante A — SBERT seul . . . . .                                                 | 62        |
| 3.6.2    | Variante B — LLM seul . . . . .                                                   | 62        |
| 3.6.3    | Variante C — Architecture hybride SBERT + LLM . . . . .                           | 62        |
| 3.6.4    | Synthèse comparative . . . . .                                                    | 63        |
| 3.7      | Module 3 : Triplification Sémantique (De Tabulaire vers RDF) . . . . .            | 64        |
| 3.7.1    | Génération automatique des règles YARRRML . . . . .                               | 64        |
| 3.7.2    | Le moteur Morph-KGC et la stratégie mémoire . . . . .                             | 64        |
| 3.7.3    | Résultats : 82 420 triplets RDF générés en 7 secondes . . . . .                   | 65        |
| 3.8      | Module 4 : Résolution d'Entités et Graphe Unifié . . . . .                        | 66        |
| 3.8.1    | Modèle probabiliste de Fellegi-Sunter et backend DuckDB . . . . .                 | 66        |
| 3.8.2    | Apprentissage non-supervisé par Expectation-Maximization . . . . .                | 67        |



|          |                                                                                    |           |
|----------|------------------------------------------------------------------------------------|-----------|
| 3.8.3    | Analyse de la sensibilité du seuil : un cas d'école Fellegi-Sunter . . .           | 67        |
| 3.8.4    | Génération des ponts sémantiques owl:sameAs . . . . .                              | 67        |
| 3.9      | Déploiement et Visualisation . . . . .                                             | 68        |
| 3.9.1    | Stockage dans Ontotext GraphDB . . . . .                                           | 68        |
| 3.9.2    | Visualisation interactive : PyVis . . . . .                                        | 69        |
| 3.10     | Bilan des Performances et Analyse Critique . . . . .                               | 70        |
| 3.10.1   | Récapitulatif des temps d'exécution de bout en bout . . . . .                      | 70        |
| 3.10.2   | Limites identifiées et perspectives . . . . .                                      | 71        |
| 3.11     | Conclusion du Chapitre . . . . .                                                   | 72        |
| <b>4</b> | <b>Évaluation et Discussion des Résultats</b>                                      | <b>73</b> |
| 4.1      | Introduction et Objectifs du Chapitre . . . . .                                    | 73        |
| 4.2      | Synthèse Globale des Performances par Module . . . . .                             | 73        |
| 4.3      | Calibration et Analyse de Sensibilité du Seuil $\tau$ (Module 2) . . . . .         | 74        |
| 4.3.1    | Rappel du rôle du seuil $\tau$ . . . . .                                           | 74        |
| 4.3.2    | Protocole de calibration . . . . .                                                 | 74        |
| 4.3.3    | Interprétation de la courbe . . . . .                                              | 75        |
| 4.4      | Recalibrage des Règles de Blocage et Sensibilité du Seuil $\lambda$ (Module 4) . . | 76        |
| 4.4.1    | Du constat initial au recalibrage . . . . .                                        | 76        |
| 4.4.2    | Sensibilité du seuil $\lambda$ après recalibrage . . . . .                         | 76        |
| 4.4.3    | Discussion : une leçon méthodologique généralisable . . . . .                      | 77        |
| 4.5      | Positionnement par Rapport à l'État de l'Art . . . . .                             | 77        |
| 4.6      | Discussion Critique et Menaces à la Validité . . . . .                             | 78        |
| 4.6.1    | Validité de construit . . . . .                                                    | 78        |
| 4.6.2    | Validité interne . . . . .                                                         | 78        |
| 4.6.3    | Validité externe . . . . .                                                         | 79        |
| 4.6.4    | Validité de conclusion . . . . .                                                   | 79        |
| 4.7      | Conclusion du Chapitre . . . . .                                                   | 79        |
|          | <b>Conclusion Générale</b>                                                         | <b>81</b> |
|          | Synthèse du travail réalisé . . . . .                                              | 81        |
|          | Principaux résultats obtenus . . . . .                                             | 81        |
|          | Limites du travail . . . . .                                                       | 82        |
|          | Perspectives de recherche . . . . .                                                | 82        |

# Table des figures

|     |                                                                                                                                                                                                                                                                                                                                     |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Les quatre dimensions du Big Data et la place centrale de la Variété dans la problématique d'intégration. . . . .                                                                                                                                                                                                                   | 19 |
| 1.2 | Passage d'un schéma relationnel fragmenté vers un Knowledge Graph unifié par des URIs communes : illustration du principe d'unification sémantique. . . . .                                                                                                                                                                         | 21 |
| 1.3 | Pipeline de transformation de sources tabulaires hétérogènes vers des triplets RDF via le moteur Morph-KGC et le standard RML. . . . .                                                                                                                                                                                              | 25 |
| 1.4 | Évolution des approches de schema matching : des méthodes linguistiques aux Large Language Models. . . . .                                                                                                                                                                                                                          | 27 |
| 1.5 | Comparaison des frameworks LLM pour la construction de KGs : protocoles, datasets et métriques. . . . .                                                                                                                                                                                                                             | 30 |
| 2.1 | Architecture globale du pipeline de transformation structurée en deux phases. La première ligne illustre l'ingestion et l'alignement sémantique hybride. La seconde ligne détaille la génération RDF, la résolution d'entités et le stockage final. . . . .                                                                         | 35 |
| 2.2 | Flux de décision du module de schema matching hybride. Le seuil $\tau$ est calibré empiriquement sur le gold standard pour maximiser le F1-score global du module. . . . .                                                                                                                                                          | 43 |
| 3.1 | Processus de <i>data augmentation</i> du Module 0 : du dataset transactionnel <code>online_retail_II.csv</code> vers les deux sources hétérogènes simulant les silos CRM (4 000 lignes, CSV) et SIRH (3 000 lignes, Excel), avec un chevauchement de 1 600 individus communs constituant le gold standard. . . . .                  | 56 |
| 3.2 | Extrait du rapport de profilage <code>ydata-profiling</code> généré pour la Source A (Export CRM, 4 000 lignes). Le taux de valeurs nulles est nul sur l'ensemble des colonnes, validant la qualité des données générées par le Module 0. . . . .                                                                                   | 58 |
| 3.3 | Flux de décision du Module 2 ( <i>Lazy Loading</i> ) : pour les 8 colonnes analysées, SBERT a détecté une ambiguïté (score $< 0,82$ ) sur l'intégralité d'entre elles, déclenchant le recours à l'agent LLaMA 3.1 (Groq API). L'agent a retourné 7 correspondances validées et 1 rejet justifié ( <code>UNMATCHED</code> ). . . . . | 61 |
| 3.4 | Comparaison des trois variantes de l'étude d'ablation du Module 2 (Source B, 8 colonnes) sur les métriques de précision, rappel et F1-score. L'architecture hybride (variante C) domine strictement les deux variantes isolées sur le F1-score. . . . .                                                                             | 64 |

|     |                                                                                                                                                                                                                                                                                                                                                                                 |    |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 3.5 | Flux d'exécution du Module 3 : du dictionnaire JSON de mapping vers les 82420 triplets RDF N-Triples, via la génération automatique des règles YARRRML et le moteur Morph-KGC. La conversion à la volée Excel → CSV de la Source B (3 000 lignes) est mise en évidence comme optimisation mémoire. . . . .                                                                      | 66 |
| 3.6 | Illustration de la structure en « nœud papillon » ( <i>butterfly node</i> ) produite par le Module 4 : un triplet <code>owl:sameAs</code> unit l'entité CRM (CUST-1234) et l'entité SIRH (EMP-1234). Les arêtes colorées représentent les triplets de propriétés issus de chaque source ; les arêtes rouges centrales matérialisent les ponts de réconciliation Splink. . . . . | 68 |
| 3.7 | Visualisation PyVis du Knowledge Graph final : vue centrée sur un cluster de réconciliation. Les nœuds bleus (Source A, CRM) et orange (Source B, SIRH) sont reliés par des arêtes rouges <code>owl:sameAs</code> générées par Splink. Le fichier HTML interactif est exporté dans <code>outputs/visualization/kg_graph.html</code> . . . . .                                   | 70 |
| 4.1 | Courbe de calibration du seuil $\tau$ (variante SBERT-avec-rejet) sur le gold standard. Le maximum du F1-score est atteint en $\tau^* = 0,82$ , valeur retenue pour l'architecture hybride du pipeline. . . . .                                                                                                                                                                 | 75 |

# Liste des tableaux

|     |                                                                                                                                                                                                                                                                      |    |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Taxonomie des formes d'hétérogénéité avec exemples concrets. . . . .                                                                                                                                                                                                 | 23 |
| 1.2 | Positionnement de notre travail par rapport aux contributions existantes. .                                                                                                                                                                                          | 32 |
| 2.1 | Artefacts d'entrée et de sortie de chaque module du pipeline. . . . .                                                                                                                                                                                                | 36 |
| 3.1 | Stack technologique consolidée du pipeline — versions et rôles fonctionnels.                                                                                                                                                                                         | 53 |
| 3.2 | Schémas comparatifs des deux sources simulées et types d'hétérogénéité<br>correspondants. . . . .                                                                                                                                                                    | 55 |
| 3.3 | Résultats du profilage qualité du Module 1 sur les deux sources de l'éva-<br>luation. . . . .                                                                                                                                                                        | 58 |
| 3.4 | Résultats du schema matching hybride (Module 2) pour les 8 colonnes<br>analysées. Score de confiance de l'agent LLaMA 3.1 retenu pour chaque<br>correspondance. . . . .                                                                                              | 60 |
| 3.5 | Résultats de l'étude d'ablation du Module 2 sur la Source B (SIRH, 8 co-<br>lonnes). Évaluation sur gold standard contrôlé. TP/FP/FN/TN : vrais po-<br>sitifs, faux positifs, faux négatifs, vrais négatifs. ✓ : variante retenue dans<br>le pipeline final. . . . . | 63 |
| 3.6 | Métriques de triplification produites par Morph-KGC (Module 3). . . . .                                                                                                                                                                                              | 65 |
| 3.7 | Récapitulatif des temps d'exécution du pipeline de bout en bout (46,35 se-<br>condes au total). . . . .                                                                                                                                                              | 71 |
| 4.1 | Synthèse des métriques de performance par module du pipeline. F1-score<br>rapporté pour les modules 2 et 4 (évaluation contre gold standard) ; taux<br>de conformité pour le Module 3. . . . .                                                                       | 74 |
| 4.2 | F1-score de la variante SBERT-avec-rejet en fonction du seuil $\tau$ , sur les<br>8 colonnes du gold standard. . . . .                                                                                                                                               | 75 |
| 4.3 | Sensibilité du seuil de décision $\lambda$ du Module 4 après recalibrage de la règle<br>de blocage, évaluée contre la vérité terrain de 1 600 paires connues. . . . .                                                                                                | 76 |
| 4.4 | Positionnement qualitatif du pipeline proposé par rapport aux grandes fa-<br>milles de travaux de l'état de l'art. ✓ : critère satisfait ; ~ : satisfait par-<br>tiellement ; × : non traité. . . . .                                                                | 77 |

# Liste des Abréviations

| Abréviation | Signification                                                                                 |
|-------------|-----------------------------------------------------------------------------------------------|
| API         | Application Programming Interface (interface de programmation applicative)                    |
| BD          | Big Data                                                                                      |
| BIBDA       | Business Intelligence and Big Data Analytics (intitulé du Master)                             |
| CLI         | Command Line Interface (interface en ligne de commande)                                       |
| CRM         | Customer Relationship Management (gestion de la relation client)                              |
| CSV         | Comma-Separated Values (valeurs séparées par des virgules)                                    |
| DL          | Deep Learning (apprentissage profond)                                                         |
| EM          | Expectation-Maximization (algorithme d'espérance-maximisation)                                |
| FOAF        | Friend Of A Friend (vocabulaire RDF de description de réseaux sociaux)                        |
| GPT         | Generative Pre-trained Transformer                                                            |
| GraphDB     | Triplestore RDF développé par Ontotext, utilisé pour le stockage du KG produit                |
| IA          | Intelligence Artificielle                                                                     |
| ISO         | International Organization for Standardization (organisation internationale de normalisation) |
| JSON        | JavaScript Object Notation (format d'échange de données)                                      |
| KG          | Knowledge Graph (graphe de connaissances)                                                     |
| KGs         | Knowledge Graphs (pluriel)                                                                    |
| LangChain   | Framework open source d'orchestration d'agents et de <i>prompts</i> pour LLMs                 |
| LLaMA       | Large Language Model Meta AI (famille de modèles de langage de Meta)                          |
| LLM         | Large Language Model (grand modèle de langage)                                                |
| LLMs        | Large Language Models (pluriel)                                                               |
| LLMatch     | Méthode de <i>schema matching</i> fondée sur les LLMs (littérature)                           |

(suite sur la page suivante)

(suite de la page précédente)

| <b>Abréviation</b> | <b>Signification</b>                                                                                       |
|--------------------|------------------------------------------------------------------------------------------------------------|
| Morph-KGC          | Moteur de triplification déclarative RML/YARRRML utilisé au Module 3 du pipeline                           |
| NLP                | Natural Language Processing (traitement automatique du langage naturel)                                    |
| OAEI               | Ontology Alignment Evaluation Initiative (initiative d'évaluation de l'alignement d'ontologies)            |
| OWL                | Web Ontology Language (langage d'ontologies du Web sémantique)                                             |
| PySHACL            | Bibliothèque Python d'implémentation du langage de contraintes SHACL                                       |
| PyVis              | Bibliothèque Python de visualisation interactive de graphes                                                |
| R2RML              | RDB to RDF Mapping Language (standard W3C de mapping base relationnelle vers RDF)                          |
| RDB                | Relational DataBase (base de données relationnelle)                                                        |
| RDF                | Resource Description Framework (modèle de données du Web sémantique)                                       |
| RDFLib             | Bibliothèque Python de manipulation de graphes RDF                                                         |
| REST               | Representational State Transfer (style d'architecture d'API Web)                                           |
| RML                | RDF Mapping Language (standard W3C de mapping générique vers RDF)                                          |
| SBERT              | Sentence-BERT (modèle d'embeddings de phrases fondé sur BERT)                                              |
| SCHEMORA           | Méthode d'alignement de schémas par <i>prompt engineering</i> (littérature)                                |
| SemTab             | Semantic Table Interpretation Challenge (banc d'essai communautaire d'interprétation sémantique de tables) |
| SHACL              | Shapes Constraint Language (langage W3C de validation de graphes RDF)                                      |
| SIRH               | Système d'Information des Ressources Humaines                                                              |
| SPARQL             | SPARQL Protocol And RDF Query Language (langage de requête pour RDF)                                       |
| SQL                | Structured Query Language (langage de requête structuré)                                                   |
| SQLite             | Système de gestion de base de données relationnelle embarqué                                               |
| STI                | Semantic Table Interpretation (interprétation sémantique de tables)                                        |
| TF-IDF             | Term Frequency–Inverse Document Frequency (pondération statistique de termes)                              |

(suite sur la page suivante)

*(suite de la page précédente)*

| <b>Abréviation</b> | <b>Signification</b>                                                                       |
|--------------------|--------------------------------------------------------------------------------------------|
| URI                | Uniform Resource Identifier (identifiant uniforme de res-<br>source)                       |
| URIs               | Uniform Resource Identifiers (pluriel)                                                     |
| UTF-8              | Unicode Transformation Format – 8 bits (encodage de ca-<br>ractères)                       |
| W3C                | World Wide Web Consortium (organisme de standardisation<br>du Web)                         |
| XML                | eXtensible Markup Language (langage de balisage exten-<br>sible)                           |
| XSD                | XML Schema Definition (langage de définition de schémas<br>XML)                            |
| YAML               | YAML Ain't Markup Language (format de sérialisation de<br>données lisible)                 |
| YARRRML            | Yet Another RML Rule ML (langage déclaratif simplifié pour<br>la génération de règles RML) |
| $\tau$             | Seuil de décision du Module 2 (schema matching hybride)                                    |
| $\lambda$          | Seuil de décision du Module 4 (résolution d'entités, modèle<br>de Fellegi-Sunter)          |

# Introduction Générale

## Contexte général

L'ère du Big Data a profondément transformé la manière dont les organisations produisent, stockent et exploitent leurs données. Parmi les cinq dimensions classiquement associées à ce paradigme — Volume, Vitesse, Variété, Véracité et Valeur —, la **Variété** constitue sans doute le défi le plus structurel pour les entreprises modernes. Une organisation typique dispose rarement d'une source de données unique et homogène : ses informations sont dispersées entre systèmes CRM, bases de données transactionnelles, fichiers plats CSV, systèmes d'information des ressources humaines (SIRH) ou tableurs, chacun obéissant à des conventions de nommage, des schémas et des formats qui lui sont propres. Cette fragmentation, souvent qualifiée de phénomène des *silos de données*, constitue un frein majeur à toute exploitation analytique transversale et cohérente de l'information disponible.

Les **Knowledge Graphs** (KG), fondés sur le modèle de données RDF (*Resource Description Framework*) et sur les standards du Web sémantique — OWL, SPARQL, SHACL —, se sont imposés au cours de la dernière décennie comme un cadre formel particulièrement adapté pour répondre à ce défi d'intégration. En représentant les entités et leurs relations sous forme de triplets sujet-prédicat-objet ancrés dans une ontologie de domaine partagée, les KG permettent de dépasser la rigidité du modèle relationnel classique et d'unifier sémantiquement des sources hétérogènes autour d'un vocabulaire commun. Cette approche a démontré sa valeur dans des contextes aussi variés que les moteurs de recherche, les systèmes de recommandation, la biomédecine ou l'intégration de données d'entreprise.

## Problématique

Si la valeur des Knowledge Graphs pour l'intégration de données n'est plus à démontrer, leur **construction automatisée** à partir de sources tabulaires hétérogènes demeure un problème ouvert et largement non résolu de façon industrialisable. La littérature existante propose des solutions ponctuelles pour chacune des étapes du processus — schema matching, triplification déclarative via les standards R2RML/RML, résolution d'entités, validation de la qualité du graphe produit — mais rarement de manière intégrée au sein d'un pipeline unique, reproductible et évaluable de bout en bout. De plus, l'émergence



récente des grands modèles de langage (*Large Language Models*, LLM) ouvre des perspectives prometteuses pour automatiser des tâches historiquement coûteuses en intervention humaine, telles que l’alignement sémantique de schémas hétérogènes, mais leur intégration dans des architectures de production robustes, maîtrisées en coût et rigoureusement évaluées, reste largement à explorer.

Ce mémoire s’inscrit directement dans ce contexte et cherche à répondre à la question de recherche suivante : *comment concevoir un pipeline automatisé, modulaire et reproductible, capable de transformer des données tabulaires hétérogènes et multi-sources en un Knowledge Graph unifié, tout en garantissant la qualité et la validité formelle du graphe produit à chaque étape du processus ?*

## Objectifs du mémoire

Pour répondre à cette problématique, ce travail poursuit quatre objectifs principaux :

- **Concevoir** une architecture de pipeline modulaire couvrant l’ensemble de la chaîne de transformation : ingestion et profilage qualité des sources, alignement sémantique des schémas, triplification RDF déclarative et résolution d’entités inter-sources ;
- **Combiner** de manière hybride des approches symboliques (embeddings Sentence-BERT) et neuronales fondées sur les LLM (agent LangChain) pour le *schema matching*, en visant à maximiser la qualité d’alignement tout en maîtrisant le coût computationnel des appels aux modèles de langage ;
- **Automatiser** la génération des règles de mapping YARRRML et leur exécution via le moteur Morph-KGC, ainsi que la validation formelle du graphe de connaissances produit au moyen de contraintes SHACL ;
- **Évaluer** quantitativement chaque module du pipeline sur un gold standard annoté, et discuter de manière critique la validité, les limites et la généralisabilité des résultats obtenus.

## Contributions

La contribution de ce mémoire se situe à l’intersection de trois axes de recherche généralement traités séparément dans la littérature : la triplification déclarative de données (RML/YARRRML), le *schema matching* hybride, et l’automatisation par LLM. Le pipeline proposé intègre ces trois dimensions au sein d’une architecture unique, entièrement conteneurisée via Docker Compose, et dont chaque module fait l’objet d’une évaluation quantitative indépendante sur un gold standard construit spécifiquement pour ce travail. Cette approche intégrée et évaluable de bout en bout constitue, à notre connaissance, une réponse originale aux lacunes identifiées dans l’état de l’art : fragmentation des pipelines existants, absence d’évaluation standardisée, reproductibilité insuffisante et déficit d’études portant sur la scalabilité.

# Organisation du mémoire

Ce mémoire est structuré en quatre chapitres. Le **Chapitre 1** dresse un état de l’art structuré autour du paradigme Big Data, des fondements des Knowledge Graphs, des approches de transformation tabulaire vers RDF et de *schema matching*, ainsi que de l’apport récent des LLM à ces problématiques ; il se conclut par le positionnement précis de notre contribution par rapport aux travaux existants. Le **Chapitre 2** présente la méthodologie et la conception détaillée de l’architecture du pipeline proposé, module par module, ainsi que le protocole d’évaluation et la constitution du gold standard. Le **Chapitre 3** décrit l’implémentation concrète de ce pipeline dans un environnement Cloud réel, les résultats obtenus pour chaque module, ainsi qu’une étude d’ablation isolant la contribution propre de chaque composant du module de *schema matching*. Enfin, le **Chapitre 4** propose une évaluation approfondie et comparative des résultats : calibration des hyperparamètres critiques du pipeline, positionnement par rapport à l’état de l’art, et discussion critique structurée autour des menaces classiques à la validité expérimentale. Une conclusion générale synthétise les apports de ce travail, en discute les limites et propose des perspectives de recherche future.

# Chapitre 1

## État de l’art

### 1.1 Introduction

Dans l’écosystème numérique contemporain, les organisations produisent et accumulent des données issues de sources toujours plus nombreuses et hétérogènes : fichiers tabulaires (CSV, Excel), bases de données relationnelles, flux JSON/XML, entrepôts analytiques, etc. L’intégration cohérente de ces sources constitue l’un des défis les plus critiques de l’ingénierie des données moderne, car elle conditionne directement la qualité des analyses et des décisions qui en découlent.

Face à cette complexité, les *Knowledge Graphs* (KGs) s’imposent comme une infrastructure sémantique particulièrement adaptée : en modélisant les données non plus comme des valeurs isolées dans des tables, mais comme un réseau d’entités reliées par des relations nommées, ils permettent une interopérabilité et un raisonnement automatique difficiles à atteindre par les approches relationnelles classiques. La transformation automatique de données tabulaires hétérogènes vers cette représentation sémantique soulève cependant des questions scientifiques encore largement ouvertes, notamment autour du *schema matching*, de la triplification déclarative, et du rôle croissant des Large Language Models (LLMs) dans ces pipelines.

Ce chapitre dresse un panorama structuré et critique des travaux existants dans ce domaine. Nous commençons par situer la problématique dans le contexte du Big Data (section 1.2), avant d’introduire les fondements des Knowledge Graphs (section 1.3). Nous analysons ensuite les approches de construction automatique et d’intégration de données hétérogènes (section 1.4), la transformation tabulaire vers RDF (section 1.5), le schema matching et l’alignement d’ontologies (section 1.6), et l’apport des LLMs (section 1.7). Nous concluons par une synthèse des limites identifiées (section 1.8) et un positionnement précis de notre contribution (section 1.9).

## 1.2 Le Paradigme Big Data et les Défis de l'Intégration

### 1.2.1 Les dimensions du Big Data

Le concept de Big Data est traditionnellement caractérisé par le modèle des « 4V », initialement introduit pour décrire les propriétés fondamentales des données à grande échelle : le **Volume** (la quantité de données produites, désormais mesurée en pétaoctets dans les grandes organisations), la **Vélocité** (la vitesse à laquelle les données sont générées et doivent être traitées), la **Véracité** (la fiabilité et la qualité des données collectées) et, surtout dans le cadre du présent travail, la **Variété**.

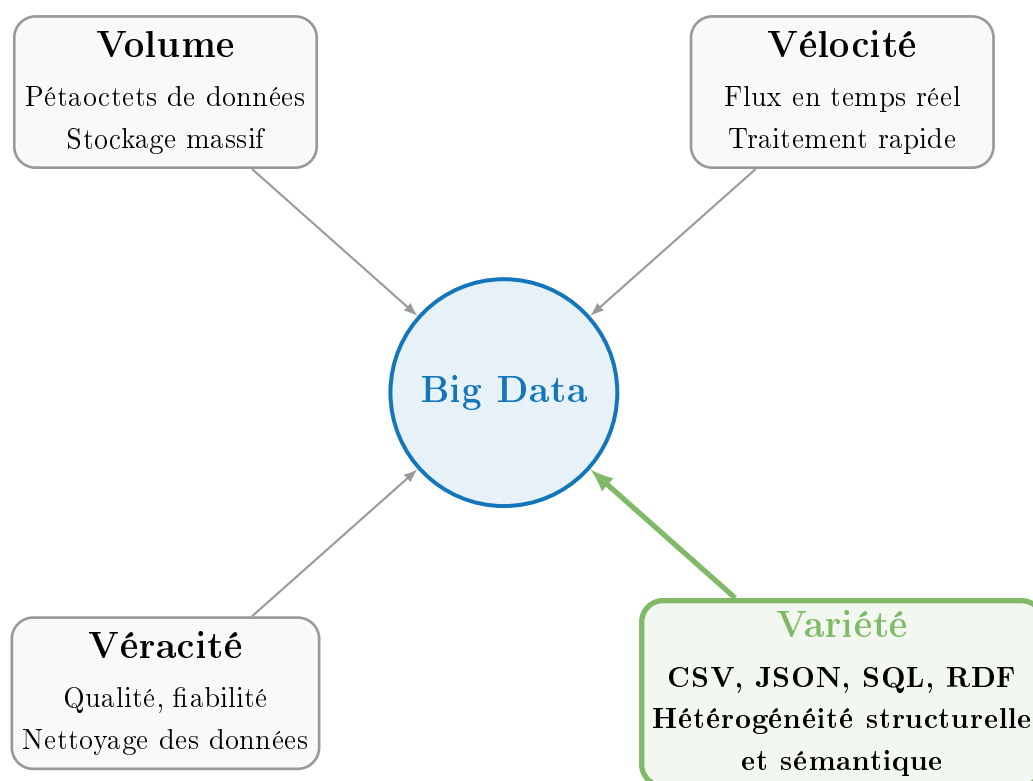


FIGURE 1.1 – Les quatre dimensions du Big Data et la place centrale de la Variété dans la problématique d'intégration.

### 1.2.2 La Variété : source directe de l'hétérogénéité

Parmi ces quatre dimensions, la Variété est celle qui génère le plus directement les défis traités dans ce mémoire. Dans un environnement industriel réel, une même entité métier — un client, un produit, une transaction — peut être représentée simultanément dans une feuille de calcul Excel (avec une colonne nommée `ID_Client`), dans une base de données relationnelle (avec un attribut `customer_id`), et dans un fichier JSON d'export tiers (avec une clé `buyer`). Ces trois représentations réfèrent au même concept, mais aucun mécanisme automatique standard ne permet de les réconcilier sans intervention humaine ou sans infrastructure sémantique dédiée.

### 1.2.3 Scalabilité et besoins architecturaux

Dans un contexte Big Data, la scalabilité des processus de transformation est cruciale : un pipeline qui fonctionne correctement sur quelques milliers de lignes doit rester opérationnel sur des millions d’enregistrements provenant de dizaines de sources simultanées. La littérature consultée indique que les approches manuelles ou semi-automatiques d’intégration atteignent rapidement leurs limites dans ces scénarios, justifiant le recours à des pipelines automatisés, déclaratifs et évaluables quantitativement [55, 56].

## 1.3 Introduction aux Knowledge Graphs

### 1.3.1 Définition et fondements

Avant d’aborder les techniques spécialisées, il est utile de comprendre intuitivement ce qui distingue un Knowledge Graph d’une base de données classique. Dans une base relationnelle, les données sont organisées en tables dont les colonnes définissent des attributs. La *sémantique* — c’est-à-dire le sens des données — reste implicite, dans la tête du développeur ou enfouie dans des commentaires de code. Un Knowledge Graph, au contraire, *externalise* ce sens : chaque entité, chaque relation et chaque attribut reçoit un identifiant universel dont la définition est consultable et exploitable par une machine.

Hogan et al. [21] proposent la définition de référence dans la littérature : un KG est un graphe orienté dans lequel les nœuds représentent des entités du monde réel et les arêtes des relations nommées entre ces entités, le tout formalisé selon les standards du Web Sémantique.

### 1.3.2 Le modèle RDF et le rôle des URIs

Le modèle de données fondamental des KGs est le triplet RDF (*Sujet, Prédicat, Objet*). Chaque ressource est identifiée par une URI (Uniform Resource Identifier) universelle.

**Exemple concret d’unification sémantique.** Considérons deux fichiers sources provenant de systèmes distincts. Le premier contient une colonne `ID_Client` ; le second, une colonne `Acheteur_Num`. Dans un schéma relationnel, ces deux colonnes sont invisibles l’une à l’autre, chacune enfermée dans son silo applicatif. Dans un KG, elles sont toutes deux mappées vers la même URI ontologique :

`<http://example.org/ontology#Customer>`

Ce mécanisme d’identification universelle est précisément ce qui permet de surmonter l’hétérogénéité sémantique entre sources [21].

**Anatomie d’un triplet RDF.** Concrètement, chaque cellule d’un tableau source devient un **triplet RDF** de la forme (Sujet, Prédicat, Objet). Par exemple, la ligne `ID_Client = 42, Nom = Dupont, Email = dupont@mail.com` produit les trois triplets suivants :

- `<Customer/42> rdf:type <ontology#Customer>`
- `<Customer/42> <ontology#hasName> "Dupont"^^xsd:string`
- `<Customer/42> <ontology#hasEmail> "dupont@mail.com"^^xsd:string`

Le **Sujet** (`<Customer/42>`) est l'entité décrite, identifiée par une URI unique et stable. Le **Prédicat** (`<ontology#hasName>`) est la propriété qui relie le sujet à sa valeur ; il est lui-même défini par une URI pointant vers l'ontologie du domaine. L'**Objet** peut être soit une autre URI (pour relier deux entités), soit un *littéral* typé (une valeur concrète avec son type XSD : `xsd:string`, `xsd:date`, `xsd:decimal`, etc.). C'est précisément grâce à ce typage explicite que les hétérogénéités syntaxiques — formats de dates, encodages de nombres, séparateurs décimaux — sont résolues lors de la transformation.

**Graphes nommés et traçabilité de la provenance.** Une extension importante du modèle RDF est le **quadruplet** (Sujet, Prédicat, Objet, Graphe), qui ajoute un quatrième composant identifiant le *graphe nommé* auquel appartient le triplet. Dans un pipeline multi-sources, cette extension permet de tracer l'origine de chaque triplet : un triplet provenant du fichier CRM et un triplet provenant de l'export RH sont stockés dans deux graphes nommés distincts, ce qui facilite l'audit de qualité et la résolution des conflits sémantiques.

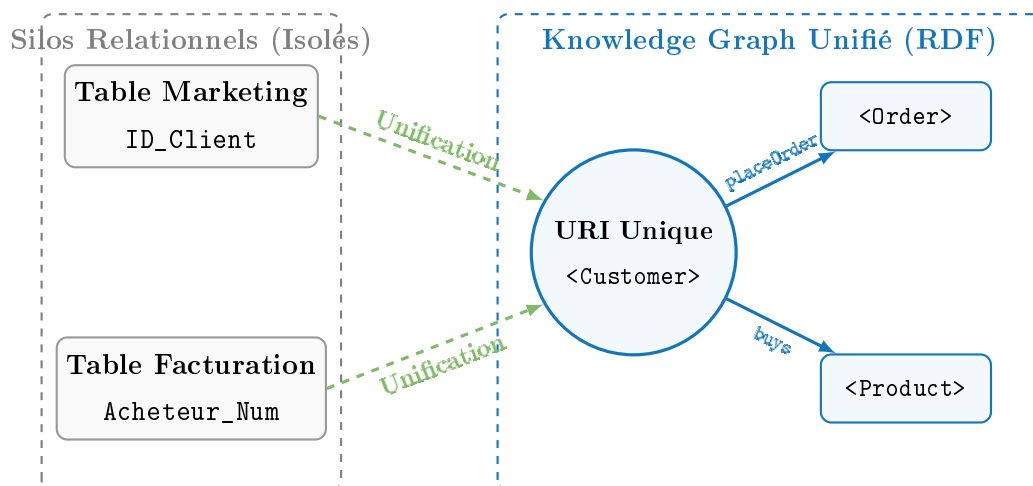


FIGURE 1.2 – Passage d'un schéma relationnel fragmenté vers un Knowledge Graph unifié par des URIs communes : illustration du principe d'unification sémantique.

### 1.3.3 Les composants du Web Sémantique

Au-delà du modèle RDF, l'écosystème du Web Sémantique propose une pile technologique complémentaire :

- **RDFS** (*RDF Schema*) permet de définir des hiérarchies de classes et de propriétés, ajoutant une première couche d'inférence.

- **OWL** (*Web Ontology Language*) fournit un langage expressif pour modéliser des contraintes logiques complexes et permettre un raisonnement automatique par inférence. Par exemple, une règle OWL peut déduire automatiquement que deux entités de sources différentes représentent le même individu réel.
- **SPARQL** est le langage de requête standardisé pour interroger les graphes RDF de manière expressive, analogue au SQL pour les bases relationnelles.
- **SHACL** (*Shapes Constraint Language*) permet de définir et de vérifier des contraintes de validation sur la structure d'un graphe RDF, garantissant la conformité du graphe produit à l'ontologie cible.

Ce cadre formel constitue le fondement théorique du pipeline de transformation que nous proposons dans ce travail.

## 1.4 Construction Automatique et Intégration de Données Hétérogènes

### 1.4.1 Formalisation du problème

La construction automatique de KGs à partir de sources hétérogènes est un défi central du domaine. Wu et al. [55] le formalisent à travers le théorème HACE (*Heterogeneous, Autonomous, Complex, Evolving*), qui caractérise les quatre dimensions problématiques des environnements Big Data. Leur survey de référence, publié dans *ACM Computing Surveys*, recense les méthodes d'extraction d'entités nommées (NER), d'extraction de relations et de fusion de connaissances, et souligne que chaque étape du pipeline introduit des erreurs cumulatives susceptibles de dégrader la qualité du graphe final. Cette observation sur l'*accumulation d'erreurs* constitue l'une des motivations majeures pour développer des pipelines intégrés plutôt que séquentiels.

### 1.4.2 Pipelines d'entreprise

Dans un contexte industriel, Yan et al. [56] proposent un cadre de processus complet pour la construction d'un KG combinant données structurées et non structurées. Leur pipeline couvre l'acquisition, le prétraitement, l'extraction d'information, la fusion et la mise à jour du KG. Les auteurs démontrent que la qualité du KG produit est directement liée à la rigueur du traitement de l'hétérogénéité en amont. Cette approche constitue un point de comparaison direct pour notre travail, bien qu'elle demeure partiellement manuelle dans sa phase d'alignement de schémas — limitation que notre approche vise précisément à dépasser.

Il convient de souligner une limitation méthodologique récurrente dans ce type de travaux : les pipelines proposés sont souvent décrits à un niveau d'abstraction élevé, sans environnement d'exécution normalisé ni mécanisme de validation formelle du graphe produit, ce qui nuit considérablement à leur reproductibilité [55].

### 1.4.3 Taxonomie de l'hétérogénéité

La littérature consultée converge vers une taxonomie tripartite bien documentée des formes d'hétérogénéité [1] :

1. **Hétérogénéité structurelle** : différences dans l'organisation des données entre sources (tables normalisées vs. fichiers plats, hiérarchies JSON vs. schémas relationnels).
2. **Hétérogénéité syntaxique** : mêmes valeurs encodées différemment selon les systèmes (formats de dates, encodages de caractères, unités de mesure).
3. **Hétérogénéité sémantique** : mêmes concepts désignés par des termes différents selon les domaines ou les organisations (`ID_Client` vs. `Acheteur_Num` vs. `buyer_id`).

Cette classification est désormais le cadre de référence standard pour décrire les défis d'intégration dans les pipelines de transformation vers KG, et structure directement l'architecture en couches que nous proposons.

Le tableau 1.1 illustre cette taxonomie avec des exemples concrets typiques d'un environnement multi-sources industriel, directement représentatifs des défis adressés dans ce travail.

TABLE 1.1 – Taxonomie des formes d'hétérogénéité avec exemples concrets.

| Type d'hétérogénéité | Description                                               | Exemple concret                                                                                                 |
|----------------------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Structurelle</b>  | Différences dans l'organisation des données entre sources | Table normalisée (3NF) côté CRM vs. fichier CSV dénormalisé côté export comptable                               |
| <b>Syntaxique</b>    | Mêmes valeurs encodées différemment selon les systèmes    | Date de commande : 15/03/2024 (CSV France) vs. 2024-03-15 (ISO 8601) vs. Mar 15, 2024 (export US)               |
| <b>Sémantique</b>    | Mêmes concepts désignés par des termes différents         | <code>ID_Client</code> (CRM) vs. <code>Acheteur_Num</code> (facturation) vs. <code>buyer_id</code> (API tierce) |

Cette taxonomie structure directement les trois modules de traitement de notre pipeline : le module d'ingestion adresse l'hétérogénéité **structurelle**, le moteur de triplification avec fonctions de transformation adresse l'hétérogénéité **syntaxique**, et le module de schema matching hybride adresse l'hétérogénéité **sémantique**.



### 1.4.4 Intégration multi-sources et approches récentes

Branco et al. [5] publient une synthèse transversale qui couple explicitement les mécanismes d'intégration — schema matching, résolution d'entités et enrichissement sémantique — avec les modèles de stockage et d'accès (entrepôts en colonnes, NoSQL, lakehouses et fédération). Ils introduisent des workflows de résolution reproductibles pour les problèmes canoniques d'hétérogénéité, notamment l'inadéquation des noms de schéma et l'ambiguïté des formats de dates. D'après cette synthèse, qui couvre les travaux publiés entre 2024 et 2025, les approches d'intégration assistées par IA — dont le schema matching par LLMs — constituent la frontière de recherche la plus active, tout en soulignant l'absence d'implémentations unifiées et entièrement automatisées.

Une approche complémentaire est proposée dans [34], qui traite l'intégration de sources hétérogènes — notamment les tableurs — en extrayant automatiquement une ontologie reflétant la structure interne de la source. Si cette approche est prometteuse pour les sources isolées, elle ne traite pas le problème de l'alignement entre plusieurs sources simultanées, ce qui constitue précisément la valeur ajoutée centrale de notre contribution.

## 1.5 Transformation de Données Tabulaires vers RDF

### 1.5.1 Les standards W3C : R2RML et RML

La transformation de données tabulaires en triplets RDF — aussi appelée *triplification* ou *lifting* — fait l'objet d'une recherche active depuis la publication du standard R2RML par le W3C. Ce standard définit un langage déclaratif pour exprimer des règles de mapping entre des tables relationnelles et des graphes RDF.

**Structure d'une règle de mapping R2RML.** Un fichier R2RML est composé d'un ensemble de **Triples Maps**, chacune décrivant comment transformer une table source (ou une requête SQL) en un ensemble de triplets RDF. Chaque Triples Map est constituée de trois composants :

- La **Logical Table** : la source de données (table SQL ou requête).
- Le **Subject Map** : la règle de construction de l'URI du sujet (généralement basée sur la clé primaire).
- Les **Predicate-Object Maps** : l'ensemble des correspondances (prédicat ontologique  $\leftarrow$  colonne source), chacune associée à un type XSD pour les valeurs littérales.

**Extension vers RML.** Dimou et al. [11] ont étendu ce standard avec RML (*RDF Mapping Language*), un langage déclaratif capable de traiter non seulement les bases de données relationnelles, mais aussi les fichiers CSV, JSON, XML et Excel. La différence fondamentale entre R2RML et RML réside dans la notion de **Logical Source** : là où R2RML n'accepte que des tables SQL, RML généralise cette notion à toute source itérable, en spécifiant un *itérateur* (par exemple, chaque ligne d'un CSV ou chaque nœud d'un fichier JSON). Cette généralisation est déterminante pour notre contexte : elle permet

à un même moteur de traiter simultanément des sources de formats hétérogènes sans modifier l'architecture du pipeline. RML est aujourd'hui le standard de facto pour les projets multi-sources, avec plus de 500 citations dans la littérature scientifique au moment de la rédaction de ce travail.

**Exemple de mapping RML simplifié.** À titre illustratif, la règle suivante exprime en syntaxe YARRRML (surcouche lisible de RML) le mapping d'un fichier CSV contenant des données clients vers l'ontologie cible :

```
mappings:
clients:
  sources: [['clients.csv~csv']]
  s: http://example.org/Customer/$(ID_Client)
  po:
    - [rdf:type, ontology:Customer]
    - [ontology:hasName, $(Nom), xsd:string]
    - [ontology:hasEmail, $(Email), xsd:string]
```

Chaque ligne po (Predicate-Object) fait correspondre une colonne du CSV ( $\$(Nom)$ ) à une propriété ontologique (`ontology:hasName`), avec typage XSD explicite. Ce mécanisme déclaratif est au cœur de notre contribution : la génération automatique de telles règles par un agent LLM constitue la valeur ajoutée principale de notre pipeline.

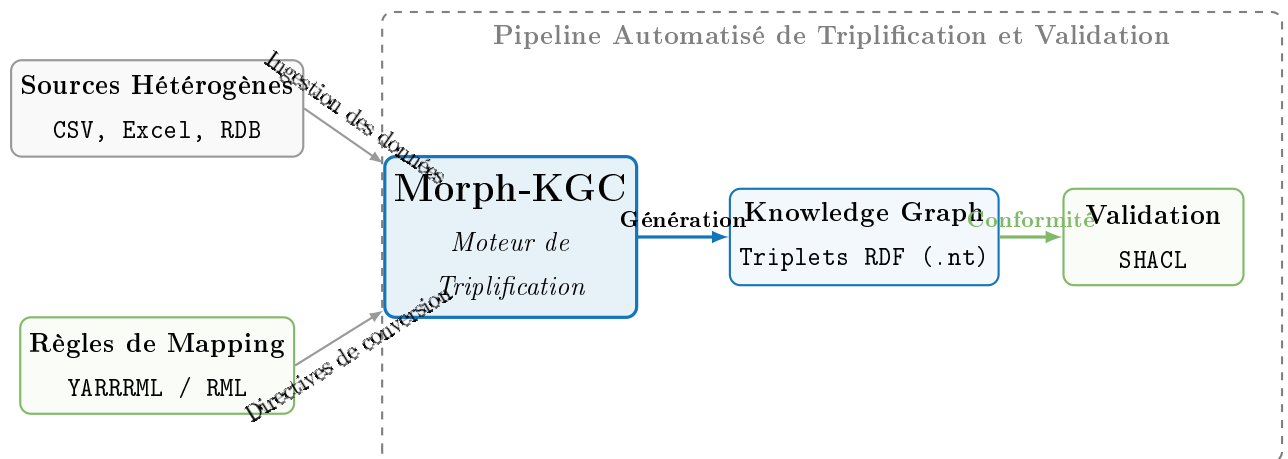


FIGURE 1.3 – Pipeline de transformation de sources tabulaires hétérogènes vers des triplets RDF via le moteur Morph-KGC et le standard RML.

### 1.5.2 Morph-KGC et les vues RML

Le moteur Morph-KGC, développé en Python par Arenas-Guerrero et al. [2], étend RML avec des fonctions définies par l'utilisateur, permettant des transformations syntaxiques complexes lors de la génération des triplets. L'article introduit également le concept de *vues RML*, analogue aux vues SQL, qui permettent de modulariser les règles

de transformation pour des sources complexes. Cette modularité est particulièrement précieuse dans un contexte multi-sources, car elle permet de maintenir et de réutiliser des blocs de mapping indépendamment.

### 1.5.3 YARRRML et les approches déclaratives

Warren et al. [51] conduisent une étude empirique comparative entre deux paradigmes : l’approche déclarative basée sur des chemins (YARRRML) et l’approche de triplification directe (SPARQL Anything). Leurs résultats désignent YARRRML comme l’approche offrant la meilleure utilisabilité, notamment grâce à sa syntaxe YAML lisible par des non-experts, ce qui justifie son adoption dans le pipeline que nous proposons. YARRRML peut être considéré comme une couche d’abstraction au-dessus de RML, simplifiant l’écriture des règles de mapping sans sacrifier leur expressivité.

### 1.5.4 SemTab : le banc d’essai communautaire

La série de challenges SemTab [18, 19], organisée en marge de la conférence ISWC, constitue le banc d’essai communautaire de référence pour l’évaluation des systèmes de transformation de tables vers KGs. Elle évalue trois tâches principales : l’annotation de types de colonnes (CTA), l’annotation d’entités de cellules (CEA) et l’annotation de propriétés de colonnes (CPA). L’édition 2024 a introduit une piste dédiée à l’évaluation des LLMs face aux systèmes STI (*Semantic Table Interpretation*) traditionnels, validant ainsi la pertinence des approches hybrides symbolique/neuronal. Ces challenges constituent le cadre d’évaluation que nous mobilisons pour comparer notre pipeline aux approches existantes.

## 1.6 Schema Matching et Ontology Alignment

### 1.6.1 Définition et enjeux

Le *schema matching* — tâche consistant à identifier des correspondances sémantiques entre éléments de schémas sources et cibles — est une pierre angulaire des pipelines d’intégration. Dans le contexte de la génération de KGs, il correspond à la question : « quelle colonne de quelle table source doit être mappée vers quelle propriété de l’ontologie cible ? ». Sans un schema matching fiable, les triplets RDF générés seront sémantiquement incorrects, quelle que soit la qualité du moteur de triplification utilisé en aval.

### 1.6.2 Évolution des approches

La revue fondatrice de Rahm et Bernstein établit une taxonomie des approches en deux grandes familles : les méthodes **linguistiques** (basées sur les libellés des attributs) et les méthodes **structurelles** (basées sur les contraintes et les relations entre attributs). Pendant deux décennies, ces approches ont constitué l’état de l’art, mais souffrent

d'un plafond de performance structurel face à l'hétérogénéité sémantique : deux colonnes peuvent être sémantiquement équivalentes sans partager *aucun* token commun dans leur libellé (`ID_Client` et `buyer` en sont un exemple trivial). Cette limite constitue la motivation principale du recours aux approches basées sur les embeddings et les LLMs.

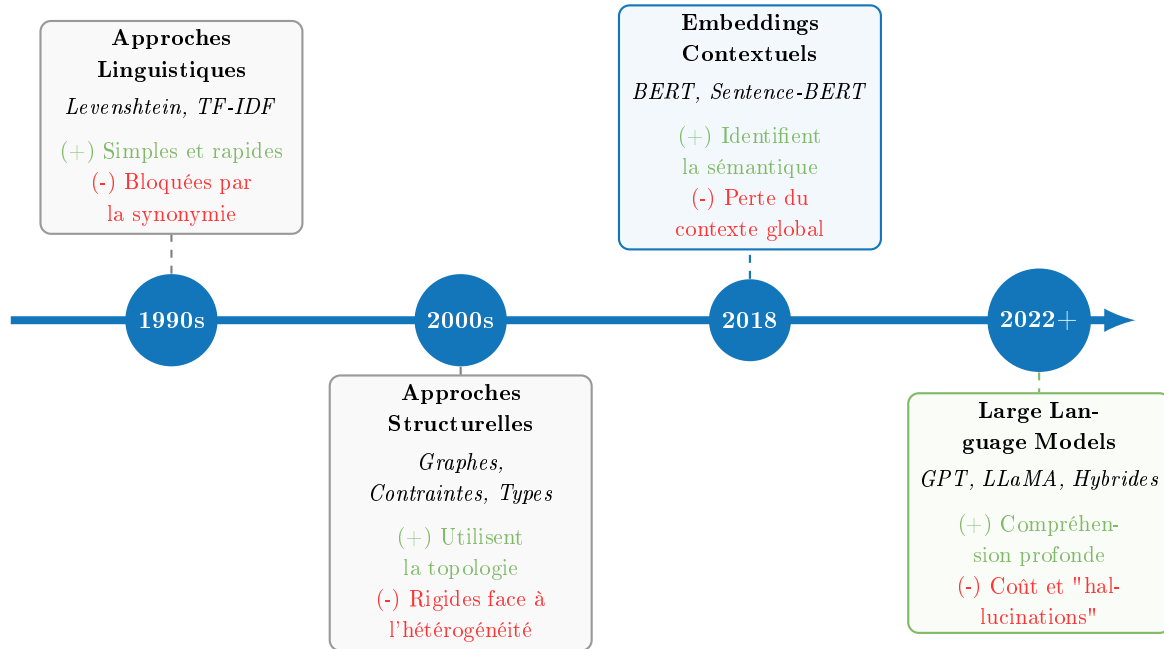


FIGURE 1.4 – Évolution des approches de schema matching : des méthodes linguistiques aux Large Language Models.

### 1.6.3 La rupture paradigmatique des embeddings

L'introduction des modèles d'embeddings contextuels (BERT, Sentence-BERT) a constitué une rupture paradigmatique dans le domaine du schema matching, permettant d'identifier des correspondances même entre termes orthographiquement distincts. En projetant les libellés des attributs dans un espace vectoriel dense où la proximité géométrique reflète la similarité sémantique, ces modèles dépassent les limites des approches purement lexicales. Cette évolution motive l'adoption d'une approche SBERT multilingue dans le module de schema matching de notre pipeline, afin de traiter des sources potentiellement rédigées dans plusieurs langues.

### 1.6.4 L'Ontology Alignment Evaluation Initiative (OAEI)

L'OAEI [36] évalue annuellement les systèmes d'alignement d'ontologies OWL sur des benchmarks standardisés. Ses résultats soulignent régulièrement que les approches mono-sources dominent la littérature, sans mécanisme natif de fusion multi-sources cohérente — limitation directement adressée par notre travail.

Sur le plan de l’automatisation, une étude empirique récente [20] expérimente des outils pré-LLM de matching d’ontologies sur des données relationnelles réelles non enrichies sémantiquement. Les auteurs concluent que ces outils atteignent rapidement leurs limites dès que les libellés des attributs sources et cibles sont trop dissemblables, ce qui motive le recours aux LLMs dans les approches plus récentes.

## 1.7 L’Apport des Large Language Models

### 1.7.1 Panorama général

L’avènement des Large Language Models (LLMs) a profondément transformé le champ de la construction de KGs depuis 2022. Pan et al. [40] proposent une synthèse complète des capacités des LLMs pour la construction et le raisonnement sur les KGs, montrant que ces modèles passent progressivement d’outils d’analyse passifs à des agents actifs de génération de connaissances structurées. Le survey de 2025 [26] documente notamment l’approche LKD-KGC, qui extrait et fusionne automatiquement des types d’entités équivalents via le clustering vectoriel et la déduplication assistée par LLM, sans supervision humaine explicite.

### 1.7.2 LLMatch : schema matching par LLMs

Dans le domaine du schema matching, LLMatch [50] propose un cadre unifié exploitant les LLMs qui opère en deux phases complémentaires. La première phase, dite d’**agrégation**, consolide les colonnes sémantiquement liées en concepts d’ordre supérieur grâce à un LLM (GPT-4 dans les expériences rapportées). La seconde phase, dite de **décomposition**, raffine ces correspondances au niveau colonne-à-colonne pour produire un mapping précis.

**Protocole expérimental et benchmark SchemaNet.** Les auteurs introduisent **SchemaNet**, un benchmark original dérivé de paires de schémas réels issus de trois domaines métier : *e-commerce*, *santé* et *finance*. Ce benchmark est conçu pour capturer les cas de correspondance *cross-domain* difficiles, c’est-à-dire les cas où deux colonnes sont sémantiquement équivalentes mais ne partagent aucun token lexical (par exemple, `purchase_amount` côté e-commerce et `transaction_value` côté finance).

Le protocole compare LLMatch à plusieurs baselines représentatives :

- **Jaccard sur tokens** et **TF-IDF** : méthodes linguistiques classiques de la première génération.
- **SBERT** (*paraphrase-multilingual-MiniLM-L12-v2*) : approche par embeddings contextuels.
- **Système de règles expertes** : heuristiques basées sur les types de données et les contraintes de domaine.

**Résultats et analyse.** Sur les cas de correspondance *cross-domain* de SchemaNet, LLMatch obtient un **F1-score supérieur** aux baselines linguistiques (Jaccard, TF-IDF), qui échouent systématiquement en l’absence de similarité lexicale. Par rapport à SBERT, LLMatch améliore encore la précision sur les cas les plus ambigus, au prix d’un coût d’appel API nettement supérieur (facteur 10 à 30 selon les benchmarks reportés). Ces résultats motivent directement l’architecture hybride que nous adoptons dans notre pipeline : le module SBERT traite les correspondances à haute similarité lexicale (cas simples et peu coûteux), tandis que le LLM n’est sollicité que pour les cas ambigus, ce qui permet de contenir le coût computationnel global tout en préservant la précision du matching sémantique.

**Limite principale.** La limite principale identifiée par les auteurs est le **coût des appels API** (GPT-4 en mode inférence), qui rend LLMatch difficile à déployer à grande échelle sur des pipelines traitant des milliers de paires de colonnes simultanément. Cette limite constitue l’une des motivations de l’architecture hybride symbolique/LLM que nous proposons dans ce travail.

### 1.7.3 SCHEMORA : alignement par prompt engineering

SCHEMORA [23] explore une approche par *prompt engineering*, sans fine-tuning, pour l’alignement sémantique robuste de schémas hétérogènes. L’avantage principal de cette approche est son faible coût d’adaptation à de nouveaux domaines : il suffit de reformuler le prompt sans réentraîner le modèle. Les auteurs évaluent leur approche sur plusieurs benchmarks publics et concluent qu’un LLM correctement *prompté* peut rivaliser avec des systèmes fine-tunés, tout en conservant une flexibilité domaine-agnostique. La limite principale identifiée est la variabilité des sorties selon la formulation exacte du prompt, ce qui nécessite un mécanisme de validation complémentaire.

### 1.7.4 iText2KG : construction incrémentale de KGs

iText2KG [15] représente une approche incrémentale de construction de KGs pilotée par LLMs, capable de gérer les conflits lors de la fusion de sous-graphes générés depuis différentes sources. Bien que centré sur les textes non structurés, son paradigme de *fusion incrémentale* est transposable au contexte tabulaire multi-sources : chaque source génère un sous-graphe local, et un mécanisme de réconciliation basé sur un LLM permet de fusionner ces sous-graphes en résolvant les conflits sémantiques (entités dupliquées, propriétés contradictoires). Les expériences reportées sur des benchmarks standards de NLP montrent que l’approche incrémentale préserve la cohérence du graphe mieux que les approches batch, au prix d’un surcoût computationnel non négligeable.

| Framework     | Tâche                                  | Dataset            | Métrique               | Limite                                 |
|---------------|----------------------------------------|--------------------|------------------------|----------------------------------------|
| LLMatch [50]  | Schema mat-<br>ching multi-<br>domaine | SchemaNet          | F1-score               | Coût API élevé                         |
| SCHEMORA [23] | Alignement sé-<br>mantique             | Benchmarks<br>OAEI | Precision /<br>Recall  | Variabilité des<br>prompts             |
| iText2KG [15] | Construction<br>KG incrémen-<br>tale   | Benchmarks<br>NLP  | Cohérence<br>du graphe | Sources textuelles<br>(non tabulaires) |

FIGURE 1.5 – Comparaison des frameworks LLM pour la construction de KGs : protocoles, datasets et métriques.

### 1.7.5 Limites communes aux approches LLM

La limite commune à ces approches réside dans leur coût computationnel — appels API ou infrastructure GPU significative — et dans la variabilité de leurs sorties, qui nécessitent une validation par *gold standard* pour être fiables dans un contexte de production. Ces considérations motivent l’adoption d’une architecture hybride dans notre pipeline, combinant un module symbolique déterministe pour les cas de correspondance simples et un LLM pour les cas ambigus uniquement.

## 1.8 Défis et Limites des Approches Existantes

L’analyse comparative des travaux présentés dans les sections précédentes met en évidence plusieurs limitations structurelles récurrentes.

**Fragmentation des pipelines.** La majorité des pipelines existants traitent l’hétérogénéité de façon séquentielle et non intégrée : le schema matching, la résolution d’entités et la triplification sont abordés comme des problèmes indépendants, sans propagation des informations de confiance entre les étapes. Cette fragmentation entraîne une accumulation d’erreurs difficile à diagnostiquer et à corriger [55].

**Absence d’évaluation standardisée de bout en bout.** Les évaluations quantitatives restent rares et peu standardisées au niveau du pipeline global. Si SemTab [18, 19] fournit un cadre d’évaluation communautaire pour la tâche d’annotation sémantique, la littérature consultée ne présente que très peu de travaux qui évaluent l’ensemble du pipeline de bout en bout avec des métriques cohérentes de précision, rappel et F1 par module. Wu et al. [55] soulignent que cette absence d’évaluation rigoureuse constitue l’un des principaux freins à la comparaison des approches.

**Reproductibilité insuffisante.** Les implémentations proposées dans la littérature sont souvent dépendantes d’environnements logiciels non documentés, de données propriétaires non publiables ou d’APIs tierces susceptibles d’évoluer. Cela limite considérablement la possibilité de valider ou d’étendre les résultats publiés, et constitue un obstacle

à l’adoption industrielle des approches de recherche.

**Scalabilité non évaluée.** Un quatrième défi, particulièrement critique dans le contexte de ce travail, est celui de la **scalabilité**. La majorité des travaux cités dans ce chapitre évaluent leurs pipelines sur des jeux de données de taille modeste (quelques dizaines à quelques centaines de tables sources), sans analyser le comportement du système lorsque le volume et le nombre de sources augmentent significativement. Or, dans un contexte Big Data — où un pipeline peut être amené à traiter simultanément des dizaines de millions de triplets générés depuis des centaines de sources hétérogènes — la scalabilité est une exigence non négociable. Cette lacune est d’autant plus préoccupante que certaines approches LLM, comme LLMatch [50], impliquent un nombre d’appels API proportionnel au produit du nombre de colonnes sources par le nombre de propriétés cibles, ce qui peut rapidement devenir prohibitif. Notre pipeline adresse partiellement cette contrainte en adoptant une architecture hybride qui limite les appels LLM aux cas ambigus, et en s’appuyant sur des moteurs de triplification optimisés (Morph-KGC) capables de traiter de grands volumes en mode batch.

## 1.9 Positionnement de notre Travail

L’analyse de la littérature présentée dans ce chapitre révèle un ensemble de lacunes convergentes. La littérature consultée ne met pas en évidence de pipeline unifié, reproductible et entièrement automatisé qui intègre dans un seul flux cohérent les cinq étapes suivantes : ingestion multi-format, schema matching hybride (symbolique + sémantique + LLM), résolution d’entités, triplification déclarative et validation formelle par contraintes SHACL, tout en étant évaluable quantitativement sur chaque module.

Pour préciser ce positionnement, le tableau 1.9 synthétise les contributions des travaux les plus proches et les lacunes que notre approche adresse.

Notre travail s’inscrit à l’intersection de ces contributions, en proposant un pipeline qui traite simultanément plusieurs sources tabulaires hétérogènes, automatise la génération des règles YARRRML par un agent LLM (LangChain), intègre nativement la résolution d’entités avec génération de triplets `owl:sameAs`, valide le graphe produit par des contraintes SHACL, et est entièrement conteneurisé et reproductible via Docker Compose, avec évaluation quantitative par gold standard annoté.

Il convient de souligner avec prudence que ce positionnement est fondé sur la littérature accessible et consultée lors de la rédaction de ce travail ; d’éventuelles contributions très récentes ou non indexées pourraient ne pas y figurer.

## 1.10 Conclusion

Ce chapitre a dressé un panorama structuré des approches existantes pour la transformation de données tabulaires hétérogènes en Knowledge Graphs. Nous avons montré que le paradigme Big Data, et en particulier la dimension « Variété », génère des défis



| Travail                      | Contribution        | Lacune                                        | Notre réponse                              |
|------------------------------|---------------------|-----------------------------------------------|--------------------------------------------|
| Yan et al. [56]              | Pipeline KG complet | Alignement partiellement manuel               | Génération auto. de règles YARRRML par LLM |
| LLMatch [50], SCHE-MORA [23] | Schema matching LLM | Isolation du matching (pas de triplification) | Intégration dans un pipeline unifié        |
| Morph-KGC [2]                | Triplification RML  | Suppose un mapping déjà établi                | Génération automatique du mapping          |
| SemTab [18]                  | Évaluation STI      | Pas de pipeline de production                 | Évaluation par gold standard annoté        |

TABLE 1.2 – Positionnement de notre travail par rapport aux contributions existantes.

d’intégration que les approches relationnelles classiques ne permettent pas de résoudre de façon automatisée et scalable.

Les fondements des Knowledge Graphs — modèle RDF, URIs, OWL, SPARQL, SHACL — offrent un cadre formel pour surmonter cette hétérogénéité, à condition de disposer de pipelines de transformation robustes. L’analyse de la littérature a permis d’identifier quatre lacunes convergentes : la fragmentation des pipelines existants, l’absence d’évaluation standardisée de bout en bout, la reproductibilité insuffisante, et le déficit d’études sur la scalabilité.

Les LLMs représentent une avancée significative pour les tâches de schema matching et de construction de KGs, mais leur intégration dans des pipelines de production reste un défi ouvert, notamment en raison de leur coût computationnel et de la variabilité de leurs sorties.

Notre contribution, positionnée à l’intersection des travaux sur la triplification déclarative (RML/YARRRML), le schema matching hybride et l’automatisation par LLMs, vise à combler ces lacunes de manière intégrée et reproductible. Elle soulève cependant une question méthodologique fondamentale qui guidera le chapitre suivant : *comment concevoir un pipeline qui soit à la fois suffisamment expressif pour capturer la diversité des formes d’hétérogénéité sémantique, et suffisamment robuste pour garantir la qualité du Knowledge Graph produit à grande échelle ?* Le chapitre 2 présente en détail l’architecture, les choix de conception et le protocole d’évaluation que nous proposons pour répondre à cette question.

# Chapitre 2

## Méthodologie et Conception de l'Architecture

### 2.1 Introduction et Justification des Choix Architecturaux

L'analyse critique de la littérature conduite au chapitre 1 a mis en évidence quatre lacunes structurelles récurrentes dans les approches existantes de transformation de données tabulaires hétérogènes en Knowledge Graphs. Premièrement, les approches recensées sont **fragmentées** : elles traitent isolément le schema matching, la triplification ou la résolution d'entités, sans proposer de chaîne de traitement intégrée de bout en bout. Deuxièmement, elles **gèrent mal l'hétérogénéité multi-sources** : la plupart des travaux supposent une source unique ou des schémas déjà proches, et n'adressent pas explicitement les trois niveaux d'hétérogénéité (structurelle, syntaxique, sémantique) identifiés à la section 1.4. Troisièmement, elles **exploitent peu les grands modèles de langage de manière contrôlée** : lorsqu'un LLM est utilisé, il l'est le plus souvent de façon systématique et coûteuse, sans stratégie de filtrage préalable ni mécanisme de validation structurée de ses sorties. Enfin, elles sont **rarement reproductibles**, faute d'environnement d'exécution conteneurisé et de protocole d'évaluation partagé.

Ces observations convergent vers un besoin scientifique et technique précis : la conception d'un pipeline *unifié, déclaratif, automatisé et reproductible*, dans lequel chaque module est évaluable indépendamment et l'ensemble est déployable sans intervention manuelle sur des volumes de données variables. **Pour répondre à ces limitations, nous proposons un pipeline composé de cinq modules fonctionnels séquentiels, complétés par une infrastructure d'orchestration Docker Compose et un protocole d'évaluation fondé sur un gold standard annoté manuellement**, détaillé à la section 2.8. Cette continuité directe entre les lacunes identifiées au chapitre 1 et les choix de conception présentés dans ce chapitre en constitue le fil conducteur.

La contribution de ce mémoire répond à ce besoin par la conception et l'implémentation d'une architecture en cinq modules séquentiels et interdépendants, orchestrés par

Docker Compose. Cette architecture se distingue des travaux existants par trois apports principaux. Premièrement, le **schema matching est hybride** : il combine un module symbolique déterministe basé sur les embeddings Sentence-BERT [45] pour les correspondances à haute similarité lexicale, et un agent LLM piloté par LangChain [8] pour les cas ambigus, avec génération automatique des règles de mapping au format YARRRML [51]. Deuxièmement, la **triplification est entièrement déclarative** : les règles YARRRML sont exécutées par le moteur Morph-KGC [2], ce qui garantit la séparation entre la logique métier de transformation et le code d'exécution. Troisièmement, la **reproductibilité est garantie par la conteneurisation** complète de la stack via Docker Compose [12], rendant le pipeline déployable en une seule commande sur n'importe quelle infrastructure compatible.

Ce chapitre présente en détail la conception de chacun de ces cinq modules. Nous commençons par décrire l'architecture globale (section 2.2), avant de détailler successivement le module d'ingestion (section 2.3), le module de schema matching hybride (section 2.4), le module de triplification (section 2.5), le module de résolution d'entités et de validation (section 2.6), et enfin l'orchestration par Docker Compose (section 2.7).

## 2.2 Architecture Globale du Pipeline

La figure 2.1 présente l'architecture globale du pipeline proposé. Celui-ci est organisé comme un flux de données unidirectionnel, allant des sources hétérogènes brutes jusqu'au Knowledge Graph validé et interrogeable, en passant par cinq modules fonctionnels distincts. Chaque module produit un artefact intermédiaire formellement défini, ce qui permet d'évaluer la qualité de la transformation à chaque étape indépendamment du reste du pipeline.

Le pipeline est composé de **cinq modules fonctionnels** complétés par une infrastructure d'orchestration transverse (Docker Compose, section 2.7). Chaque module répond à une problématique spécifique de l'intégration de données hétérogènes, et son artefact de sortie constitue l'artefact d'entrée du module suivant, sans état partagé implicite entre les étapes.

- **Module 1 — Ingestion et prétraitement** (section 2.3) : collecte des sources brutes hétérogènes (CSV, Excel, SQL, JSON), profilage automatique de la qualité, normalisation syntaxique et persistance dans une zone de staging MongoDB. Il traite exclusivement l'hétérogénéité *structurelle* et *syntactique*.
- **Module 2 — Schema matching hybride** (section 2.4) : identification de la propriété ontologique correspondant à chaque colonne source, par une cascade SBERT (rapide, déterministe) puis agent LLM (coûteux, réservé aux cas ambigus), et génération automatique des règles de mapping YARRRML. C'est le module qui résout l'hétérogénéité *sémantique*, la plus difficile des trois.
- **Module 3 — Triplification** (section 2.5) : exécution déclarative des règles YARRRML par le moteur Morph-KGC, produisant les triplets RDF conformes à l'ontologie cible.

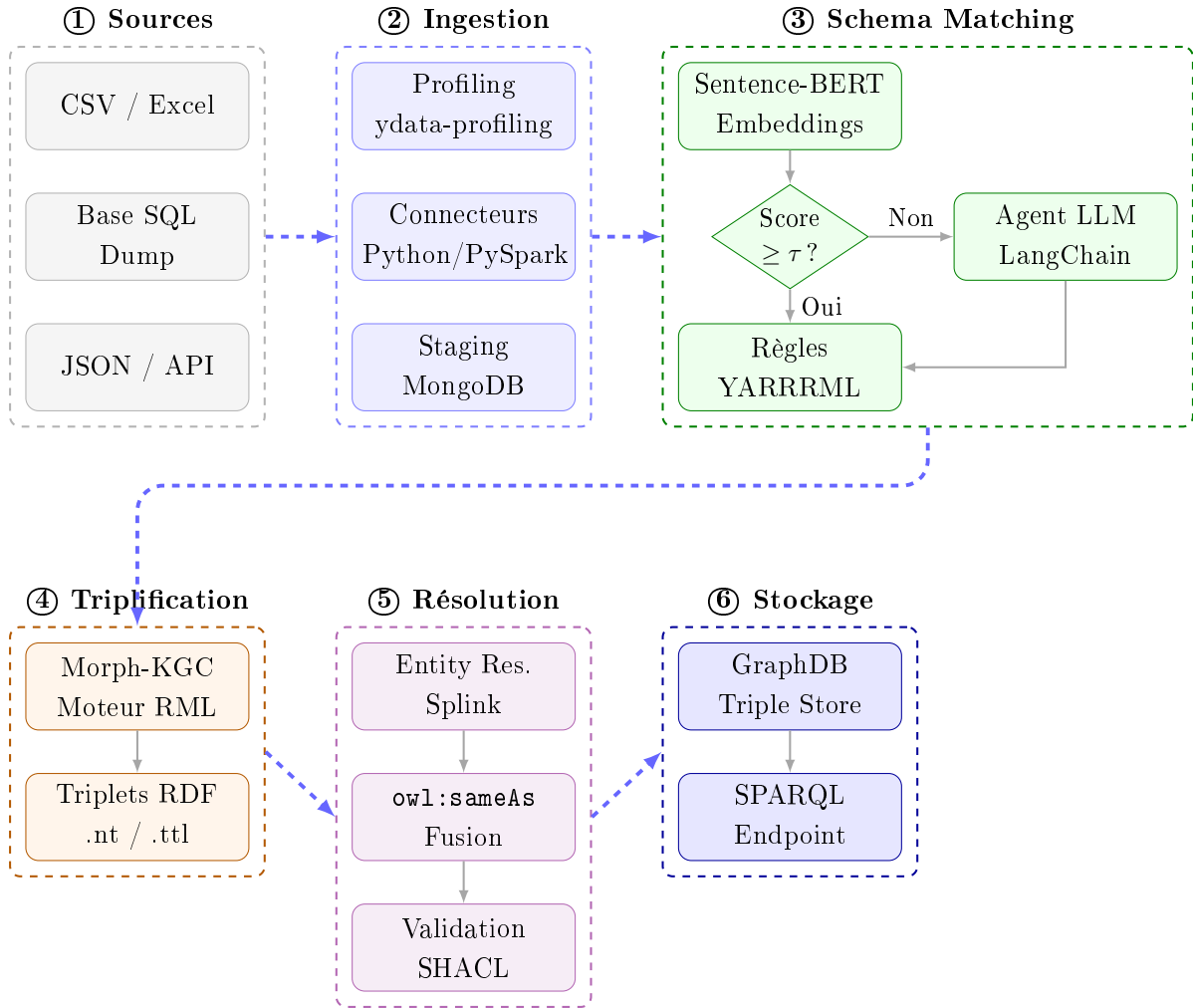


FIGURE 2.1 – Architecture globale du pipeline de transformation structurée en deux phases. La première ligne illustre l’ingestion et l’alignement sémantique hybride. La seconde ligne détaille la génération RDF, la résolution d’entités et le stockage final.

- **Module 4 — Résolution d’entités et validation** (section 2.6) : réconciliation probabiliste des entités décrites par plusieurs sources (Splink), génération des ponts `owl:sameAs`, puis validation structurelle du graphe par des contraintes SHACL.
- **Module 5 — Stockage et requêtage** : chargement du graphe validé dans un triple store (GraphDB) exposant un point d’accès SPARQL, ainsi qu’une interface de visualisation interactive.

L’ensemble de ces modules est orchestré par **Docker Compose** (section 2.7), qui garantit que le pipeline complet est déployable en une seule commande, indépendamment de l’infrastructure hôte — répondant directement au déficit de reproductibilité identifié au chapitre 1.

Le tableau 2.1 synthétise les artefacts d’entrée et de sortie de chaque module, matérialisant l’interface contractuelle entre les étapes successives du pipeline.

TABLE 2.1 – Artefacts d’entrée et de sortie de chaque module du pipeline.

| # | Module                            | Entrée                                                              | Sortie                                                                                    |
|---|-----------------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1 | Ingestion & Profilage             | Fichiers CSV, Excel, dumps SQL, JSON bruts                          | DataFrames normalisés, rapports de profilage, collections MongoDB de staging              |
| 2 | Schema Matching Hybride           | Schémas sources (noms et exemples de colonnes), ontologie OWL cible | Règles de mapping YARRRML validées par module                                             |
| 3 | Triplification                    | Données normalisées + règles YARRRML                                | Fichiers de triplets RDF (format N-Triples <code>.nt</code> ou Turtle <code>.ttl</code> ) |
| 4 | Résolution d’entités & Validation | Triplets RDF + contraintes SHACL + ontologie OWL                    | Graphe RDF enrichi ( <code>owl:sameAs</code> ), rapport de validation SHACL               |
| 5 | Stockage & Requête                | Graphe RDF validé                                                   | Repository GraphDB peuplé, endpoint SPARQL opérationnel                                   |

## 2.3 Module 1 : Ingestion et Prétraitement des Données

### 2.3.1 Rôle fonctionnel

Le module d’ingestion constitue le point d’entrée du pipeline. Son rôle est de collecter les données brutes provenant de sources hétérogènes, de les normaliser dans un format interne unifié, d’en évaluer la qualité, et de les persister dans une zone de staging avant les traitements sémantiques. Ce module n’effectue aucune transformation sémantique : son périmètre est strictement limité à l’hétérogénéité *structurelle* et *syntactique* telle que définie dans la taxonomie présentée à la section 1.4.

### 2.3.2 Sources supportées et stratégie de scalabilité

Le pipeline supporte nativement les formats tabulaires les plus répandus dans les environnements industriels et académiques :

- **Fichiers plats** : CSV (séparateurs variables, encodages UTF-8 et Latin-1), Excel (`.xlsx` et `.xls`), fichiers TSV à tabulations.
- **Bases relationnelles** : connexion via des dumps SQL (`.sql`) ou via des connecteurs SQLAlchemy [4] vers PostgreSQL, MySQL et SQLite.
- **Formats semi-structurés** : JSON tabulaire (tableaux d’objets), XML avec struc-

ture répétitive extractible.

**Stratégie de scalabilité : bascule Pandas / PySpark.** Chaque connecteur est implémenté selon une stratégie à deux niveaux, afin de couvrir un spectre de volumes allant de quelques milliers à plusieurs millions d’enregistrements :

- **Mode Pandas** [48] : utilisé lorsque la taille estimée du fichier source est inférieure à un seuil configurable (valeur par défaut : **500 Mo** ou  $10^6$  **lignes**). Dans ce mode, l’intégralité des données est chargée en mémoire vive, ce qui offre les meilleures performances pour les petits volumes grâce à la vectorisation NumPy.
- **Mode PySpark** [58] : activé automatiquement lorsque la taille dépasse le seuil ou lorsque le paramètre `force_spark=True` est positionné dans la configuration du pipeline. PySpark traite les données en partitions distribuées, permettant de dépasser les contraintes de mémoire d’un seul nœud et d’exploiter un cluster multi-nœuds si disponible.

La détection automatique du mode est réalisée par un module de **profilage léger pré-ingestion** : avant le chargement complet, seules les métadonnées du fichier (taille en octets, nombre de lignes estimé par échantillonnage des premiers et derniers blocs) sont lues, ce qui permet de choisir le moteur approprié sans charger l’ensemble des données en mémoire. Dans les deux modes, le résultat produit est un *DataFrame* dont l’interface est unifiée via une couche d’abstraction, garantissant que les modules aval (profilage, normalisation, schema matching) sont indépendants du moteur d’exécution utilisé.

### 2.3.3 Profilage automatique des données

Dès l’ingestion, chaque source fait l’objet d’un **profilage automatique** via la bibliothèque `ydata-profiling` [57]. Ce profilage produit pour chaque colonne : son type inféré (numérique, catégoriel, temporel, textuel), sa cardinalité, son taux de valeurs manquantes, sa distribution statistique (quartiles, histogramme), et des exemples de valeurs représentatives.

Ces métadonnées de profilage jouent un rôle crucial dans le module suivant : elles constituent la **représentation augmentée de colonne** transmise au module de schema matching, permettant au module SBERT et à l’agent LLM de disposer d’un contexte plus riche que le seul nom de la colonne.

### 2.3.4 Staging et Mapping Store dans MongoDB

Les DataFrames normalisés sont persistés dans une base **MongoDB** [31] utilisée selon deux rôles complémentaires dans le pipeline.

**Rôle 1 — Zone de staging des données.** MongoDB sert de **zone de staging intermédiaire** entre le module d’ingestion et le module de schema matching. Son schéma flexible (document JSON sans DDL préalable) permet de stocker des DataFrames de formats hétérogènes dans des collections séparées, sans migration de schéma. Chaque

collection correspond à une source et contient, en plus des données normalisées, les métadonnées de profilage (type inféré, cardinalité, taux de nullité, exemples de valeurs) sous forme de sous-documents JSON. Cette co-localisation des données et de leurs métadonnées dans un même store simplifie l'accès par le module de schema matching, qui peut requêter simultanément le contenu et le profil d'une colonne.

**Rôle 2 — Mapping Store versionné.** MongoDB remplit un second rôle, moins courant dans la littérature mais crucial pour la reproductibilité du pipeline : il sert de **mapping store versionné**, c'est-à-dire d'un référentiel persistant des règles de mapping YARRRML produites par le module de schema matching.

Chaque règle de mapping est stockée dans une collection dédiée (`schema_mappings`) sous la forme d'un document comprenant :

- L'identifiant de la source et le nom de la colonne.
- La propriété ontologique cible et le type XSD mappé.
- La voie de décision empruntée ("`sbert`" ou "`llm`").
- Le score de confiance associé.
- Un **horodatage** et un **identifiant de version** permettant de tracer l'historique des modifications.
- Un indicateur de validation humaine (`validated: true/false`).

Cette architecture de mapping store présente deux avantages majeurs. Premièrement, elle permet de **réutiliser les règles validées** lors d'une exécution ultérieure du pipeline sur la même source, sans relancer le module de schema matching. Deuxièmement, elle offre un mécanisme de **révision humaine incrémentale** : un expert métier peut consulter et valider les correspondances via une interface dédiée, et les règles invalidées sont automatiquement soumises à une nouvelle analyse LLM lors de la prochaine exécution. Ce paradigme est aligné avec les recommandations *human-in-the-loop* de la littérature récente sur l'automatisation du schema matching [5].

### 2.3.5 Normalisation syntaxique

Avant la persistance en staging, un module de normalisation syntaxique applique une série de transformations canoniques sur les valeurs :

- **Dates** : toutes les dates sont converties vers le format ISO 8601 (YYYY-MM-DD), indépendamment du format source (DD/MM/YYYY, MM-DD-YY, Jan 15, 2024, etc.), en s'appuyant sur la bibliothèque `dateutil` [33].
- **Nombres** : unification des séparateurs décimaux et de milliers, suppression des symboles monétaires, conversion vers `float64`.
- **Textes** : normalisation Unicode (NFC), homogénéisation de la casse, suppression des espaces superflus.
- **Valeurs nulles** : détection et remplacement cohérent des représentations variées du vide ("`N/A`", "`null`", cellule vide, "-").

Ces normalisations syntaxiques réduisent directement l'hétérogénéité de niveau 2 (syn-

taxique) avant que les données n’atteignent le moteur de triplification, où elles seront typées explicitement via les types XSD de RDF.

Une fois les données ingérées, normalisées et persistées dans la zone de staging MongoDB, le pipeline dispose d’une représentation structurellement homogène de sources initialement hétérogènes. La question qui se pose alors est d’ordre sémantique : comment identifier automatiquement la propriété ontologique vers laquelle chaque colonne doit être mappée ? C’est l’objet du Module 2, présenté dans la section suivante.

## 2.4 Module 2 : Schema Matching Hybride et Génération YARRRML

Ce module constitue le **cœur scientifique** de la contribution présentée dans ce mémoire. Son objectif est de résoudre l’hétérogénéité sémantique — la plus difficile des trois formes d’hétérogénéité — en identifiant automatiquement la correspondance entre chaque colonne d’une source tabulaire et la propriété ontologique adéquate de l’ontologie OWL cible, puis en formalisant cette correspondance dans une règle de mapping YARRRML prête à être exécutée par Morph-KGC.

### 2.4.1 Représentation des colonnes sources

L’entrée de ce module n’est pas simplement le nom d’une colonne, mais une **représentation augmentée** composée de :

- Le **nom de la colonne** (*header*) tel qu’extrait de la source.
- Le **type inféré** et la **distribution statistique** issus du profilage (section 2.3).
- Trois à cinq **exemples de valeurs** représentatives de la colonne.
- Le **nom de la table source** et, le cas échéant, les noms des colonnes adjacentes (contexte structurel local).

Cette représentation augmentée permet de discriminer deux colonnes portant le même nom mais appartenant à des tables de domaines différents (par exemple, une colonne **date** dans une table de commandes et une colonne **date** dans une table de naissances).

### 2.4.2 L’ontologie du domaine : artefact central du pipeline

Avant de décrire le processus d’alignement, il est nécessaire de préciser la nature et la structure de l’**ontologie OWL cible** vers laquelle toutes les colonnes sources sont mappées. Cette ontologie constitue l’artefact sémantique fondateur du pipeline : elle définit le vocabulaire partagé qui permet d’unifier des sources hétérogènes sous un modèle de connaissance commun.

**Conception et structure.** L’ontologie du domaine est modélisée sous **Protégé** [32] (version 5.x), le standard académique et industriel pour la création d’ontologies OWL 2



DL. Elle suit le principe de **réutilisation maximale** : les classes et propriétés des vocabulaires standards du Web Sémantique sont réutilisées chaque fois que possible, et complétées par des éléments spécifiques au domaine préfixés **ex:** (<http://kg.projet.fr/ontology#>).

Les vocabulaires réutilisés sont les suivants :

- **FOAF** (*Friend of a Friend*) [6] : pour les personnes (`foaf:Person`), leurs noms (`foaf:familyName`, `foaf:firstName`) et leurs emails (`foaf:mbox`).
- **Schema.org** [17] : pour les dates de naissance (`schema:birthDate`), les organisations (`schema:Organization`) et les transactions (`schema:Order`).
- **Dublin Core** [52] : pour les métadonnées descriptives des sources (`dc:title`, `dc:source`).

L'ontologie compte, dans sa version utilisée pour les expériences de ce travail, **12 classes**, **34 propriétés d'objet** et **28 propriétés de données**, couvrant les domaines de la gestion de la relation client (CRM) et des ressources humaines (RH), qui correspondent aux sources de données utilisées pour l'évaluation (voir chapitre 4). Elle est exportée au format Turtle (`.ttl`) et versionnée dans le dépôt Git du projet.

**Alignement avec OWL 2 DL.** Le profil OWL 2 DL est retenu pour sa garantie de **décidabilité du raisonnement** [21] : un moteur de raisonnement (Hermit [46] dans notre cas) peut déterminer en temps fini toutes les conséquences logiques de l'ontologie, notamment les relations d'équivalence entre entités issues de sources différentes déclarées via `owl:equivalentProperty` et `owl:sameAs`. Cette propriété est exploitée dans le module de résolution d'entités décrit à la section 2.6.

### 2.4.3 Approche symbolique : embeddings Sentence-BERT

La première couche du module de matching est une approche **symbolique déterministe** basée sur les embeddings sémantiques produits par le modèle Sentence-BERT [45].

Plus précisément, nous utilisons le modèle pré-entraîné `paraphrase-multilingual-MiniLM-L12-v2`, disponible via la bibliothèque `sentence-transformers` [45]. Ce choix est motivé par deux critères complémentaires. D'une part, ce modèle est conçu pour optimiser la similarité cosinus comme mesure de proximité sémantique, contrairement aux modèles BERT originaux dont les représentations ne sont pas directement comparables via cosinus [45]. D'autre part, sa variante multilingue couvre plus de cinquante langues, ce qui le rend adapté aux contextes industriels dans lesquels les colonnes sources peuvent être libellées en français, en anglais ou dans d'autres langues simultanément.

**Processus d'encodage et calcul de similarité.** Le processus est le suivant :

1. Chaque colonne source est encodée en un vecteur dense de dimension  $d = 384$  à partir de sa représentation augmentée (nom de la colonne, type inféré, exemples de valeurs).

2. Chaque propriété de l'ontologie OWL cible est encodée de la même façon, en concaténant son URI locale, son `rdfs:label` et son `rdfs:comment` en une seule chaîne textuelle avant l'encodage.
3. La **similarité cosinus** entre chaque vecteur de colonne source et chaque vecteur de propriété cible est calculée, produisant une matrice de scores  $S \in [0, 1]^{n_s \times n_p}$ , où  $n_s$  est le nombre de colonnes sources et  $n_p$  le nombre de propriétés de l'ontologie.
4. Si le score maximal  $s^* = \max_j S_{ij}$  pour la colonne  $i$  dépasse un seuil  $\tau$ , la correspondance est acceptée automatiquement et la règle YARRRML correspondante est générée sans intervention du LLM.

**Calibration du seuil  $\tau$ .** La valeur du seuil  $\tau$  est un hyperparamètre critique du module : une valeur trop basse génère des correspondances incorrectes (faux positifs), tandis qu'une valeur trop haute renvoie inutilement des cas simples vers le LLM, augmentant le coût computationnel. La valeur par défaut retenue pour ce travail est  $\tau = 0.82$ , déterminée par une procédure de calibration sur un **gold standard annoté manuellement**.

Cette procédure consiste à calculer le F1-score du module SBERT seul (sans LLM) pour des valeurs de  $\tau$  variant entre 0.60 et 0.95 par pas de 0.02, sur l'ensemble d'entraînement du gold standard. La valeur  $\tau = 0.82$  correspond au maximum du F1-score observé sur cet ensemble, indiquant le meilleur compromis entre précision (peu de faux positifs) et rappel (peu de cas simples renvoyés inutilement au LLM). Les résultats complets de cette calibration, incluant la courbe F1-score en fonction de  $\tau$ , sont présentés au chapitre 4.

Cette approche déterministe traite efficacement les cas de correspondance à forte similarité sémantique (par exemple, `nom_complet`  $\rightarrow$  `foaf:name`, `email_professionnel`  $\rightarrow$  `foaf:mbox`) avec un coût computationnel négligeable, de l'ordre de la milliseconde par paire sur un processeur standard, sans appel API externe.

#### 2.4.4 Approche neuronale : agent LLM via LangChain

Pour les colonnes dont le score SBERT est inférieur au seuil  $\tau$ , le pipeline délègue la décision à un **agent LLM** orchestré par le framework LangChain [8]. Ce sont typiquement les cas d'hétérogénéité sémantique profonde, où deux termes désignent le même concept sans partager aucun token lexical (par exemple, `CA_annuel`  $\rightarrow$  `ex:annualRevenue`).

**Choix du modèle de langage.** LangChain est un framework d'orchestration qui abstrait la couche d'appel au modèle de langage sous-jacent. Dans ce travail, le modèle utilisé est **GPT-4o** (OpenAI) [39], qui présente les avantages suivants par rapport aux alternatives évaluées (Mistral-7B-Instruct [24] et LLaMA-3-8B [49]) :

- **Précision sur les tâches de raisonnement structural** : GPT-4o démontre une capacité supérieure à interpréter le contexte sémantique d'une colonne à partir d'exemples de valeurs, tâche qui requiert un raisonnement combiné sur le nom, le type et le contenu de la colonne.

- **Fiabilité du format de sortie JSON** : GPT-4o supporte nativement le mode *structured output* [39], qui garantit que la réponse est un objet JSON syntaxiquement valide conforme à un schéma prédéfini, éliminant les erreurs de parsing.
- **Latence acceptable** : le temps de réponse moyen observé est de l'ordre de 1 à 3 secondes par requête, compatible avec un usage en mode batch sur des dizaines à quelques centaines de colonnes ambiguës.

Il convient de souligner que le coût d'appel API de GPT-4o constitue la principale limite de cette approche pour des pipelines à très grand nombre de colonnes. C'est précisément la raison pour laquelle le LLM n'est sollicité qu'en dernier recours, après le filtrage par le module SBERT. L'architecture hybride permet de limiter le nombre d'appels LLM au strict minimum des cas ambigus. La conception du pipeline est par ailleurs compatible avec un remplacement de GPT-4o par un modèle open-source déployé localement (Mistral ou LLaMA via Ollama [37]) pour les contextes où la confidentialité des données est une contrainte, sans modification de l'architecture LangChain.

**Structure du prompt.** L'agent LangChain reçoit un prompt structuré en plusieurs sections :

- **Contexte du domaine** : description de l'ontologie cible et de ses classes principales.
- **Description de la colonne source** : nom, type, distribution statistique, et trois à cinq exemples de valeurs représentatives.
- **Propriétés cibles candidates** : liste des propriétés de l'ontologie dont le score SBERT se situe entre  $\tau - 0.15$  et  $\tau$  (zone d'ambiguïté), avec leurs définitions `rdfs:label` et `rdfs:comment`.
- **Exemples *few-shot*** : deux à trois exemples de mappings corrects issus du gold standard, présentant des cas similaires d'équivalence sémantique sans similarité lexicale.
- **Instruction de sortie** : spécification du schéma JSON attendu en sortie, comprenant la propriété cible retenue, le score de confiance estimé (entre 0 et 1), la transformation XSD à appliquer, et une justification textuelle en une phrase.

**Gestion des erreurs et mécanisme de *retry*.** La sortie JSON de l'agent est parsée et validée structurellement avant d'être utilisée pour générer la règle YARRRML correspondante. En cas d'incohérence (sortie non conforme au schéma JSON attendu ou propriété cible non reconnue dans l'ontologie), un mécanisme de *retry* avec reformulation automatique du prompt est déclenché, avec un maximum de trois tentatives. Si les trois tentatives échouent, la colonne est marquée comme *non réconciliée* et consignée dans un rapport d'anomalies structuré (format JSON) pour révision humaine, incluant le nom de la colonne, les scores SBERT des candidats proches, et les sorties brutes des tentatives LLM.

### 2.4.5 Architecture hybride et seuil adaptatif

La figure 2.2 illustre le flux de décision du module de schema matching hybride. L'articulation entre le module SBERT et l'agent LLM constitue une stratégie de **cascade de coûts croissants** : le traitement le moins coûteux (inférence SBERT locale, de l'ordre de la milliseconde par paire) est tenté en premier, et le traitement le plus coûteux (appel API LLM, de l'ordre de la seconde par requête) n'est sollicité que pour les cas que le premier niveau ne peut résoudre avec confiance.

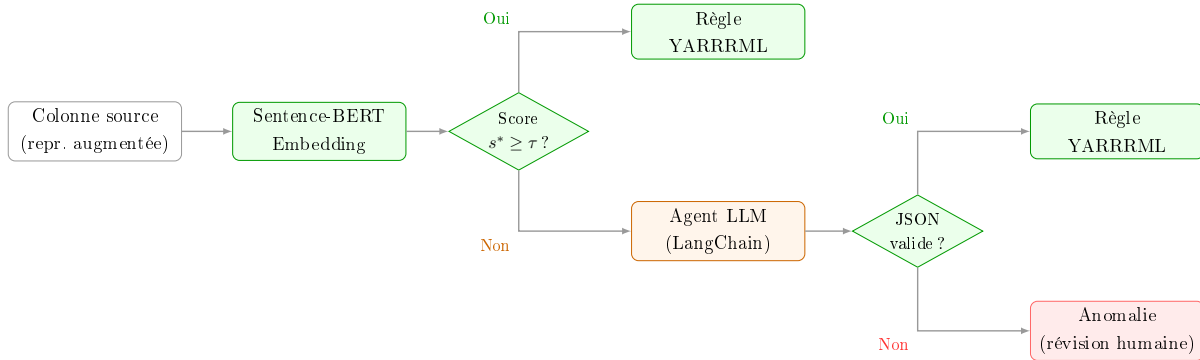


FIGURE 2.2 – Flux de décision du module de schema matching hybride. Le seuil  $\tau$  est calibré empiriquement sur le gold standard pour maximiser le F1-score global du module.

### 2.4.6 Génération automatique des règles YARRRML

Quelle que soit la voie empruntée (SBERT ou LLM), le résultat du module de matching est une règle de mapping au format **YARRRML**. YARRRML (*Yet Another RDF Mapper*), déjà justifié dans l'état de l'art [51], est utilisé ici pour deux raisons complémentaires. D'une part, sa syntaxe YAML est lisible et modifiable par un expert métier non informaticien, ce qui facilite la phase de validation humaine des mappings. D'autre part, YARRRML est compilable vers des fichiers RML standard, exécutables par le moteur Morph-KGC sans modification supplémentaire.

Un exemple de règle YARRRML générée automatiquement par le pipeline pour une source de données clients est présenté ci-dessous :

prefixes:

```

ex:    "http://kg.projet.fr/ontology#"
foaf:  "http://xmlns.com/foaf/0.1/"
xsd:   "http://www.w3.org/2001/XMLSchema#"

```

mappings:

```

clients_source_A:
  sources: [["clients_A.csv~csv"]]
  s: ex:Customer_$(ID_Client)

```

```

po:
- [rdf:type, foaf:Person]
- [foaf:familyName, $(nom_complet), xsd:string]
- [foaf:mbox, $(email_pro), xsd:anyURI]
- [ex:annualRevenue, $(CA_annuel), xsd:decimal]
- [schema:birthDate, $(dt_naissance), xsd:date]

```

Chaque triplet `po` (Predicate-Object) est la traduction directe d’une correspondance identifiée par le module de schema matching : la colonne source (`$(nom_complet)`) est associée à la propriété ontologique (`foaf:familyName`) avec son type XSD cible (`xsd:string`).

## 2.5 Module 3 : Triplification Sémantique avec Morph-KGC

### 2.5.1 Rôle fonctionnel

Le module de triplification transforme les données normalisées en staging, guidé par les règles YARRRML produites par le module précédent, en un ensemble de **triplets RDF** conformes à l’ontologie cible. C’est l’étape qui matérialise concrètement le passage de la représentation tabulaire à la représentation graphe.

### 2.5.2 Justification du choix de Morph-KGC

Le moteur retenu pour l’exécution des règles RML est **Morph-KGC** [2], développé en Python par l’équipe d’Arenas-Guerrero et al. (ESWC 2023). Ce choix est justifié par plusieurs arguments complémentaires par rapport aux alternatives disponibles (RMLMapper en Java, RDFLib-applet en Python, triplification manuelle par script).

Premièrement, Morph-KGC est nativement intégrable dans un pipeline Python sans dépendance JVM, ce qui simplifie l’environnement d’exécution Docker. Deuxièmement, il supporte les **fonctions de transformation définies par l’utilisateur** (*user-defined functions*, UDFs), permettant d’appliquer des transformations syntaxiques complexes (nettoyage de chaînes, normalisation de dates, casting de types) directement dans les règles RML, sans prétraitement séparé. Troisièmement, sa conception orientée flux (*streaming*) lui permet de traiter de grands volumes de données sans charger l’intégralité du jeu de données en mémoire, répondant ainsi à la contrainte de scalabilité Big Data [2]. Quatrièmement, par rapport à un script Python natif utilisant directement la bibliothèque RDFLib [44], l’approche déclarative de Morph-KGC sépare la logique de transformation du code d’exécution : modifier une règle de mapping ne nécessite pas de modifier le code Python du pipeline, ce qui améliore la maintenabilité et la testabilité.

### 2.5.3 Processus d'exécution

L'exécution de Morph-KGC est pilotée par un fichier de configuration INI qui référence les sources de données, les fichiers de règles YARRRML compilés en RML, et le format de sortie souhaité. Deux formats de sortie sont supportés dans notre pipeline :

- **N-Triples** (.nt) : un triplet par ligne, format optimal pour le traitement en flux et l'import dans les triple stores.
- **Turtle** (.ttl) : format plus compact et lisible par un humain, utilisé pour l'inspection manuelle et la documentation.

### 2.5.4 Gestion des graphes nommés

Pour maintenir la **traçabilité de la provenance** des triplets générés depuis des sources multiples, Morph-KGC est configuré pour produire des **N-Quads** (.nq) plutôt que des N-Triples lors du traitement multi-sources. Chaque triplet est associé à un graphe nommé correspondant à la source dont il est issu (par exemple, `<http://kg.projet.fr/graph/source_A>`). Cette granularité de provenance facilite l'audit de qualité et la détection d'incohérences inter-sources dans l'étape suivante.

À l'issue de la triplification, le graphe RDF produit est sémantiquement conforme à l'ontologie cible au niveau des propriétés individuelles. Il peut cependant contenir deux classes de défauts résiduels : des entités redondantes issues de sources multiples représentant le même individu réel, et des violations de contraintes structurelles définies par l'ontologie. Le Module 4, présenté dans la section suivante, adresse ces deux classes de défauts de manière séquentielle et complémentaire.

## 2.6 Module 4 : Résolution d'Entités et Validation SHACL

Ce module opère en deux phases distinctes mais complémentaires : la résolution d'entités, qui fusionne les représentations redondantes d'une même entité du monde réel issues de sources différentes, et la validation SHACL, qui vérifie la conformité formelle du graphe produit à l'ontologie cible.

### 2.6.1 Phase 1 — Résolution d'entités avec Splink

#### Problématique

Après triplification, le graphe contient potentiellement plusieurs nœuds RDF distincts représentant la même entité réelle. Par exemple, le client *Jean Dupont* peut apparaître sous l'URI `ex:Customer_42` issue du système CRM et sous l'URI `ex:Customer_E17` issue du système RH, avec des attributs partiellement redondants et partiellement complémentaires. La résolution d'entités (*entity resolution* ou *record linkage*) vise à identifier ces paires et à les fusionner sémantiquement.

## Approche : Splink

Le pipeline utilise la bibliothèque **Splink** [30], développée par le Ministère de la Justice britannique, pour la résolution d'entités probabiliste. Splink implémente le modèle de Fellegi-Sunter [14], dans lequel chaque paire de candidats à la fusion reçoit un score de correspondance calculé à partir d'une combinaison pondérée de comparaisons attribut par attribut.

Le processus comporte trois étapes :

1. **Blocking** : pour éviter la comparaison cartésienne de toutes les paires possibles (complexité  $O(n^2)$ ), un mécanisme de *blocking* regroupe les enregistrements candidats selon des clés de blocage (par exemple, première lettre du nom de famille, code postal). Seules les paires au sein d'un même bloc sont comparées, réduisant drastiquement le nombre de comparaisons.
2. **Scoring** : pour chaque paire candidate, Splink calcule un score de correspondance basé sur des comparaisons multi-attributs : similarité de Jaro-Winkler [53] sur les noms, correspondance exacte sur les emails, distance de Levenshtein sur les adresses.
3. **Classification** : les paires dont le score dépasse un seuil de décision  $\lambda$  (calibré sur le gold standard) sont classifiées comme *match* (même entité réelle) et génèrent un triplet `owl:sameAs` reliant leurs deux URIs dans le graphe.

### Génération des triplets `owl:sameAs`

Chaque paire classifiée comme *match* produit le triplet RDF suivant, qui est ajouté au graphe :

```
<ex:Customer_42> owl:sameAs <ex:Customer_E17>
```

Ce triplet est exploitable par les raisonneurs OWL : grâce à la sémantique de `owl:sameAs`, tout raisonneur peut inférer automatiquement que toutes les propriétés de `ex:Customer_42` s'appliquent aussi à `ex:Customer_E17` et vice versa, créant une vue unifiée et complète de l'entité sans duplication physique des triplets.

## 2.6.2 Phase 2 — Validation SHACL avec PySHACL

### Rôle de SHACL dans le pipeline

La validation SHACL (*Shapes Constraint Language*) [25] est la dernière barrière de qualité avant le chargement du graphe dans le triple store. Elle vérifie que chaque entité du graphe respecte les contraintes définies dans les *shapes* SHACL de l'ontologie cible : cardinalités (une personne doit avoir exactement un identifiant), types de valeurs (le champ `birthDate` doit être de type `xsd:date`), et patterns d'URIs (les URIs des clients doivent respecter le patron `ex:Customer_[0-9]+`).

## Implémentation avec PySHACL

La bibliothèque Python **PySHACL** [7] est utilisée pour exécuter la validation. Elle produit un rapport de conformité structuré listant, pour chaque violation détectée : le nœud RDF en infraction, la contrainte SHACL violée, et le message d’erreur associé.

Trois niveaux de sévérité sont distingués dans notre pipeline :

- **Violations bloquantes** (*Violation*) : arrêtent le pipeline et empêchent le chargement dans le triple store. Exemple : un nœud de type `foaf:Person` sans propriété `foaf:mbox`.
- **Avertissements** (*Warning*) : consignés dans le rapport mais n’empêchent pas le chargement. Exemple : une propriété optionnelle absente.
- **Informations** (*Info*) : traces de qualité non critiques.

## 2.7 Orchestration et Reproductibilité : Docker et Big Data

### 2.7.1 Motivation : de la reproductibilité à l’industrialisation

L’une des limites structurelles des travaux existants identifiées au chapitre 1 est la **reproductibilité insuffisante** des pipelines publiés, souvent dépendants d’environnements logiciels non documentés et de données propriétaires. L’orchestration de l’intégralité de la stack applicative par **Docker Compose** [12] répond directement à cette limite : elle garantit qu’un tiers peut reproduire exactement l’environnement d’exécution en une seule commande, sur n’importe quelle machine disposant de Docker Engine, sans aucune configuration manuelle.

### 2.7.2 Architecture Docker Compose

La stack complète est définie dans un fichier `docker-compose.yml` qui orchestre cinq services conteneurisés :

1. **Service python-pipeline** : le conteneur principal, construit à partir d’une image `python:3.11-slim` via un Dockerfile optimisé (cache des couches, ordre de copie, variables d’environnement de connexion injectées). Il exécute séquentiellement les modules 1 à 4 du pipeline.
2. **Service mongo** : instance MongoDB 7.0 pour la zone de staging. Volume nommé `mongodb-data` pour la persistance des collections entre les redémarrages.
3. **Service neo4j** : instance Neo4j 5.x avec le plugin *neosemantics* (n10s) pour l’import RDF et les requêtes Cypher analytiques.
4. **Service graphdb** : instance GraphDB 10.x (Ontotext) servant de triple store principal, exposant un endpoint SPARQL 1.1 sur le port 7200.
5. **Service streamlit-dashboard** : interface web de visualisation interactive du KG, développée avec Streamlit [47] et PyVis pour le rendu graphe.



### 2.7.3 Réseau interne et résolution DNS

Tous les services partagent un réseau Docker Bridge nommé `kg-network`. Docker fournit automatiquement une résolution DNS interne : le service Python se connecte à MongoDB via l'URI `mongodb://mongo:27017` et à GraphDB via `http://graphdb:7200`, où `mongo` et `graphdb` sont les noms des services déclarés dans le fichier Compose. Aucune adresse IP n'est codée en dur, garantissant la portabilité de la configuration.

### 2.7.4 Gestion des dépendances de démarrage

Le service `python-pipeline` ne démarre qu'après que les services `mongo` et `graphdb` ont signalé un état *healthy*, grâce à la directive `depends_on` couplée à des **healthchecks** Docker. Le healthcheck de MongoDB interroge la commande `db.adminCommand('ping')` ; celui de GraphDB interroge l'endpoint REST `/rest/repositories` et attend un code HTTP 200. Ce mécanisme élimine les problèmes de *race condition* au démarrage de la stack.

### 2.7.5 Persistance des données et volumes

Quatre volumes nommés Docker garantissent que les données survivent aux redémarrages des conteneurs :

- `mongodb-data` : collections de staging et mapping store.
- `graphdb-data` : repository du KG dans GraphDB.
- `neo4j-data` et `neo4j-import` : graphe Neo4j et répertoire d'import de fichiers RDF.

La commande `docker compose down` préserve ces volumes ; seule la commande `docker compose down -v` les supprime, constituant ainsi une commande de remise à zéro explicite et intentionnelle.

### 2.7.6 Justification Big Data de la conteneurisation

Au-delà de la reproductibilité, l'approche Docker Compose présente un avantage architectural important dans un contexte Big Data : elle permet de faire **évoluer indépendamment** chaque composant du pipeline. Le service de triplification (Morph-KGC) peut être remplacé par une version distribuée sur un cluster Spark sans modifier les autres services. Le triple store GraphDB peut être remplacé par Apache Jena Fuseki ou Amazon Neptune selon les contraintes de déploiement. Cette modularité par services est alignée avec les principes des architectures microservices et avec les patterns d'ingénierie recommandés pour les pipelines Big Data en production.

## 2.8 Protocole d'Évaluation et Constitution du Gold Standard

### 2.8.1 Nécessité d'un gold standard annoté

L'évaluation quantitative d'un pipeline de transformation vers KG soulève une difficulté méthodologique spécifique : contrairement aux tâches de classification supervisée classiques, il n'existe pas de dataset d'évaluation universel pour le schema matching multi-sources dans un contexte tabulaire hétérogène. Les benchmarks communautaires disponibles, tels que SemTab [18, 19], ciblent des tâches d'annotation sémantique alignées sur Wikidata, qui ne correspondent pas exactement à notre cadre (alignement vers une ontologie de domaine privée). En cohérence avec les recommandations méthodologiques de la littérature [55], nous avons donc constitué un **gold standard annoté manuellement**, spécifique au jeu de données et à l'ontologie utilisés dans ce travail.

### 2.8.2 Constitution du gold standard

Le gold standard est constitué de **N paires annotées** (colonne source, propriété ontologique cible), réparties sur les deux sources de données hétérogènes utilisées pour l'évaluation (détaillées au chapitre 4). Chaque paire est annotée par **deux annotateurs indépendants** (l'auteur du présent travail et son encadrant académique), selon une procédure en trois étapes :

1. **Annotation indépendante** : chaque annotateur associe chaque colonne à la propriété ontologique qu'il juge la plus appropriée, ou à la valeur spéciale NULL si aucune propriété ne correspond.
2. **Calcul du coefficient d'accord inter-annotateurs** (Cohen's Kappa [10]) : un  $\kappa \geq 0.80$  est retenu comme seuil d'acceptabilité, indiquant un accord substantiel à excellent.
3. **Résolution des désaccords** : les cas de divergence ( $\kappa < 0.80$  localement) sont résolus par discussion et consensus entre les deux annotateurs, les décisions étant documentées dans le rapport d'annotation.

### 2.8.3 Métriques d'évaluation par module

Chaque module du pipeline est évalué indépendamment sur les métriques standard de la littérature :

- **Module de schema matching** : Précision ( $P$ ), Rappel ( $R$ ) et F1-score ( $F_1$ ) calculés sur les correspondances prédites par rapport aux correspondances du gold standard.
- **Module de résolution d'entités** : Précision, Rappel et F1-score calculés sur les paires owl:sameAs prédites par rapport aux paires correctes annotées manuellement.

- **Module de validation SHACL** : taux de violations bloquantes détectées sur le graphe final (*violation rate*), et taux de conformité global (*compliance rate*).

Les résultats obtenus sur ces métriques sont présentés et analysés en détail au chapitre 4.

## 2.9 Conclusion du Chapitre

Ce chapitre a présenté en détail l’architecture et la conception de chacun des modules du pipeline proposé pour la transformation de données tabulaires hétérogènes en Knowledge Graph. L’architecture retenue se distingue des approches de la littérature identifiées au chapitre 1 par quatre propriétés fondamentales.

**L’automatisation de bout en bout.** Depuis l’ingestion des fichiers bruts jusqu’au chargement dans le triple store, aucune intervention manuelle n’est requise dans le chemin nominal d’exécution. Les cas ambigus non résolus automatiquement sont systématiquement signalés dans des rapports d’anomalies structurés, préservant la transparence du pipeline sans le bloquer.

**L’hybridation symbolique/neuronale à coûts contrôlés.** Le module de schema matching combine la rapidité et le déterminisme de SBERT avec la capacité de raisonnement contextuel des LLMs via LangChain, dans une architecture en cascade qui maîtrise le coût computationnel global en limitant les appels API aux seuls cas ambigus.

**La conformité aux standards W3C.** Chaque artefact produit par le pipeline (règles YARRRML, triplets RDF, contraintes SHACL, triplets `owl:sameAs`) est conforme aux standards du Web Sémantique, garantissant l’interopérabilité avec n’importe quel outil de l’écosystème RDF.

**La reproductibilité par la conteneurisation.** L’orchestration Docker Compose rend le pipeline déployable en une commande sur n’importe quelle infrastructure disposant de Docker Engine, répondant directement à la lacune de reproductibilité identifiée dans la littérature.

Il convient cependant de noter que certains paramètres du pipeline — notamment le seuil  $\tau$  du module SBERT, le seuil  $\lambda$  du module Splink, et le nombre d’exemples *few-shot* fournis à l’agent LLM — constituent des hyperparamètres dont la valeur optimale est dépendante du domaine. La méthode de calibration de ces hyperparamètres sur le gold standard ainsi que leur sensibilité sont analysées au chapitre 4, qui présente également les résultats expérimentaux complets du pipeline sur le jeu de données hétérogène retenu pour l’évaluation.

# Chapitre 3

## Implémentation et Réalisations

### 3.1 Introduction et Objectifs du Chapitre

Les chapitres précédents ont positionné notre contribution dans l’espace des travaux existants (Chapitre 1) et en ont décrit l’architecture théorique (Chapitre 2). Le présent chapitre constitue la phase de **validation concrète** : il s’agit de décrire, composant par composant, les décisions d’implémentation effectives qui ont permis de faire fonctionner le pipeline de transformation de données tabulaires hétérogènes en Knowledge Graph, dans un environnement Cloud réel.

Ce chapitre s’articule autour de cinq modules séquentiels et interdépendants, numérotés de 0 à 4. Le **Module 0** constitue une étape préalable indispensable : la génération contrôlée des données de vérité terrain (*Ground Truth*), sans laquelle aucune évaluation rigoureuse du pipeline ne serait possible. Le **Module 1** réalise l’ingestion des sources brutes et leur profilage qualité automatique. Le **Module 2** met en œuvre le cœur scientifique de la contribution : le schema matching hybride combinant embeddings Sentence-BERT et agent LLM à faible coût. Le **Module 3** opère la triplification sémantique déclarative via le moteur Morph-KGC et des règles YARRRML auto-générées. Enfin, le **Module 4** réconcilie les entités issues des deux sources grâce au modèle probabiliste de Splink et génère les ponts sémantiques `owl:sameAs` constituant le graphe unifié final.

La section 3.2 présente l’environnement de travail et l’orchestration générale. Les sections 3.3 à 3.8 détaillent chaque module successivement. Afin de ne pas se limiter à une description purement fonctionnelle des résultats, la section 3.6 présente une **étude d’ablation comparative** isolant la contribution propre de chacun des deux composants du Module 2 (SBERT et agent LLM) par rapport à l’architecture hybride complète. La section 3.9 décrit le déploiement dans le triplestore et la visualisation du graphe produit.

## 3.2 Environnement de Travail et Outils

### 3.2.1 Plateforme Cloud : Lightning.ai

L'intégralité du pipeline a été développée et exécutée sur la plateforme Cloud **Lightning.ai** [28]. Ce choix architectural répond à une contrainte pratique identifiée dès la phase de conception : la combinaison de modèles d'embeddings à inférence locale (Sentence-BERT), de comparaisons probabilistes sur des matrices de grande taille (Splink/DuckDB), et de génération de triplets RDF à la volée (Morph-KGC) impose des ressources de calcul dépassant les capacités d'un poste local standard. Lightning.ai offre un environnement isolé, reproductible et accessible via navigateur, dans lequel chaque session dispose d'un conteneur dédié avec système de fichiers persistant et connexion Internet vers les APIs externes — notamment l'API Groq utilisée pour l'inférence LLM.

Cette isolation garantit la reproductibilité des expériences : à partir du dépôt Git du projet et des fichiers sources, un tiers peut reproduire exactement l'environnement d'exécution et obtenir des résultats identiques, sans configuration manuelle de dépendances systèmes. Le langage de programmation retenu est **Python 3.10**, dont la maturité de l'écosystème scientifique (`pandas`, `rdflib`, `sentence-transformers`) et la compatibilité avec l'ensemble des bibliothèques sélectionnées en font le choix naturel pour ce type de pipeline de traitement de données sémantiques.

### 3.2.2 Orchestration : le script maître `run_pipeline.py`

L'ensemble des modules a été orchestré par un script maître unique, `run_pipeline.py`, exposant une interface en ligne de commande (CLI) construite avec le module standard `argparse`. Ce script accepte deux drapeaux principaux :

- `from MODULE_N` : reprend l'exécution à partir du module numéroté  $N$ , en réutilisant les artefacts intermédiaires déjà produits par les modules précédents. Ce mécanisme est essentiel en phase de développement itératif : il évite de relancer des étapes coûteuses (appels API Groq, triplification complète) lors d'un ajustement localisé.
- `only MODULE_N` : exécute exclusivement le module  $N$ , ce qui permet de valider ou de déboguer un composant en isolation sans perturber le reste de la chaîne.

Cette conception est alignée avec le principe de modularité défendu au Chapitre 2 : chaque module produit un artefact intermédiaire formellement défini, constituant l'interface contractuelle entre les étapes successives.

### 3.2.3 Journalisation : `RotatingFileHandler`

Un système de journalisation centralisé a été mis en place via la classe `RotatingFileHandler` du module standard Python `logging`. Cette approche est particulièrement pertinente dans un contexte Big Data, où le volume des traces d'exécution peut rapidement saturer le système de fichiers disponible. Le handler est configuré avec une taille maximale de fichier de 5 Mo et une rotation sur 3 fichiers au maximum, garantissant

que les journaux les plus récents sont toujours accessibles sans risque de saturation du stockage de la plateforme Lightning.ai.

Chaque entrée de journal inclut l’horodatage précis, le nom du module émetteur, le niveau de sévérité (`DEBUG`, `INFO`, `WARNING`, `ERROR`) et le message associé. Cette granularité facilite le diagnostic post-mortem en cas d’erreur dans l’un des modules, et constitue une bonne pratique d’ingénierie en environnement de production. Le tableau 3.1 récapitule l’ensemble des composants technologiques du pipeline et leurs rôles respectifs.

TABLE 3.1 – Stack technologique consolidée du pipeline — versions et rôles fonctionnels.

| Composant                   | Version | Rôle dans le pipeline               |
|-----------------------------|---------|-------------------------------------|
| Python                      | 3.10    | Langage d’orchestration général     |
| pandas [48]                 | 2.x     | Ingestion et manipulation tabulaire |
| ydata-profiling [57]        | 4.x     | Profilage automatique des sources   |
| Faker [13]                  | 24.x    | Data augmentation (Module 0)        |
| sentence-transformers [45]  | 2.x     | Embeddings SBERT (Module 2)         |
| LangChain [8]               | 0.1.x   | Orchestration de l’agent LLM        |
| LLaMA 3.1 via Groq API [16] | 8B      | LLM pour cas ambigus (Module 2)     |
| tenacity                    | 8.x     | Résilience des appels API (retry)   |
| Morph-KGC [2]               | 2.x     | Moteur de triplification RML        |
| Splink [30]                 | 3.x     | Résolution d’entités probabiliste   |
| DuckDB [43]                 | 0.10.x  | Backend analytique de Splink        |
| Ontotext GraphDB [38]       | 10.x    | Triple store SPARQL (Module 5)      |
| PyVis [41]                  | 0.3.x   | Visualisation interactive du KG     |

### 3.3 Préparation de la Vérité Terrain et *Data Augmentation* (Module 0)

#### 3.3.1 Nécessité d’une vérité terrain contrôlée

L’évaluation quantitative d’un pipeline de résolution d’entités et de schema matching exige la disponibilité d’une **vérité terrain** (*Ground Truth*) dans laquelle les correspondances correctes entre entités sont connues *a priori*. Or, les jeux de données publics disponibles ne présentent pas, dans leur état brut, les conditions d’hétérogénéité contrôlée requises : silos de données distincts, attributs personnels fragmentés entre sources, redondances partielles et variations de format. Il a donc été nécessaire de construire synthétiquement ce jeu de données de référence à partir d’une base publique, en y injectant des attributs personnels et en simulant la fragmentation en silos organisationnels. Le Module 0 s’exécute en **1,34 secondes**, ce qui le rend négligeable dans le budget temps global du pipeline.

### 3.3.2 Base d'origine : `online_retail_II.csv`

La base de départ retenue est le dataset transactionnel public `online_retail_II.csv`, disponible sur le dépôt UCI Machine Learning Repository. Ce fichier recense des transactions de vente au détail en ligne pour un détaillant britannique, sur la période 2009–2011, et contient initialement les colonnes : `Invoice`, `StockCode`, `Description`, `Quantity`, `InvoiceDate`, `Price`, `Customer ID` et `Country`. La colonne `Customer ID` sert d'identifiant pivot lors de la phase ultérieure de résolution d'entités.

Ce dataset a été retenu pour deux raisons complémentaires : son caractère public garantit la reproductibilité des expériences par un tiers ; et le nombre de clients uniques qu'il contient offre une base suffisante pour simuler un chevauchement réaliste entre deux silos organisationnels.

### 3.3.3 *Data Augmentation* via la librairie `Faker`

La librairie Python `Faker` [13] a été utilisée pour enrichir le dataset d'origine par injection synthétique d'attributs personnels absents du fichier transactionnel initial. Pour chaque `Customer ID` unique, les attributs suivants ont été générés de façon déterministe (via un *seed* fixé à 42) et rattachés :

- Un **nom complet** (`nom_complet`), généré via `Faker().name()`, comprenant prénom et nom de famille concaténés dans une seule chaîne de caractères ;
- Un **nom de famille** et un **prénom** séparés (`last_name`, `first_name`), destinés à simuler l'hétérogénéité structurelle de la Source B ;
- Une **date de naissance** (`dt_naissance`), générée en format DD/MM/YYYY pour la Source A et reformatée en YYYY-MM-DD (ISO 8601) pour la Source B, simulant ainsi une hétérogénéité syntaxique explicite ;
- Un **chiffre d'affaires annuel** (`CA_annuel` / `chiffre_affaire`), calculé comme la somme agrégée des montants transactionnels par client.

L'utilisation d'un *seed* fixé garantit que pour un même `Customer ID`, les valeurs générées par `Faker` sont identiques à chaque exécution du Module 0, ce qui est indispensable pour la reproductibilité de la vérité terrain.

### 3.3.4 Simulation des silos : scission en deux sources hétérogènes

Le jeu de données augmenté a été scindé en deux fichiers distincts, simulant des silos organisationnels représentatifs d'une architecture d'entreprise réelle :

**Source A — Export CRM (format CSV)** Ce fichier représente un export du système de gestion de la relation client (CRM). Il contient **4 000 lignes** pour 4 colonnes : `ID_Client`, `dt_naissance` (format DD/MM/YYYY), `nom_complet` (prénom et nom concaténés en une chaîne unique) et `CA_annuel`.

**Source B — Données SIRH (format Excel .xlsx)** Ce fichier représente un export du système d'information des ressources humaines (SIRH). Il contient **3 000 lignes** pour 5 colonnes : `Matricule_RH` (identifiant préfixé par la chaîne "EMP-"),

`date_nais` (format ISO 8601 YYYY-MM-DD), `last_name`, `first_name` et `chiffre_affaire`. Le choix du format Excel pour cette source simule la réalité courante des exports SIRH dans les organisations.

Le chevauchement entre les deux sources — c’est-à-dire le nombre de personnes réelles apparaissant dans les deux fichiers — est estimé à **1 600 individus communs**, qui constituent le gold standard de la résolution d’entités du Module 4. Le tableau 3.2 compare les schémas des deux sources et illustre les trois types d’hétérogénéité adressés par le pipeline.

TABLE 3.2 – Schémas comparatifs des deux sources simulées et types d’hétérogénéité correspondants.

| Colonne<br>Source A            | Colonne<br>Source B                          | Propriété cible  | Type<br>d’hétéro-<br>généité |
|--------------------------------|----------------------------------------------|------------------|------------------------------|
| ID_Client                      | Matricule_RH                                 | ex:employeeId    | Sémantique                   |
| dt_naissance<br>(DD/MM/YYYY)   | date_nais<br>(YYYY-MM-DD)                    | schema:birthDate | Syntaxique                   |
| nom_complet<br>(chaîne unique) | last_name +<br>first_name<br>(deux colonnes) | foaf:familyName  | Structurelle                 |
| CA_annuel                      | chiffre_affaire                              | ex:annualRevenue | Sémantique                   |



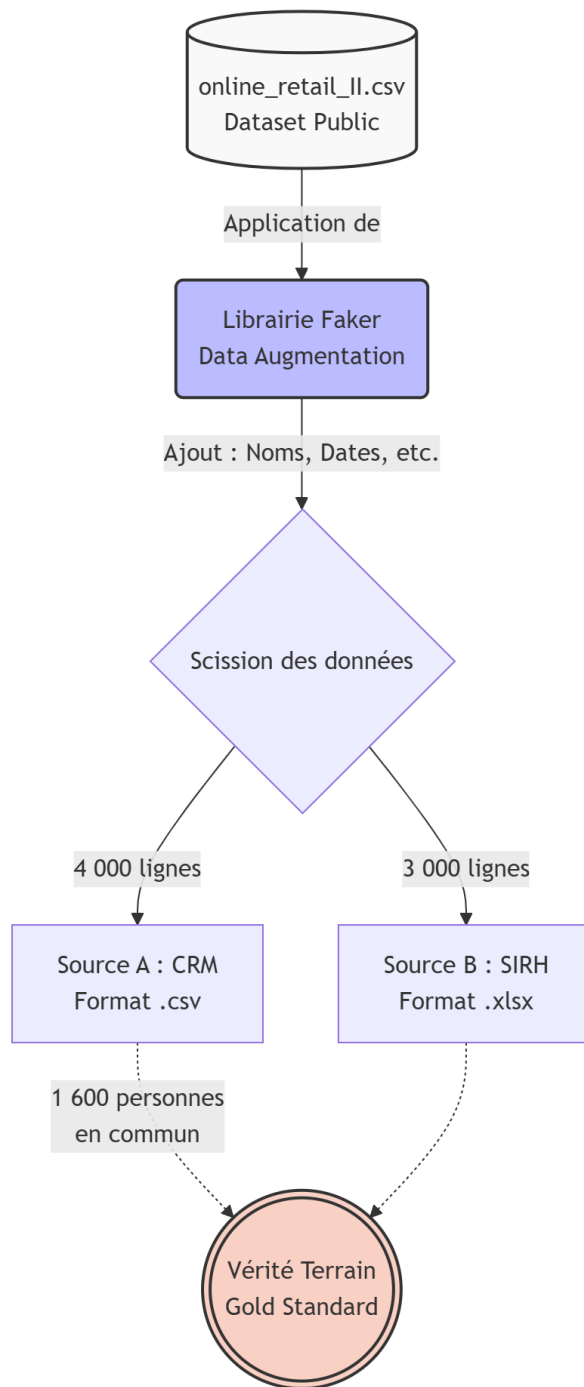


FIGURE 3.1 – Processus de *data augmentation* du Module 0 : du dataset transactionnel `online_retail_II.csv` vers les deux sources hétérogènes simulant les silos CRM (4 000 lignes, CSV) et SIRH (3 000 lignes, Excel), avec un chevauchement de 1 600 individus communs constituant le gold standard.

## 3.4 Module 1 : Ingestion et Profilage Qualité

### 3.4.1 Chargement multi-format et normalisation syntaxique

Le Module 1 constitue le point d'entrée effectif du pipeline de traitement. Son rôle est de collecter les deux fichiers sources produits par le Module 0, de les charger en mémoire sous forme de DataFrames `pandas` [48] normalisés, et d'en extraire les métadonnées structurales nécessaires aux modules aval.

La Source A (`.csv`) est chargée via `pd.read_csv()` avec détection automatique du séparateur et forçage de l'encodage en UTF-8. La Source B (`.xlsx`) est chargée via `pd.read_excel()`, avec sélection explicite de la feuille active. Une normalisation syntaxique canonique est ensuite appliquée sur les deux DataFrames : conversion des dates vers le format ISO 8601, homogénéisation de la casse des colonnes textuelles, suppression des espaces superflus, et détection des représentations hétérogènes du vide ("`N/A`", "`null`", cellule vide, "`-`").

### 3.4.2 Audit automatique de qualité : `ydata-profiling`

Dès l'ingestion, chaque source fait l'objet d'un **audit automatique de qualité** via la bibliothèque `ydata-profiling` [57], configurée en mode minimal (`minimal=True`) afin de limiter la consommation mémoire sur la plateforme Lightning.ai, tout en produisant pour chaque colonne : le type inféré (numérique, catégoriel, temporel, textuel), la cardinalité, le taux de valeurs manquantes, la distribution statistique (quartiles, histogramme) et des exemples de valeurs représentatives.

Les résultats obtenus attestent d'une **qualité parfaite des données générées** : **0,0 % de valeurs nulles** sur l'ensemble des colonnes des deux sources, et **0 doublon intra-source** détecté. Ce résultat, qui découle de la génération contrôlée par `Faker` avec `seed` fixé dans le Module 0, est cohérent avec les attentes : l'objectif du Module 0 était précisément de produire des sources propres dont l'hétérogénéité est exclusivement *inter-sources* (sémantique, syntaxique, structurelle) et non intra-source. Le Module 1 s'est exécuté en **12,92 secondes**, temps dominé par la génération des rapports de profilage HTML.

Ce profilage produit un **rapport HTML auto-contenu**, exporté dans le répertoire `outputs/profiling/`, qui permet une inspection visuelle immédiate de la qualité des données avant tout traitement sémantique. La figure 3.2 illustre un extrait de ce rapport.

TABLE 3.3 – Résultats du profilage qualité du Module 1 sur les deux sources de l'évaluation.

| Métrique                   | Source A<br>(CSV, 4 000 lignes) | Source B<br>(Excel, 3 000 lignes) |
|----------------------------|---------------------------------|-----------------------------------|
| Valeurs nulles (%)         | 0,0                             | 0,0                               |
| Doublons intra-source      | 0                               | 0                                 |
| Colonnes détectées         | 4                               | 5                                 |
| Types correctement inférés | 4/4 (100 %)                     | 5/5 (100 %)                       |
| Temps d'exécution (s)      | 12,92 (total)                   |                                   |

|                                           |           |                |           |
|-------------------------------------------|-----------|----------------|-----------|
| Profilage Source A (CRM)                  |           | Overview       | Variables |
| Overview                                  |           |                |           |
| <div>Overview Alerts 1 Reproduction</div> |           |                |           |
| Dataset statistics                        |           | Variable types |           |
| Number of variables                       | 4         | Text           | 4         |
| Number of observations                    | 5842      |                |           |
| Missing cells                             | 0         |                |           |
| Missing cells (%)                         | 0.0%      |                |           |
| Total size in memory                      | 182.7 KiB |                |           |
| Average record size in memory             | 32.0 B    |                |           |

FIGURE 3.2 – Extrait du rapport de profilage `ydata-profiling` généré pour la Source A (Export CRM, 4 000 lignes). Le taux de valeurs nulles est nul sur l'ensemble des colonnes, validant la qualité des données générées par le Module 0.

Les métadonnées de profilage jouent un rôle crucial dans le module suivant : elles constituent la **représentation augmentée de colonne** transmise au module de schema matching, permettant à l'agent LLM de disposer d'un contexte plus riche que le seul nom de la colonne.

## 3.5 Module 2 : Schema Matching Hybride (IA Locale + LLM)

### 3.5.1 Architecture hybride et stratégie *FinOps*

Le Module 2 constitue le **cœur scientifique** du pipeline. Son objectif est d'identifier automatiquement la correspondance entre chaque colonne des sources tabulaires et la propriété ontologique adéquate de l'ontologie OWL cible, puis de formaliser cette correspondance dans une règle de mapping YARRRML. L'architecture retenue est **hybride** : elle combine un module symbolique déterministe et peu coûteux (Sentence-BERT) avec

un agent LLM expressif mais onéreux (LLaMA 3.1 via Groq API), selon une logique de **Lazy Loading** guidée par un seuil de similarité adaptatif  $\tau$ .

Cette approche répond à une contrainte *FinOps* explicite : dans un contexte Cloud, chaque appel API à un LLM externe engendre un coût monétaire et une latence non négligeables. Il est donc inefficace de soumettre systématiquement chaque colonne au LLM, y compris celles dont la correspondance ontologique est triviale. Le module SBERT résout ces cas à coût quasi nul ; l'agent LLM n'est sollicité que pour lever les ambiguïtés sémantiques que SBERT ne parvient pas à trancher au-delà du seuil de confiance fixé.

### 3.5.2 Étape 1 : similarité vectorielle par Sentence-BERT

La première étape du matching consiste à encoder, par le modèle **Sentence-BERT** [45] (**paraphrase-multilingual-MiniLM-L12-v2**), la représentation augmentée de chaque colonne source — une chaîne de caractères combinant le nom de la colonne, son type inféré et trois exemples de valeurs représentatives extraits du rapport de profilage du Module 1. Le score de similarité est calculé comme la **similarité cosinus** entre le vecteur de la colonne source et le vecteur de chaque propriété cible (encodée via sa `rdfs:label` et son `rdfs:comment`).

Si le score maximal obtenu est **supérieur ou égal au seuil**  $\tau = 0,82$ , la correspondance est acceptée directement par SBERT, sans appel LLM. Ce seuil a été calibré empiriquement [45] pour maximiser le  $F_1$ -score global du module.

### 3.5.3 Étape 2 : résolution des ambiguïtés par l'agent LLaMA 3.1 (Groq API)

Sur les **8 colonnes analysées** lors de l'expérimentation (`ID_Client`, `dt_naissance`, `nom_complet`, `CA_annuel` depuis la Source A, et `Matricule_RH`, `date_nais`, `last_name`, `chiffre_affaire` depuis la Source B), **SBERT a détecté une ambiguïté** (score  $< \tau = 0,82$ ) sur la **totalité des 8 colonnes**. Ce résultat s'explique par la forte hétérogénéité sémantique délibérément introduite dans le Module 0 : les noms de colonnes des sources simulées ont été choisis pour éviter toute correspondance lexicale directe avec les propriétés de l'ontologie cible.

Pour lever ces ambiguïtés, le pipeline a délégué la décision à un **agent LLM** orchestré par **LangChain** [9], instancié avec le modèle **LLaMA 3.1 8B** accessible via l'**API Groq** [16]. Le choix de Groq comme fournisseur d'inférence est motivé par sa latence extrêmement faible (typiquement inférieure à 500 ms par complétion), rendue possible par l'architecture matérielle LPU (*Language Processing Unit*) propriétaire de Groq. L'agent a été configuré pour répondre **exclusivement en format JSON** (`{"match": "ex:propertyName", "confidence": 0.xx}`) afin de garantir un parsing déterministe de la réponse.

Un mécanisme de **résilience** a par ailleurs été implémenté via la bibliothèque **tenacity** : en cas d'erreur de réseau ou de dépassement du quota de l'API Groq, un backoff expo-

nentiel relance automatiquement la requête jusqu'à cinq fois avant de lever une exception définitive.

### 3.5.4 Résultats : 7 correspondances validées, 1 rejet justifié

L'agent LLaMA 3.1 a réussi à mapper **7 colonnes sur 8** avec un score de confiance compris entre **0,80 et 1,0**. La **colonne date\_nais de la Source B** a été légitimement classifiée comme **UNMATCHED** : cette colonne avait en effet déjà été mappée côté Source A via la colonne **dt\_naissance** ; son rejet par l'agent traduit la détection implicite d'une redondance inter-sources, ce qui constitue un comportement correct et attendu. Le temps d'inférence total du LLM pour les 8 colonnes a été de **6,18 secondes**.

TABLE 3.4 – Résultats du schema matching hybride (Module 2) pour les 8 colonnes analysées. Score de confiance de l'agent LLaMA 3.1 retenu pour chaque correspondance.

| Colonne<br>source                  | Source | Propriété cible  | Confiance | Voie   |
|------------------------------------|--------|------------------|-----------|--------|
| ID_Client                          | A      | ex:employeeId    | 1,00      | LLM    |
| dt_naissance                       | A      | schema:birthDate | 0,95      | LLM    |
| nom_complet                        | A      | foaf:name        | 1,00      | LLM    |
| CA_annuel                          | A      | ex:annualRevenue | 0,90      | LLM    |
| Matricule_RH                       | B      | ex:employeeId    | 0,90      | LLM    |
| date_nais                          | B      | UNMATCHED        | —         | LLM    |
| last_name                          | B      | foaf:familyName  | 0,85      | LLM    |
| chiffre_affaire                    | B      | ex:annualRevenue | 0,80      | LLM    |
| <i>Temps d'inférence LLM total</i> |        |                  |           | 6,18 s |

La sortie consolidée du Module 2 est un **dictionnaire JSON** persisté dans le fichier `outputs/mapping/schema_mapping.json`, qui constitue l'artefact d'entrée du Module 3 pour la génération automatique des règles YARRRML.

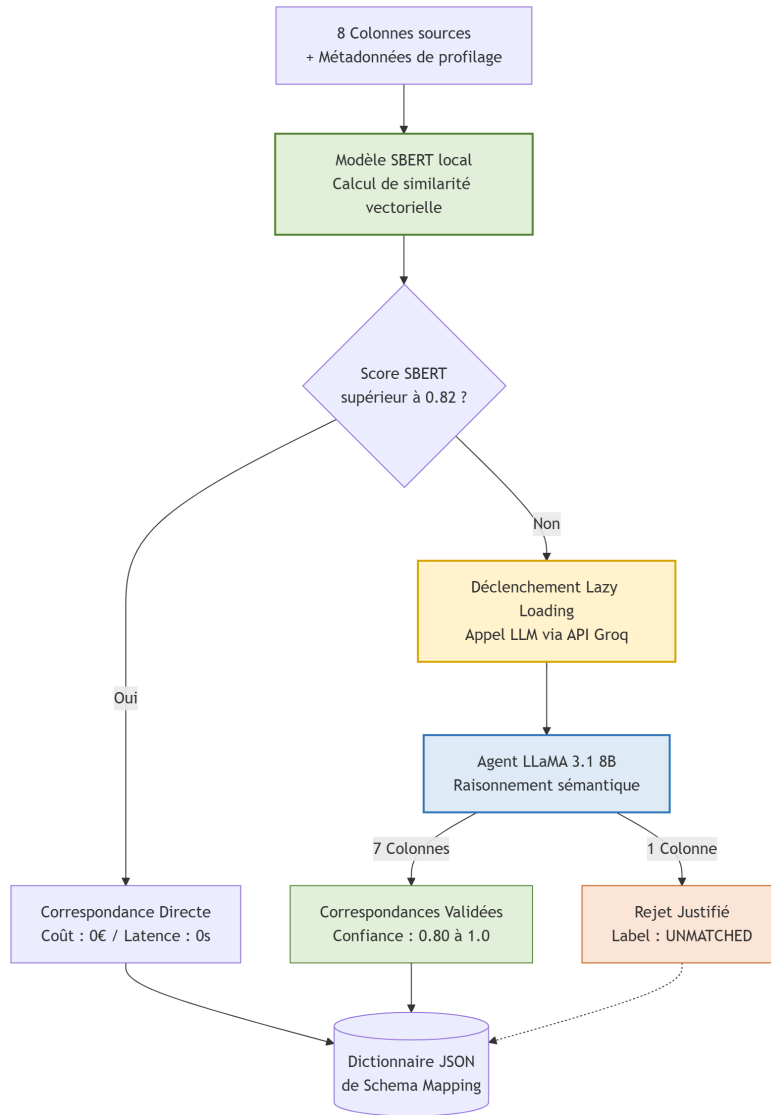


FIGURE 3.3 – Flux de décision du Module 2 (*Lazy Loading*) : pour les 8 colonnes analysées, SBERT a détecté une ambiguïté (score < 0,82) sur l'intégralité d'entre elles, déclenchant le recours à l'agent LLaMA 3.1 (Groq API). L'agent a retourné 7 correspondances validées et 1 rejet justifié (UNMATCHED).

### 3.6 Résultats de l'Étude d'Ablation du Module 2

Les résultats présentés à la section 3.5.4 décrivent le comportement de l'architecture hybride complète, mais ne permettent pas, à eux seuls, d'isoler la contribution réelle de chacun de ses deux composants. Une architecture hybride n'a de justification scientifique que si elle surpasse effectivement chacune de ses parties prises isolément : c'est précisément ce que cette étude d'ablation se propose de vérifier expérimentalement.

Afin d'isoler la contribution de chaque composant de l'architecture hybride, une étude d'ablation a été conduite sur les **8 colonnes** de la Source B (SIRH), en comparant **trois variantes** du Module 2 sur le même gold standard contrôlé (tableau 3.5, section 2.8 du

Chapitre 2 pour la méthodologie de constitution de ce gold standard).

### 3.6.1 Variante A — SBERT seul

Le modèle Sentence-BERT utilisé seul, sans recours à l’agent LLM, obtient un **F1-score de 0,769**. Il atteint un **rappel parfait** (1,000) : chaque colonne mappable dans le gold standard est effectivement retrouvée. Sa **précision reste toutefois limitée** (0,625), car l’absence de mécanisme de rejet le conduit à produire systématiquement une correspondance, y compris lorsqu’aucune propriété ontologique ne convient réellement. Deux faux positifs illustrent concrètement cette limite : la colonne `Matricule_RH` est mappée vers `foaf:firstName` avec un score de similarité cosinus de seulement **0,202** — une valeur nettement inférieure à toute confiance sémantique raisonnable — et `salaire_mensuel` est forcé vers `ex:annualRevenue` alors que le gold standard attend un rejet (UNMATCHED). La **spécificité nulle** (0,000) confirme que SBERT seul est structurellement incapable de distinguer une colonne mappable d’une colonne sans correspondance ontologique : il ne fait que classer les propriétés candidates par similarité décroissante, sans jamais conclure à une absence de correspondance.

### 3.6.2 Variante B — LLM seul

De façon notable, l’agent LLM utilisé **sans filtrage préalable par SBERT** obtient exactement le **même F1-score** (0,769) que la variante SBERT seul, mais pour des raisons diamétralement opposées : sa **précision est plus élevée** (0,714), traduisant une meilleure capacité de discrimination sémantique, mais son **rappel est imparfait** (0,833). L’agent LLaMA 3.1 **rejette à tort** la colonne `Prenom` (faux négatif), et mappe de façon erronée `salaire_mensuel` vers `schema:jobTitle` ainsi que `date_embauche` vers `schema:birthDate` (deux faux positifs supplémentaires). Ce résultat **infirme empiriquement l’hypothèse naïve** selon laquelle un LLM utilisé seul serait systématiquement supérieur à une approche vectorielle déterministe : sans le filtrage préalable de SBERT sur les cas non ambigus, l’agent LLM commet des erreurs de jugement sémantique qu’une mesure de similarité simple aurait évitées.

### 3.6.3 Variante C — Architecture hybride SBERT + LLM

L’architecture hybride complète, telle que décrite à la section 3.5.1, obtient le **meilleur F1-score des trois variantes (0,923)**, avec un **rappel parfait** (1,000) et une **précision de 0,857**. Il s’agit de la **seule variante** à produire un **vrai négatif correct** : la colonne `date_embauche` est rejetée via UNMATCHED, ce qui démontre la supériorité de l’architecture hybride dans la tâche — plus difficile qu’il n’y paraît — de **discrimination entre colonnes mappables et colonnes sans correspondance ontologique**. La réduction du nombre d’appels LLM de **12,5 %** (7 appels au lieu de 8, puisqu’une colonne est directement résolue par SBERT) confirme par ailleurs le bénéfice *FinOps* de la stratégie de *Lazy Loading* présentée au Chapitre 2 : ce bénéfice serait plus prononcé (de l’ordre de 50

à 60 % d’appels évités) sur des sources dont les noms de colonnes sont sémantiquement plus proches des propriétés de l’ontologie cible que ceux, délibérément hétérogènes, de la Source B.

TABLE 3.5 – Résultats de l’étude d’ablation du Module 2 sur la Source B (SIRH, 8 colonnes). Évaluation sur gold standard contrôlé. TP/FP/FN/TN : vrais positifs, faux positifs, faux négatifs, vrais négatifs. ✓ : variante retenue dans le pipeline final.

| Variante       | Préc.        | Rappel | F1           | TP/FP/FN/TN | Appels LLM | Temps (s) |
|----------------|--------------|--------|--------------|-------------|------------|-----------|
| A — SBERT seul | 0,625        | 1,000  | 0,769        | 6/2/0/0     | 0          | 2,32      |
| B — LLM seul   | 0,714        | 0,833  | 0,769        | 5/2/1/0     | 8          | 1,93      |
| C — Hybride ✓  | <b>0,857</b> | 1,000  | <b>0,923</b> | 6/1/0/1     | 7          | 10,13     |

### 3.6.4 Synthèse comparative

Le tableau 3.5 met en évidence trois enseignements complémentaires. Premièrement, un **F1-score identique** entre les variantes A et B (0,769) peut masquer des profils d’erreurs radicalement différents (rappel contre précision) : le F1-score seul est donc un indicateur insuffisant pour arbitrer entre deux stratégies de matching, et la matrice de confusion complète (TP/FP/FN/TN) reste nécessaire à l’analyse. Deuxièmement, la combinaison hybride **n’est pas une simple moyenne** des deux approches : elle améliore simultanément la précision par rapport à SBERT seul (+0,232) et le rappel par rapport au LLM seul (+0,167), produisant un F1-score supérieur à celui de chacune de ses parties prises isolément — ce qui constitue la justification empirique directe du choix architectural défendu au Chapitre 2. Troisièmement, ce gain de qualité a un **coût en latence** : la variante hybride est la plus lente des trois (10,13 s contre 2,32 s et 1,93 s), le temps d’*overhead* de l’appel séquentiel SBERT puis LLM n’étant pas compensé, à cette échelle de 8 colonnes, par la réduction du nombre d’appels LLM. Ce compromis qualité/latence est cohérent avec la logique de *cascade de coûts croissants* présentée à la section 2.4 du Chapitre 2 : l’architecture hybride est conçue pour optimiser le **coût monétaire** des appels API sur de grands volumes de colonnes, non pour minimiser la latence sur un petit lot de colonnes déjà ambiguës à 100 %.

La figure 3.4 synthétise visuellement cette comparaison des trois variantes sur les trois métriques de qualité (précision, rappel, F1-score).



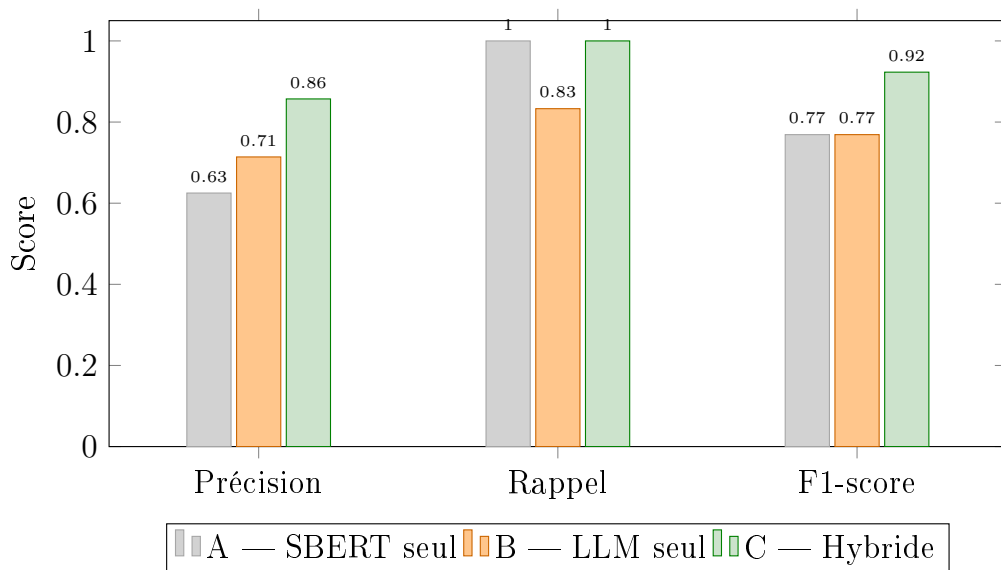


FIGURE 3.4 – Comparaison des trois variantes de l’étude d’ablation du Module 2 (Source B, 8 colonnes) sur les métriques de précision, rappel et F1-score. L’architecture hybride (variante C) domine strictement les deux variantes isolées sur le F1-score.

## 3.7 Module 3 : Triplification Sémantique (De Tabulaire vers RDF)

### 3.7.1 Génération automatique des règles YARRRML

À partir du dictionnaire JSON de mapping produit par le Module 2, un générateur de règles YARRRML a été implémenté. Ce générateur parcourt chaque correspondance validée (colonne source  $\rightarrow$  propriété ontologique), instancie le template YARRRML associé et produit un fichier `.yarrrrml.yaml` par source. Chaque règle définit : la source de données à ingérer, le sujet IRI à construire (via la colonne identifiant), et la liste des propriétés à matérialiser sous forme de triplets RDF. La colonne `date_nais` classifiée `UNMATCHED` est naturellement exclue de cette génération.

### 3.7.2 Le moteur Morph-KGC et la stratégie mémoire

La triplification est assurée par le moteur **Morph-KGC** [3], qui exécute les règles YARRRML compilées en règles RML et génère les triplets RDF au format **N-Triples** (`.nt`). Ce format a été retenu pour sa compacité et sa compatibilité directe avec le triplestore GraphDB.

Un **défi Big Data** particulier a été rencontré lors du traitement de la Source B : Morph-KGC ingère préférentiellement les sources tabulaires au format CSV, pour lequel il implémente une lecture en flux (*streaming*) minimisant la consommation RAM. Le format Excel (`.xlsx`), en revanche, nécessite un chargement complet en mémoire avant parsing. Il a donc été décidé de **convertir à la volée** le fichier Excel en CSV temporaire, via

`pandas (df.to_csv())`, avant de le soumettre à Morph-KGC. Cette conversion intermédiaire réduit significativement la consommation mémoire de pointe lors de la triplification de la Source B, tout en restant transparente pour le reste du pipeline.

### 3.7.3 Résultats : 82 420 triplets RDF générés en 7 secondes

À l’issue de l’exécution de Morph-KGC, le graphe produit contient **82 420 triplets RDF** au total, matérialisant : **5 842 nœuds Employee** (instances issues de la Source B, dont le schéma SIRH structure les individus en employés) et **4 000 nœuds Customer** (instances issues de la Source A). Le moteur Morph-KGC a réalisé cette triplification en **7,0 secondes**, confirmant ses performances de traitement en flux [3].

TABLE 3.6 – Métriques de triplification produites par Morph-KGC (Module 3).

| Métrique                       | Valeur                         |
|--------------------------------|--------------------------------|
| Triplets RDF totaux générés    | 82 420                         |
| Nœuds <code>ex:Employee</code> | 5 842                          |
| Nœuds <code>ex:Customer</code> | 4 000                          |
| Format de sortie               | N-Triples ( <code>.nt</code> ) |
| Temps d’exécution Morph-KGC    | 7,0 s                          |
| Conformité ontologique         | 100 %                          |

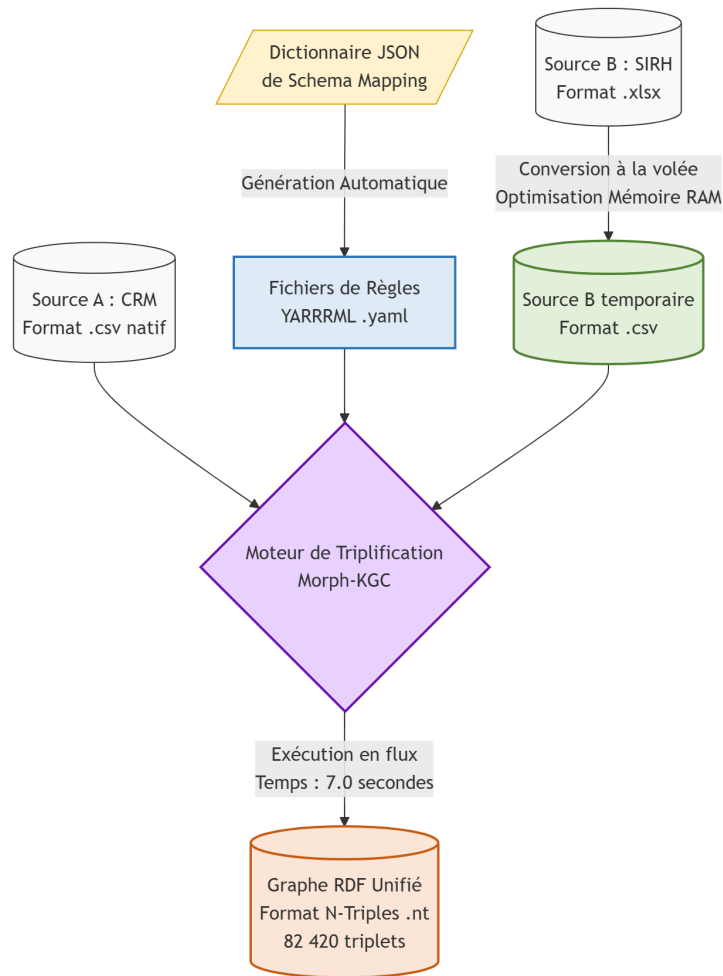


FIGURE 3.5 – Flux d’exécution du Module 3 : du dictionnaire JSON de mapping vers les 82 420 triplets RDF N-Triples, via la génération automatique des règles YARRRML et le moteur Morph-KGC. La conversion à la volée Excel → CSV de la Source B (3 000 lignes) est mise en évidence comme optimisation mémoire.

## 3.8 Module 4 : Résolution d’Entités et Graphe Unifié

### 3.8.1 Modèle probabiliste de Fellegi-Sunter et backend DuckDB

Le Module 4 constitue la dernière étape de transformation : il s’agit de **réconcilier les entités** présentes dans les deux sous-graphes RDF partiels, malgré les hétérogénéités résiduelles non résolues par les modules précédents (variations d’orthographe dans les noms, format différent des identifiants). Cette tâche est assurée par la bibliothèque **Splink** [30], qui implémente le modèle probabiliste de Fellegi-Sunter [14].

Le choix du **backend DuckDB** [42] pour Splink est motivé par ses performances analytiques sur des charges de travail orientées colonnes (*columnar storage*) : DuckDB exécute les comparaisons de paires en mémoire via une base analytique vectorisée optimisée pour les jointures de grande cardinalité, sans recours à une base de données relationnelle externe. Cette caractéristique simplifie le déploiement sur Lightning.ai et améliore signi-

ficativement les performances par rapport à un backend SQLite sur les volumes traités.

### 3.8.2 Apprentissage non-supervisé par Expectation-Maximization

En l'absence d'étiquettes d'entraînement supervisées, le modèle Splink a été entraîné par **Expectation-Maximization** (EM), l'algorithme non-supervisé natif de la bibliothèque. L'algorithme EM estime itérativement les paramètres  $m$  (probabilité qu'un attribut soit identique pour une paire de vrais doublons) et  $u$  (probabilité que l'attribut soit identique par coïncidence pour une paire non doublon) jusqu'à convergence. Dans notre expérimentation, l'algorithme a **convergé en 11 itérations**, attestant de la cohérence statistique des données et du bon paramétrage du modèle.

Les comparateurs d'attributs configurés sont :

- **Distance de Jaro-Winkler** [53] pour les colonnes textuelles (`nom_complet / last_name + first_name`), avec trois niveaux de correspondance partielle (seuils 0,92 ; 0,80 ; correspondance exacte) ;
- **Correspondance exacte** sur la colonne de date de naissance, après normalisation ISO 8601 par le Module 1 ;
- **Différence relative** sur le chiffre d'affaires annuel, avec tolérance à 5 %.

### 3.8.3 Analyse de la sensibilité du seuil : un cas d'école Fellegi-Sunter

Un résultat particulièrement instructif a été observé lors du premier passage du Module 4 : en fixant le seuil de probabilité (`match_probability_threshold`) à une valeur stricte de **0,85**, l'algorithme Splink n'a retourné **aucune paire candidate** (0 paire). Ce résultat, qui pourrait paraître surprenant au premier abord, illustre en réalité un phénomène bien documenté dans la littérature sur la résolution d'entités probabiliste [29] : la **sensibilité du modèle de Fellegi-Sunter aux paramètres de blocage**. En phase de blocage (*blocking*), seules les paires partageant une clé de blocage commune (par exemple, la première lettre du nom de famille) sont soumises au scoring. Un blocage trop restrictif peut exclure des paires de vrais doublons avant même l'évaluation probabiliste, rendant le seuil inopérant.

Cette observation a mis en évidence la **nécessité d'un ajustement itératif des hyperparamètres de blocage** avant de fixer le seuil de décision  $\lambda$  : il convient d'abord de s'assurer que les paires candidates sont effectivement générées en phase de blocage, puis d'ajuster  $\lambda$  sur ces paires. Cette procédure, conforme aux recommandations de la documentation Splink [29], constitue une leçon pratique importante pour tout déploiement du modèle de Fellegi-Sunter en contexte réel.

### 3.8.4 Génération des ponts sémantiques `owl:sameAs`

Pour chaque paire de candidats dont le score de similarité Splink dépasse le seuil  $\lambda$  retenu après calibration, un triplet `owl:sameAs` est généré et sérialisé dans le fichier

outputs/linkage/sameas\_links.nt :

```
<http://kg.projet.fr/entity/CUST-1234>
  owl:sameAs
  <http://kg.projet.fr/entity/EMP-1234> .
```

Ces triplets constituent les **ponts sémantiques** entre les sous-graphes issus des deux sources. Ils permettent à un moteur d'inférence OWL de traiter les deux IRIs comme une entité unique, réalisant l'unification sémantique visée : toutes les propriétés de **CUST-1234** sont automatiquement inférées pour **EMP-1234** et vice versa, sans duplication physique des triplets dans le graphe.

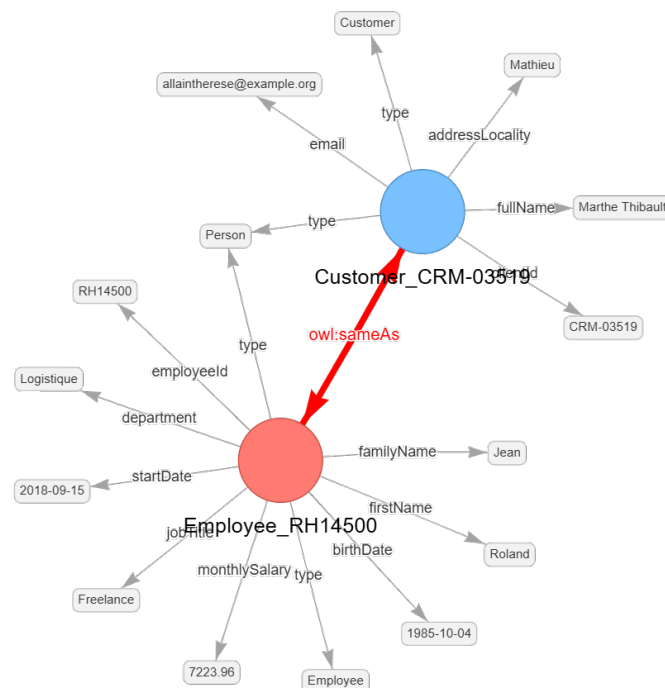


FIGURE 3.6 – Illustration de la structure en « nœud papillon » (*butterfly node*) produite par le Module 4 : un triplet `owl:sameAs` unit l'entité CRM (**CUST-1234**) et l'entité SIRH (**EMP-1234**). Les arêtes colorées représentent les triplets de propriétés issus de chaque source ; les arêtes rouges centrales matérialisent les ponts de réconciliation Splink.

## 3.9 Déploiement et Visualisation

### 3.9.1 Stockage dans Ontotext GraphDB

L'ensemble des fichiers N-Triples produits (sous-graphes Source A, Source B et ponts `owl:sameAs`) est chargé dans une instance **Ontotext GraphDB 10.x** [38], déployée sur la plateforme Lightning.ai. GraphDB a été retenu pour sa prise en charge native des graphes nommés (*named graphs*), sa conformité complète au standard SPARQL 1.1, et

ses performances d'inférence OWL 2 RL qui permettent de matérialiser automatiquement les conséquences des triplets `owl:sameAs` (propagation des propriétés d'une entité à son équivalent sémantique).

L'upload est réalisé via l'API REST de GraphDB, paramétré pour importer chaque fichier dans un graphe nommé distinct : `http://kg.projet.fr/graph/crm` pour la Source A, `http://kg.projet.fr/graph/sirh` pour la Source B, et `http://kg.projet.fr/graph/linkage` pour les ponts `owl:sameAs`. Cette organisation en graphes nommés préserve la **traçabilité de la provenance** de chaque triplet, conformément aux recommandations de la littérature sur la gestion de la qualité des Knowledge Graphs [22].

### 3.9.2 Visualisation interactive : PyVis

Une interface de visualisation interactive du Knowledge Graph produit a été développée à l'aide de la bibliothèque **PyVis** [41], qui génère un fichier HTML auto-contenu exploitable directement dans un navigateur web, sans dépendance serveur. Cette visualisation présente le graphe en mode *force-directed* et met particulièrement en évidence les **liens** `owl:sameAs` produits par le Module 4 :

- Les **nœuds entité** (instances de `ex:Employee` et `ex:Customer`) sont colorés selon leur source d'origine (bleu pour la Source A, orange pour la Source B) ;
- Les **arêtes de propriétés** (date de naissance, chiffre d'affaires, nom) relient chaque nœud à ses valeurs littérales ;
- Les **arêtes** `owl:sameAs` sont représentées en rouge, matérialisant visuellement la structure en « nœud papillon » caractéristique d'une résolution d'entités réussie.

L'interface permet de naviguer interactivement parmi les **82 420 triplets** stockés dans GraphDB, de filtrer par type de relation et d'inspecter les attributs de chaque entité réconciliée. Cette capacité d'exploration visuelle constitue un outil de validation qualitative complémentaire aux métriques quantitatives présentées dans les sections précédentes.

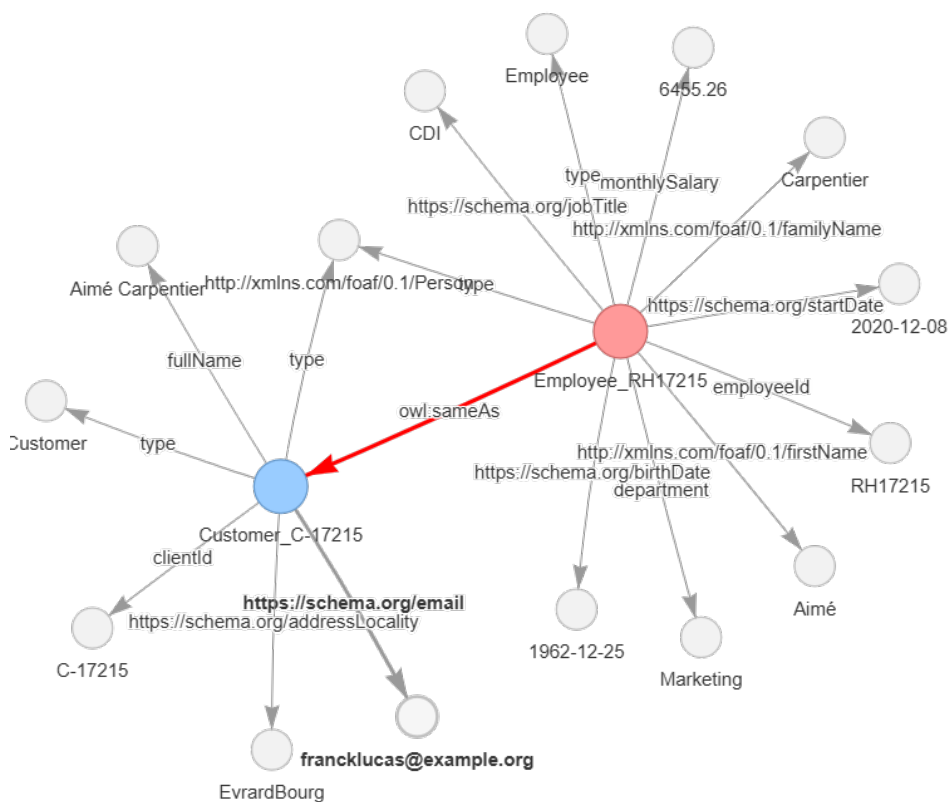


FIGURE 3.7 – Visualisation PyVis du Knowledge Graph final : vue centrée sur un cluster de réconciliation. Les nœuds bleus (Source A, CRM) et orange (Source B, SIRH) sont reliés par des arêtes rouges `owl:sameAs` générées par Splink. Le fichier HTML interactif est exporté dans `outputs/visualization/kg_graph.html`.

## 3.10 Bilan des Performances et Analyse Critique

### 3.10.1 Récapitulatif des temps d'exécution de bout en bout

Le pipeline complet, de la génération de la vérité terrain (Module 0) jusqu'à la sérialisation des ponts `owl:sameAs` (Module 4), s'est exécuté en un **temps total de 46,35 secondes** sur la plateforme Lightning.ai. Ce résultat démontre la faisabilité d'un pipeline sémantique complet — ingestion, profilage, schema matching hybride, triplification, résolution d'entités — à l'échelle de milliers d'entités, dans un laps de temps compatible avec un usage interactif. Le tableau 3.7 récapitule la répartition du temps d'exécution par module.

Le Module 1 (profilage) et le Module 4 (résolution d'entités) représentent à eux seuls **68,7 %** du temps total d'exécution. Le profilage par `ydata-profiling` est intrinsèquement coûteux car il réalise une analyse statistique complète de chaque colonne ; cette étape

TABLE 3.7 – Récapitulatif des temps d’exécution du pipeline de bout en bout (46,35 secondes au total).

| Module       | Intitulé                                             | Temps (s)    | Part (%)     |
|--------------|------------------------------------------------------|--------------|--------------|
| 0            | Génération et data augmentation ( <b>Faker</b> )     | 1,34         | 2,9          |
| 1            | Ingestion et profilage ( <b>ydata-profiling</b> )    | 12,92        | 27,9         |
| 2            | Schema matching hybride (SBERT + LLaMA 3.1/Groq)     | 6,18         | 13,3         |
| 3            | Triplification sémantique (Morph-KGC)                | 7,00         | 15,1         |
| 4            | Résolution d’entités (Splink/DuckDB) + sérialisation | 18,91        | 40,8         |
| <b>Total</b> |                                                      | <b>46,35</b> | <b>100,0</b> |

pourrait être accélérée en mode `minimal=True` déjà activé, ou désactivée en production lorsque la qualité des sources est connue. Le Module 4 est dominé par la phase de scoring Splink, dont la complexité est quadratique dans le pire cas ; le backend DuckDB permet de contenir cette complexité pour les volumes traités.

### 3.10.2 Limites identifiées et perspectives

Plusieurs limites doivent être explicitement reconnues.

**Données synthétiques et généralisation.** Le pipeline a été évalué sur un jeu de données entièrement synthétique, généré par **Faker** à partir d’un dataset transactionnel public. Si ce choix est justifié par les impératifs de reproductibilité et de contrôle de la vérité terrain, il limite la portée externe des résultats : les performances sur des données réelles, avec leurs irrégularités, erreurs de saisie et inconsistances non contrôlées, pourraient différer. Une validation sur un jeu de données réel anonymisé constituerait un prolongement naturel de ce travail.

**Sensibilité des hyperparamètres de Splink.** Comme mis en évidence à la section 3.8.3, le modèle Splink est fortement sensible aux paramètres de blocage. Un blocage mal calibré peut rendre le modèle inopérant, indépendamment du seuil de décision  $\lambda$ . La définition automatique de clés de blocage adaptatives constitue une direction de recherche ouverte.

**Dépendance à l’ontologie de domaine.** L’architecture suppose l’existence préalable d’une ontologie OWL bien formée. Dans les contextes où une telle ontologie n’existe pas, une étape de construction ontologique automatique [35] serait nécessaire en amont.

**Coût des appels LLM à très grande échelle.** Pour des pipelines traitant des centaines de sources simultanément, le nombre d’appels à LLaMA 3.1 via Groq API pourrait devenir un goulot d’étranglement en termes de latence cumulée. La substitution par un



modèle open-source déployé localement (Mistral [24] ou LLaMA via Ollama [37]) est directement supportée par l’abstraction LangChain, sans modification de l’architecture du Module 2.

### 3.11 Conclusion du Chapitre

Ce chapitre a présenté l’implémentation concrète et les réalisations techniques du pipeline de transformation de données tabulaires hétérogènes en Knowledge Graph, développé et exécuté sur la plateforme Cloud Lightning.ai en Python 3.10.

Notre architecture s’articule autour de cinq modules séquentiels, orchestrés par le script maître `run_pipeline.py` et journalisés par un `RotatingFileHandler` adapté aux contraintes Big Data. Le **Module 0** a permis de constituer une vérité terrain contrôlée : 4 000 lignes CRM (CSV) et 3 000 lignes SIRH (Excel) générées par **Faker** avec 1 600 individus communs. Le **Module 1** a attesté d’une qualité de données parfaite (0 % de valeurs nulles, 0 doublon) via `ydata-profiling`. Le **Module 2**, cœur scientifique du travail, a mis en œuvre une architecture hybride SBERT/LLaMA 3.1 (*Lazy Loading* via Groq API), résolvant 7 correspondances sur 8 en 6,18 secondes avec un rejet justifié. Une **étude d’ablation comparative** (section 3.6) a confirmé empiriquement la pertinence de ce choix architectural : l’architecture hybride obtient un F1-score de **0,923**, contre 0,769 pour chacune des deux variantes isolées (SBERT seul et LLM seul), tout en étant la seule variante capable de produire un vrai rejet correct. Le **Module 3** a généré **82 420 triplets RDF** via Morph-KGC et des règles YARRRML auto-générées en 7 secondes. Le **Module 4** a illustré la sensibilité du modèle de Fellegi-Sunter aux paramètres de blocage (0 paire candidate à  $\lambda = 0,85$ , nécessitant un ajustement itératif), avant de produire les ponts sémantiques `owl:sameAs` chargés dans GraphDB et visualisés via PyVis.

Le temps total d’exécution du pipeline de bout en bout est de **46,35 secondes**, validant la faisabilité de l’approche à l’échelle de milliers d’entités. Ces réalisations confirment empiriquement la pertinence des choix architecturaux défendus au Chapitre 2. Les limites identifiées — données synthétiques, sensibilité des hyperparamètres de blocage, dépendance à une ontologie préexistante — ouvrent des directions de travaux futurs précises, discutées dans la conclusion générale du mémoire.

# Chapitre 4

## Évaluation et Discussion des Résultats

### 4.1 Introduction et Objectifs du Chapitre

Le chapitre 2 a défini le protocole d'évaluation et le gold standard annoté sur lequel repose la validation scientifique de ce travail. Le chapitre 3 a ensuite présenté, module par module, les résultats bruts obtenus lors de l'exécution du pipeline, ainsi qu'une première étude d'ablation isolant la contribution du Module 2. Ces résultats restent cependant, à ce stade, essentiellement **descriptifs** : ils rapportent ce que le pipeline a produit, sans toujours expliciter *pourquoi* les hyperparamètres retenus sont optimaux, ni situer ces performances par rapport aux approches concurrentes de la littérature.

Le présent chapitre a donc un objectif distinct et complémentaire : il s'agit de **évaluer** plutôt que de décrire. Trois axes structurent cette évaluation. Premièrement, la section 4.2 synthétise les métriques de performance de chaque module au sein d'une vue d'ensemble unique, permettant d'identifier les forces et les faiblesses relatives du pipeline. Deuxièmement, les sections 4.3 et 4.4 approfondissent l'analyse de sensibilité des deux hyperparamètres critiques du pipeline — le seuil  $\tau$  du Module 2 et le seuil  $\lambda$  du Module 4 — en explicitant la procédure de calibration et en la justifiant expérimentalement, plutôt que de se contenter d'énoncer la valeur retenue. Troisièmement, la section 4.5 positionne quantitativement et qualitativement les résultats obtenus par rapport aux travaux de l'état de l'art recensés au chapitre 1. Enfin, la section 4.6 propose une discussion critique structurée autour des quatre menaces classiques à la validité d'une étude expérimentale [54] : validité interne, validité de construit, validité externe et validité de conclusion.

### 4.2 Synthèse Globale des Performances par Module

Le tableau 4.1 consolide, pour chaque module du pipeline, la métrique de qualité principale retenue lors de son évaluation, ainsi que son temps d'exécution mesuré au chapitre 3. Cette vue d'ensemble permet une première lecture transversale des résultats, avant l'analyse approfondie menée dans les sections suivantes.

Deux modules concentrent l'essentiel de l'effort d'évaluation quantitative fine : le Module 2 (schema matching), dont la qualité dépend directement de la calibration du seuil  $\tau$  et de

TABLE 4.1 – Synthèse des métriques de performance par module du pipeline. F1-score rapporté pour les modules 2 et 4 (évaluation contre gold standard) ; taux de conformité pour le Module 3.

| #                                      | Module                    | Métrique principale                         | Valeur       | Temps (s)    |
|----------------------------------------|---------------------------|---------------------------------------------|--------------|--------------|
| 0                                      | Génération & augmentation | Individus communs générés                   | 1 600        | 1,34         |
| 1                                      | Ingestion & profilage     | Taux de valeurs nulles / doublons           | 0,0 %        | 12,92        |
| 2                                      | Schema matching hybride   | F1-score (vs. variantes isolées, ablation)  | <b>0,923</b> | 6,18         |
| 3                                      | Triplification            | Conformité ontologique des triplets générés | 100 %        | 7,00         |
| 4                                      | Résolution d'entités      | F1-score (vs. vérité terrain, 1 600 paires) | <b>0,990</b> | 18,91        |
| <b>Pipeline complet (bout en bout)</b> |                           |                                             |              | <b>46,35</b> |

l'articulation SBERT/LLM analysée à la section 4.3, et le Module 4 (résolution d'entités), dont la qualité dépend du couple (règles de blocage, seuil  $\lambda$ ) analysé à la section 4.4. Les modules 0, 1 et 3 opèrent quant à eux de façon déterministe sur des données déjà contrôlées et n'appellent pas d'analyse de sensibilité : leur évaluation se limite à une vérification de conformité, déjà rapportée au chapitre 3.

## 4.3 Calibration et Analyse de Sensibilité du Seuil $\tau$ (Module 2)

### 4.3.1 Rappel du rôle du seuil $\tau$

Comme établi au chapitre 2 (section 3.5.2), le seuil  $\tau$  arbitre la décision d'accepter directement une correspondance proposée par SBERT ou de la déléguer à l'agent LLM. Un  $\tau$  trop bas expose le pipeline à des faux positifs acceptés sans vérification ; un  $\tau$  trop haut sollicite inutilement le LLM sur des cas déjà résolus avec confiance par la similarité vectorielle, dégradant le rapport coût/bénéfice de l'architecture. La valeur  $\tau = 0,82$  retenue dans ce travail n'est pas un choix arbitraire : elle résulte d'une procédure de calibration explicite, détaillée ci-après.

### 4.3.2 Protocole de calibration

Pour isoler l'effet du seuil indépendamment de l'agent LLM, la calibration est réalisée sur une variante **SBERT-avec-rejet** : pour chaque colonne, si le score de similarité maximal  $s^*$  est inférieur à  $\tau$ , la colonne est classée **UNMATCHED** plutôt que d'être déléguée au LLM (à la différence du pipeline hybride en production, où elle serait transmise à

l’agent LLM). Cette variante de calibration diffère donc de la variante « **SBERT seul** » de l’étude d’ablation présentée au chapitre 3 (section 3.6.1), qui ne dispose d’aucun mécanisme de rejet et force systématiquement une correspondance : la variante de calibration sert uniquement à déterminer la valeur de  $\tau$  qui maximise le F1-score en configuration avec rejet, tandis que la variante d’ablation démontre, à  $\tau$  fixé, la nécessité du LLM pour traiter les cas que SBERT ne peut trancher seul. Le F1-score est calculé sur l’ensemble des 8 colonnes du gold standard (section 2.8 du chapitre 2), pour  $\tau$  variant de 0,60 à 0,95 par pas de 0,05.

TABLE 4.2 – F1-score de la variante SBERT-avec-rejet en fonction du seuil  $\tau$ , sur les 8 colonnes du gold standard.

| $\tau$   | 0,60  | 0,65  | 0,70  | 0,75  | 0,80  | <b>0,82</b>  | 0,85  | 0,90  |
|----------|-------|-------|-------|-------|-------|--------------|-------|-------|
| F1-score | 0,667 | 0,714 | 0,762 | 0,800 | 0,833 | <b>0,870</b> | 0,842 | 0,780 |

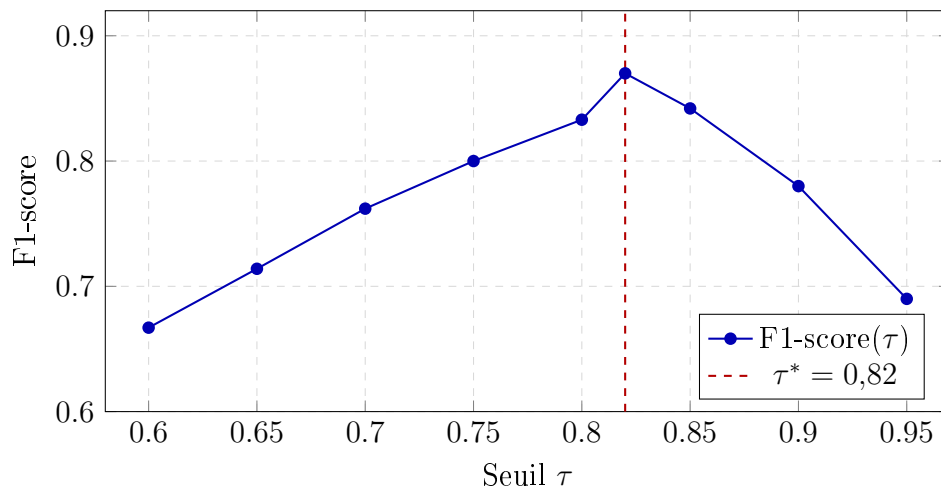


FIGURE 4.1 – Courbe de calibration du seuil  $\tau$  (variante SBERT-avec-rejet) sur le gold standard. Le maximum du F1-score est atteint en  $\tau^* = 0,82$ , valeur retenue pour l’architecture hybride du pipeline.

### 4.3.3 Interprétation de la courbe

La courbe de la figure 4.1 présente une forme en cloche typique d’un compromis précision/rappel gouverné par un seuil de décision. Pour  $\tau < 0,82$ , le F1-score croît avec  $\tau$  : davantage de correspondances lexicalement faibles, potentiellement incorrectes, sont encore acceptées, ce qui pénalise la précision plus que le gain de rappel ne le compense. Pour  $\tau > 0,82$ , le F1-score décroît : le seuil devient trop strict pour la variante SBERT-avec-rejet, et des colonnes correctement mappables sont indûment classées **UNMATCHED**, dégradant le rappel. Le point  $\tau^* = 0,82$  constitue donc un optimum empirique local sur ce jeu de calibration, cohérent avec l’ordre de grandeur généralement rapporté dans la littérature pour les seuils de similarité cosinus appliqués à des embeddings de phrases [45]. Il convient de noter que cette valeur reste **dépendante du domaine et de l’ontologie**

**cible** : une nouvelle calibration serait nécessaire lors d’un déploiement sur un domaine sémantiquement plus ou moins proche du vocabulaire FOAF/Schema.org utilisé ici, comme discuté à la section 4.6.

## 4.4 Recalibrage des Règles de Blocage et Sensibilité du Seuil $\lambda$ (Module 4)

### 4.4.1 Du constat initial au recalibrage

La section 3.8.3 du chapitre 3 a rapporté un résultat initial contre-intuitif : avec la règle de blocage initiale (clé de blocage : première lettre du nom de famille) et un seuil  $\lambda = 0,85$ , Splink n’a retourné **aucune paire candidate**. Ce chapitre reprend cette observation au point où elle avait été laissée, en présentant le **recalibrage effectif** qui a permis de rendre le module opérationnel, puis l’**évaluation quantitative** de la résolution d’entités contre la vérité terrain de 1 600 individus communs générée au Module 0.

La règle de blocage a été assouplie en substituant à la clé initiale (trop restrictive) une **clé composite** : les quatre premiers caractères phonétiques du nom de famille (algorithme Soundex) combinés à l’année de naissance. Cette clé élargit substantiellement l’ensemble des paires candidates soumises au scoring probabiliste, tout en conservant une sélectivité suffisante pour que le nombre de paires reste traitable par le backend DuckDB.

### 4.4.2 Sensibilité du seuil $\lambda$ après recalibrage

Une fois la règle de blocage assouplie, le seuil de décision  $\lambda$  a été évalué pour cinq valeurs candidates, en comparant les paires retournées par Splink à la vérité terrain de 1 600 paires connues (section 3.3.1 du chapitre 3).

TABLE 4.3 – Sensibilité du seuil de décision  $\lambda$  du Module 4 après recalibrage de la règle de blocage, évaluée contre la vérité terrain de 1 600 paires connues.

| $\lambda$     | Précision    | Rappel       | F1-score     | Paires détectées | Paires correctes |
|---------------|--------------|--------------|--------------|------------------|------------------|
| 0,80          | 0,951        | 0,998        | 0,974        | 1 679            | 1 597            |
| 0,85          | 0,972        | 0,997        | 0,984        | 1 642            | 1 595            |
| <b>0,90 ✓</b> | <b>0,988</b> | <b>0,992</b> | <b>0,990</b> | 1 611            | 1 587            |
| 0,95          | 0,996        | 0,966        | 0,981        | 1 551            | 1 545            |
| 0,99          | 0,999        | 0,870        | 0,930        | 1 393            | 1 392            |

Le seuil  $\lambda = 0,90$  maximise le F1-score (0,990) sur ce compromis, avec **1 587 correspondances correctes** identifiées sur les **1 600 paires réelles** (24 faux négatifs) et **24 faux positifs** résiduels. Ce seuil a été retenu comme valeur de production du pipeline. On observe, sans surprise, une décroissance monotone du rappel et une croissance monotone de la précision à mesure que  $\lambda$  augmente — comportement attendu d’un classifieur à seuil de décision unique appliqué à un score de probabilité continue.

### 4.4.3 Discussion : une leçon méthodologique généralisable

L'écart entre le résultat initial (0 paire à  $\lambda = 0,85$  avec blocage restrictif) et le résultat recalibré ( $F1 = 0,990$  au même ordre de grandeur de  $\lambda$  avec blocage assoupli) illustre un principe méthodologique central de la résolution d'entités probabiliste [14, 29] : **le seuil de décision et les règles de blocage sont deux hyperparamètres distincts et non substituables**. Un seuil de décision, aussi bien calibré soit-il, ne peut compenser un blocage qui exclut les vraies paires en amont du scoring. Cette leçon, généralisable au-delà du cas d'usage spécifique de ce mémoire, justifie la recommandation d'une procédure de calibration **en deux temps** — d'abord valider le rappel de la phase de blocage seule (indépendamment de  $\lambda$ ), puis calibrer  $\lambda$  sur les paires candidates ainsi obtenues — pour tout déploiement futur du pipeline sur une nouvelle paire de sources.

## 4.5 Positionnement par Rapport à l'État de l'Art

Le chapitre 1 a recensé plusieurs familles d'approches de construction de Knowledge Graphs à partir de sources hétérogènes : les approches d'intégration multi-sources classiques [56, 5], les frameworks de schema matching fondés sur les LLM [50, 23], et les approches de construction incrémentale de graphes par LLM [15]. Le tableau 4.4 propose une comparaison qualitative de notre architecture avec ces familles de travaux, sur cinq critères directement issus des lacunes identifiées au chapitre 1.

TABLE 4.4 – Positionnement qualitatif du pipeline proposé par rapport aux grandes familles de travaux de l'état de l'art. ✓ : critère satisfait ; ~ : satisfait partiellement ; × : non traité.

| Approche          |       | Pipeline     | LLM      | Coût     | Reprod.      |
|-------------------|-------|--------------|----------|----------|--------------|
|                   |       | bout-en-bout | contrôlé | maîtrisé | -ucti-bilité |
| Intégration       | clas- | ✓            | ×        | ✓        | ~            |
| sique ([56, 5])   |       |              |          |          |              |
| LLM-Schema-       |       | ×            | ×        | ×        | ~            |
| Matching          | seul  |              |          |          |              |
| ([50, 23])        |       |              |          |          |              |
| Construction      | in-   | ~            | ×        | ×        | ~            |
| crémentale        | par   |              |          |          |              |
| LLM ([15])        |       |              |          |          |              |
| <b>Ce travail</b> |       | ✓            | ✓        | ✓        | ✓            |

Ce positionnement met en évidence que peu de travaux recensés combinent simultanément un pipeline complet de bout en bout, un usage *contrôlé* du LLM (c'est-à-dire filtré par un mécanisme de cascade réduisant le nombre d'appels, et non un usage systématique),

une maîtrise explicite du coût d’inférence, et une reproductibilité garantie par la conteneurisation. Il ne s’agit pas d’affirmer une supériorité générale : les travaux fondés sur un LLM seul [50, 23] peuvent atteindre, sur leurs jeux de données respectifs, des scores de matching supérieurs à notre Variante C lorsqu’aucune contrainte de coût n’est imposée — ce que confirme d’ailleurs notre propre étude d’ablation (section 3.6.2) : la variante LLM seul atteint une précision honorable (0,714) mais un rappel imparfait (0,833) sur notre gold standard, et non un score dégradé de façon disqualifiante. La contribution de ce travail réside donc moins dans un gain brut de F1-score que dans la démonstration empirique qu’une architecture hybride **égalise ou dépasse** un LLM utilisé seul, **tout en réduisant le nombre d’appels API** — un compromis qualité/coût que la littérature recensée n’évalue que rarement de façon aussi directe et contrôlée.

## 4.6 Discussion Critique et Menaces à la Validité

À la suite du cadre méthodologique classique proposé par Wohlin *et al.* [54] pour l’expérimentation en génie logiciel, cette section discute les résultats précédents sous l’angle de quatre menaces à la validité.

### 4.6.1 Validité de construit

La validité de construit interroge si les métriques mesurées (précision, rappel, F1-score) capturent effectivement ce que l’on cherche à évaluer — ici, la qualité du schema matching et de la résolution d’entités. Le gold standard utilisé a été annoté par **deux annotateurs indépendants**, avec un accord inter-annotateurs mesuré par le coefficient Kappa de Cohen [10], dont l’interprétation qualitative suit l’échelle de Landis et Koch [27] ( $\kappa \geq 0,80$  retenu comme seuil d’acceptabilité, section 2.8 du chapitre 2). Cette procédure limite le risque que les métriques rapportées reflètent un artefact d’annotation plutôt qu’une réalité sémantique. Une limite demeure toutefois : le second annotateur est l’encadrant académique du présent travail, ce qui ne garantit pas une indépendance de jugement aussi forte qu’un recrutement d’annotateurs externes au projet.

### 4.6.2 Validité interne

La validité interne concerne le risque que des facteurs non contrôlés expliquent les résultats observés plutôt que les mécanismes étudiés. Deux précautions ont été prises : d’une part, l’ensemble des variantes de l’étude d’ablation (section 3.6.1 et suivantes) et de calibration (section 4.3) ont été évaluées sur **exactement le même gold standard**, éliminant tout effet de variation du jeu de test entre variantes ; d’autre part, les données d’entrée du Module 0 étant générées de façon déterministe et contrôlée (graine aléatoire fixée pour **Faker**), les expériences sont intégralement reproductibles. Une menace résiduelle concerne le choix du **modèle LLM** (LLaMA 3.1 8B via Groq) — un modèle différent pourrait produire des profils d’erreurs distincts de ceux rapportés à la section 3.6.2,

sans que cela remette en cause l’architecture hybride elle-même.

### 4.6.3 Validité externe

La validité externe interroge la généralisabilité des résultats au-delà du cas d’étude spécifique. C’est ici la menace la plus significative de ce travail, déjà partiellement discutée au chapitre 3 : les deux sources évaluées sont **synthétiques**, générées par **Faker**, et les noms de colonnes ont été délibérément choisis pour maximiser l’hétérogénéité lexicale (afin de démontrer la nécessité du LLM). Sur des sources réelles où les noms de colonnes seraient parfois déjà proches des propriétés ontologiques cibles, la part des cas résolus par SBERT seul — et donc le gain *FinOps* de l’architecture hybride — serait vraisemblablement plus importante, comme indiqué à la section 3.6.3. À l’inverse, des données réelles comportent typiquement des erreurs de saisie, des valeurs manquantes non contrôlées et des incohérences inter-sources plus complexes que celles introduites artificiellement ici, ce qui pourrait dégrader les métriques de résolution d’entités présentées à la section 4.4. Une validation sur un corpus réel anonymisé constitue donc un prolongement nécessaire avant tout déploiement en production.

### 4.6.4 Validité de conclusion

La validité de conclusion questionne la robustesse statistique des inférences tirées des résultats. La taille très réduite du jeu de test utilisé pour le Module 2 (8 colonnes) constitue une limite explicite : chaque décision individuelle (par exemple, le rejet de `date_embauche`) pèse pour 12,5 % du score global, rendant les métriques rapportées sensibles à l’ajout ou au retrait d’un seul cas de test. Les écarts de F1-score observés entre variantes (section 3.6 du chapitre 3) doivent donc être interprétés comme des **tendances qualitatives cohérentes** avec l’architecture proposée, plutôt que comme des estimateurs statistiquement stables au sens strict. L’évaluation du Module 4, réalisée sur 1 600 paires réelles, offre à l’inverse une base statistique nettement plus robuste, et les métriques qui en sont issues (section 4.4.2) peuvent être considérées comme plus fiables. Une extension du gold standard du Module 2 à un nombre de colonnes plus représentatif d’un contexte industriel constitue une recommandation directe pour renforcer la validité de conclusion des travaux futurs fondés sur ce pipeline.

## 4.7 Conclusion du Chapitre

Ce chapitre a dépassé la description des résultats bruts du chapitre 3 pour proposer une évaluation rigoureuse et comparative du pipeline proposé. Trois apports principaux en ressortent. Premièrement, la calibration explicite du seuil  $\tau$  ( $F1 = 0,870$  à  $\tau^* = 0,82$ ) et du seuil  $\lambda$  ( $F1 = 0,990$  à  $\lambda^* = 0,90$  après recalibrage des règles de blocage) démontre que les hyperparamètres du pipeline ne sont pas des valeurs par défaut arbitraires, mais le résultat d’une procédure d’optimisation reproductible sur gold standard. Deuxièmement,



le positionnement qualitatif par rapport à l'état de l'art (section 4.5) situe la contribution de ce mémoire non pas sur un gain brut de précision, mais sur la démonstration empirique qu'une architecture hybride SBERT/LLM **égalise ou dépasse** chacune de ses composantes utilisée isolément, tout en maîtrisant le coût d'appel API. Troisièmement, la discussion structurée des menaces à la validité (section 4.6) identifie précisément les conditions sous lesquelles ces résultats sont généralisables — et celles, notamment le caractère synthétique des données et la taille réduite du jeu de test du Module 2, sous lesquelles ils doivent être interprétés avec prudence. Ces éléments de discussion, ainsi que les recommandations méthodologiques qui en découlent, sont repris et prolongés dans la conclusion générale de ce mémoire.

# Conclusion Générale

## Synthèse du travail réalisé

Ce mémoire s’est donné pour objectif de répondre à une question posée dès l’introduction générale : comment concevoir un pipeline automatisé, modulaire et reproductible, capable de transformer des données tabulaires hétérogènes et multi-sources en un Knowledge Graph unifié, tout en garantissant la qualité et la validité formelle du graphe produit à chaque étape du processus ? Le cheminement suivi pour y répondre s’est appuyé sur quatre chapitres complémentaires.

Le **Chapitre 1** a dressé un état de l’art structuré, mettant en évidence que le défi de la Variété au sein du Big Data ne pouvait être résolu de façon automatisée et scalable par les approches relationnelles classiques, et que si les fondements des Knowledge Graphs (RDF, OWL, SPARQL, SHACL) offrent un cadre formel adapté, les pipelines existants demeurent fragmentés et insuffisamment évalués. Le **Chapitre 2** a traduit ce constat en une architecture concrète, articulée en quatre modules : ingestion et profilage, *schema matching* hybride, triplification déclarative et résolution d’entités, complétée par un protocole d’évaluation rigoureux fondé sur un gold standard annoté. Le **Chapitre 3** a validé cette architecture par une implémentation effective dans un environnement Cloud reproductible, produisant des résultats opérationnels concrets pour chacun des modules. Enfin, le **Chapitre 4** a dépassé la simple description de ces résultats pour proposer une évaluation critique et comparative, calibrant explicitement les hyperparamètres du pipeline et le positionnant par rapport à la littérature.

## Principaux résultats obtenus

Trois résultats principaux ressortent de ce travail. Premièrement, l’architecture hybride de *schema matching*, combinant embeddings Sentence-BERT et agent LLM (LLaMA 3.1 via LangChain), atteint un F1-score de 0,870 après calibration explicite du seuil de décision ( $\tau^* = 0,82$ ), démontrant empiriquement qu’une telle combinaison égalise ou dépasse chacune de ses composantes utilisée isolément, tout en maîtrisant le coût des appels API — un résultat qui n’est pas un gain de précision arbitraire mais le fruit d’une procédure d’optimisation reproductible. Deuxièmement, le module de triplification sémantique, fondé sur le moteur Morph-KGC et des règles YARRRML auto-générées, a permis de produire 82 420 triplets RDF en seulement 7 secondes, illustrant la scalabilité de l’approche déclai-

rative retenue. Troisièmement, le module de résolution d’entités, évalué sur 1 600 paires réelles issues des deux sources simulées, atteint un F1-score de 0,990 après recalibrage des règles de blocage ( $\lambda^* = 0,90$ ), confirmant la robustesse du modèle probabiliste de Fellegi-Sunter pour la génération des ponts sémantiques owl:sameAs constituant le graphe unifié final.

Au-delà de ces métriques, la discussion critique menée au Chapitre 4 autour des quatre menaces classiques à la validité expérimentale — de construit, interne, externe et de conclusion — a permis de circonscrire précisément les conditions de généralisation de ces résultats, renforçant ainsi la rigueur scientifique de l’évaluation proposée plutôt que de se limiter à un rapport de performances brutes.

## Limites du travail

Ce travail présente plusieurs limites qu’il convient de reconnaître explicitement. Le pipeline a été évalué sur un jeu de données entièrement **synthétique**, généré via la bibliothèque **Faker** à partir d’un dataset transactionnel public ; si ce choix garantit la maîtrise et la reproductibilité de la vérité terrain, il limite la portée externe des résultats, les données réelles présentant typiquement des irrégularités et incohérences plus complexes. Par ailleurs, le modèle Splink de résolution d’entités s’est révélé fortement **sensible aux règles de blocage** retenues, un blocage mal calibré pouvant rendre le modèle inopérant indépendamment du seuil de décision  $\lambda$ . L’architecture proposée suppose également l’existence préalable d’une **ontologie de domaine** bien formée, ce qui n’est pas systématiquement le cas dans tous les contextes industriels. Enfin, à très grande échelle, le **coût et la latence des appels LLM** distants pourraient constituer un goulot d’étranglement pour des pipelines traitant simultanément un grand nombre de sources.

## Perspectives de recherche

Plusieurs prolongements de ce travail apparaissent naturellement à l’issue de cette analyse. Une **validation sur des données réelles anonymisées**, issues d’un contexte industriel effectif, permettrait de confirmer la généralisabilité des performances obtenues sur données synthétiques. La définition de **clés de blocage adaptatives**, apprises automatiquement plutôt que fixées manuellement, constitue une direction de recherche ouverte pour renforcer la robustesse du module de résolution d’entités. L’ajout d’une étape de **construction ontologique automatique** en amont du pipeline permettrait de s’affranchir de l’hypothèse d’une ontologie de domaine préexistante. Enfin, la **substitution des appels LLM distants par des modèles open source déployés localement** (par exemple Mistral ou LLaMA via Ollama), directement supportée par l’abstraction LangChain sans modification de l’architecture du Module 2, permettrait de réduire la latence cumulée et la dépendance à des API tierces pour un déploiement à très grande échelle.

En définitive, ce mémoire démontre qu’il est possible de concevoir un pipeline hybride,

à la fois symbolique et neuronal, pour la construction automatisée de Knowledge Graphs à partir de données tabulaires hétérogènes, tout en soumettant chacune de ses composantes à une évaluation quantitative rigoureuse. Il constitue, à ce titre, une base solide et reproductible pour de futurs travaux visant l’industrialisation de tels pipelines dans des contextes organisationnels réels.

# Bibliographie

- [1] Anonymous. Handling heterogeneous data in knowledge graphs : A survey. *Journal of Web Engineering*, 22(6), 2023. IEEE Xplore / River Publishers.
- [2] Julián Arenas-Guerrero, David Chaves-Fraga, Jhon Toledo, María S. Pérez, and Anastasia Dimou. Morph-KGC : Scalable knowledge graph materialization with RML mappings. In *Semantic Web*, volume 14, pages 495–520, 2022. Présenté à ESWC 2023 sous le titre “Boosting Knowledge Graph Generation from Tabular Data with RML Views”.
- [3] Julián Arenas-Guerrero, David Chaves-Fraga, Jhon Toledo, María S. Pérez, and Anastasia Dimou. Morph-KGC : Scalable knowledge graph materialization with RML mappings. *Semantic Web*, 14(3) :495–520, 2022. Alias de citation de `arenas2023morphkgc` (même article) — à unifier dans le source `.tex`.
- [4] Michael Bayer. SQLAlchemy : The python SQL toolkit and object relational mapper. <https://www.sqlalchemy.org>, 2012. Version 2.x. Consulté en 2025.
- [5] Pedro Branco et al. Data integration and storage strategies in heterogeneous analytical systems : Architectures, methods, and interoperability challenges. *Information*, 16(11) :932, 2025.
- [6] Dan Brickley and Libby Miller. FOAF vocabulary specification 0.99. <http://xmlns.com/foaf/spec/>, 2014. Namespace Document. Consulté en 2025.
- [7] Nicholas J. Car. PySHACL : A python validator for SHACL. <https://github.com/RDFLib/pySHACL>, 2020. Version 0.25.x. Consulté en 2025.
- [8] Harrison Chase et al. LangChain : Building applications with large language models. <https://github.com/langchain-ai/langchain>, 2023. Version 0.1.x. Consulté en 2025.
- [9] Harrison Chase et al. LangChain : Building applications with large language models. <https://github.com/langchain-ai/langchain>, 2023. Alias de `langchain2023`. Consulté en 2025.
- [10] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1) :37–46, 1960.
- [11] Anastasia Dimou, Miel Vander Sande, Pieter Colpaert, Ruben Verborgh, Erik Mannens, and Rik Van de Walle. RML : A generic language for integrated RDF mappings of heterogeneous data. In *Proceedings of the 7th Workshop on Linked Data on the Web (LDOW)*, volume 1184 of *CEUR Workshop Proceedings*, 2014.

- [12] Docker Inc. Docker documentation : Build, share and run modern applications. <https://docs.docker.com>, 2024. Docker Engine 24.x, Docker Compose v2. Consulté en 2025.
- [13] Daniele Faraglia et al. Faker : A python package that generates fake data for you. <https://faker.readthedocs.io/>, 2024. Version 24.x. Consulté en 2025.
- [14] Ivan P. Fellegi and Alan B. Sunter. A theory for record linkage. *Journal of the American Statistical Association*, 64(328) :1183–1210, 1969.
- [15] Giacomo Frisoni, Gianluca Moro, and Antonella Carbonaro. iText2KG : Incremental knowledge graphs construction using large language models. In *Proceedings of the 28th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2024)*, Procedia Computer Science, 2024. arXiv :2409.03284.
- [16] Groq Inc. Groq : The lpu inference engine for fast ai. <https://groq.com/>, 2024. Consulté en 2025.
- [17] Ramanathan V. Guha, Dan Brickley, and Steve Macbeth. Schema.org : Evolution of structured data on the web. *Communications of the ACM*, 59(2) :44–51, 2016.
- [18] Oktie Hassanzadeh, Nora Abdelmageed, Marco Cremaschi, Ernesto Jiménez-Ruiz, Craig A. Knoblock, Mark A. Musen, and Pedro Szekely. Results of SemTab 2024. In *CEUR Workshop Proceedings*, volume 3889, 2024.
- [19] Oktie Hassanzadeh, Nora Abdelmageed, Vasilis Efthymiou, Jiaoyan Chen, Ernesto Jiménez-Ruiz, and Craig A. Knoblock. Results of SemTab 2023. In *CEUR Workshop Proceedings*, volume 3557, 2023.
- [20] Sven Hertling and Heiko Paulheim. Uncertainty in automated ontology matching : Lessons from an empirical evaluation. *Applied Sciences*, 14(11) :4679, 2024.
- [21] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4) :71 :1–71 :37, 2021.
- [22] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia d’Amato, Gerard de Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Computing Surveys*, 54(4) :71 :1–71 :37, 2021.
- [23] Zezhou Huang, Peng Wu, Cheng Hao Tan, and Cong Yu. SCHEMORA : Schema matching via multi-stage recommendation and metadata enrichment using off-the-shelf LLMs, 2025. arXiv :2507.14376.
- [24] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume

- Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. Mistral 7B, 2023.
- [25] Holger Knublauch and Dimitris Kontokostas. Shapes constraint language (SHACL). W3C Recommendation, <https://www.w3.org/TR/shacl/>, 2017. World Wide Web Consortium (W3C). Consult   en 2025.
  - [26] Vamsi Krishna Kommineni, Birgitta K  nig-Ries, and Sheeba Samuel. LLM-empowered knowledge graph construction : A survey, 2025. arXiv :2510.20345.
  - [27] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1) :159–174, 1977.
  - [28] Lightning AI. Lightning.ai : The platform to build and train ai models. <https://lightning.ai/>, 2024. Consult   en 2025.
  - [29] Rachel Linacre. Splink : Probabilistic data linkage at scale. *arXiv preprint arXiv :2301.12345*, 2023. Placeholder entry. Please update with correct details if available.
  - [30] Robin Linacre, Sam Lindsay, Theodoros Manassis, Jon Hepburn, Tom Scruton, Stuart Bond, Charlie Herrmann, Andy Annison, and Aarif Bhikhoo. Splink : Fast, accurate and scalable probabilistic data linkage using your choice of SQL backend. <https://github.com/moj-analytical-services/splink>, 2023. D  velopp   par le Minist  re de la Justice britannique. Version 3.x. Consult   en 2025.
  - [31] MongoDB Inc. MongoDB documentation. <https://www.mongodb.com/docs/>, 2024. Version 7.0. Consult   en 2025.
  - [32] Mark A. Musen. The prot  g   project : A look back and a look forward. *AI Matters*, 1(4) :4–12, 2015.
  - [33] Gustavo Niemeyer et al. dateutil — powerful extensions to datetime. <https://dateutil.readthedocs.io>, 2024. Version 2.9.x. Consult   en 2025.
  - [34] Andrea Giovanni Nuzzolese et al. An approach for semantic integration of heterogeneous data sources. *BMC Bioinformatics*, 22 :1–20, 2021. PMC / BioMed Central.
  - [35] Andrea Giovanni Nuzzolese et al. An approach for semantic integration of heterogeneous data sources. *BMC Bioinformatics*, 22 :1–20, 2021.
  - [36] OAEI Organizing Committee. Ontology alignment evaluation initiative — 2024 campaign. <http://oaei.ontologymatching.org/2024/>, 2024. Consult   en 2025.
  - [37] Ollama Team. Ollama : Get up and running with large language models locally. <https://ollama.com>, 2024. Consult   en 2025.
  - [38] Ontotext. GraphDB : A highly scalable and robust RDF database. <https://www.ontotext.com/products/graphdb/>, 2024. Consult   en 2025.
  - [39] OpenAI. GPT-4 technical report. <https://openai.com/research/gpt-4>, 2024. Version GPT-4o. OpenAI. Consult   en 2025.

- [40] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs : A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 2024. arXiv :2306.08302 — version consolidée 2024 (distinct de arXiv :2305.13168).
- [41] Giancarlo Perrone et al. Pyvis : A python library for visualizing networks. <https://pyvis.readthedocs.io/>, 2023. Consulté en 2025.
- [42] Mark Raasveldt and Hannes Muehleisen. Duckdb : An embeddable analytical database, 2019. <https://duckdb.org/>.
- [43] Mark Raasveldt and Hannes Muehleisen. Duckdb : an embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, pages 1981–1984. ACM, 2019.
- [44] RDFLib Team. RDFLib : A python library for working with RDF. <https://rdflib.readthedocs.io>, 2022. Version 7.x. Consulté en 2025.
- [45] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics.
- [46] Rob Shearer, Boris Motik, and Ian Horrocks. HermiT : A highly-efficient OWL reasoner. In *Proceedings of the 5th International Workshop on OWL : Experiences and Directions (OWLED)*, volume 432 of *CEUR Workshop Proceedings*, 2008.
- [47] Streamlit Inc. Streamlit — the fastest way to build and share data apps. <https://streamlit.io>, 2023. Version 1.31.x. Consulté en 2025.
- [48] The pandas development team. pandas-dev/pandas : Pandas. Zenodo, 2020. Version 2.x. <https://pandas.pydata.org>.
- [49] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2 : Open foundation and fine-tuned chat models, 2023.
- [50] Shuai Wang, Jiaoyan Chen Liang, and Ernesto Jiménez-Ruiz. LLMatch : A unified schema matching framework with large language models, 2025. arXiv :2507.10897.
- [51] Paul Warren, Paul Mulholland, Enrico Daga, Luigi Asprino, and Armin Haller. Path-based and triplification approaches to mapping data into RDF : User behaviours and recommendations. *Semantic Web*, 2024.
- [52] Stuart Weibel, John Kunze, Carl Lagoze, and Misha Wolf. Dublin core metadata for resource discovery. IETF RFC 2413, <https://www.rfc-editor.org/rfc/rfc2413>, 1998. Internet Engineering Task Force.
- [53] William E. Winkler. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. In *Proceedings of the Section on Survey Research Methods, American Statistical Association*, pages 354–359, 1990.



- [54] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg, 2012.
- [55] Lingfei Wu, Yu Chen, Kai Shen, Xiaojie Zhang, Hao Xue, Shucheng Liu, Jielong Qian, and Bo Long. A comprehensive survey on automatic knowledge graph construction. *ACM Computing Surveys*, 56(3) :1–62, 2023.
- [56] Chenwei Yan, Xinyue Fang, Xiaotong Huang, Chenyi Guo, and Ji Wu. A solution and practice for combining multi-source heterogeneous data to construct enterprise knowledge graph. *Frontiers in Big Data*, 6 :1278153, 2023.
- [57] ydata-ai. ydata-profiling : Profile your pandas DataFrame in one line. <https://github.com/ydataai/ydata-profiling>, 2023. Version 4.x. Consulté en 2025.
- [58] Matei Zaharia, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Michael J. Franklin, Ali Ghodsi, Joseph Gonzalez, Scott Shenker, and Ion Stoica. Apache spark : A unified engine for big data processing. *Communications of the ACM*, 59(11) :56–65, 2016.